

A mature Development and Deployment Process

How to sleep well when your 200 developers
(and AI agents) deploy apps every day?

Version 1.8 – 2026-05-29

Author
Sascha Wildgrube

Table of Contents

Table of Contents	2
Disclaimer	7
License	7
Change Log	8
Introduction	11
Who should read this Document?	11
Purpose of this Document	11
The Problem – Increasing Complexity and Speed	12
The Solution - From Update Sets to Source Control.....	13
Back to Baseline.....	15
Terminology	16
A note on Open-Source Applications	16
An honest debate about pros and cons.....	17
10 Things You Can Do NOW To Get Started	18
Process Overview.....	19
Quality Gates	21
Implementation: Backlog Item is “Ready”	21
Implementation: Backlog Item is “Done”	22
Application Delivery: Application Version is “Baselined”	23
Test and Production Deployment: Release Package is “In Test”	24
Test and Production Deployment: Release Package is “Ready for Production”	25
Test and Production Deployment: Release Package is “Deployed to Production”	26
Process Roles	27
Platform Owner.....	27
Developers	27
Lead Developers	27
Architects.....	27
Deployers.....	27
Operations.....	27
Testers.....	27
Governance Stakeholders	27
Governance	28
Platform Owner.....	28
Demand.....	28

Product Owners	28
Development	28
Operations and Deployment	29
Data Quality	29
User Experience	29
Security	29
Artificial Intelligence	29
Testing.....	30
Doing the right thing and doing things right	30
Intentions – why are we testing?	31
Scopes – what are we testing?	31
Methods – how are we testing?	32
Tools – what do we use for testing?	33
Test Suites that yield Value over Time	34
Test Data	35
Test Users	36
Generative AI	37
AI and the Process – How does this fit?	37
Developing and Deploying AI-based Capabilities	37
AI augmented development.....	37
Test-Driven Code Generation	39
Developer Efficiency Gains	40
Building Blocks	41
Application Architecture and Dependency Management	41
Definition of Ready	45
Definition of Done	46
Coding Guideline	47
Automated Tests	49
Source Control System	51
Branching Strategy	53
Application Version Baseline Procedure	56
Deployment Automation.....	58
Technical Debt Management	59
Hotfixing and Backporting.....	61
Release Notes	64

Backlog Management	65
Instance Setups	68
Default	69
Balanced	69
Juggernaut	69
Making a Choice	70
Considerations for the (missing) CONTROLLER instance	71
Deployment Variants - Tankers, Speedboats and Lifeboats	72
Regular Releases (Tankers)	72
Interim Releases (Speedboats)	72
Hotfix Release (Lifeboats)	73
Rollbacks (Life Vests)	73
Techniques	75
Scoped and Global Applications – and App Customizations	75
Application Architecture Design	78
Application Naming Convention	80
Cross Scope Scripting	84
Installation Scripts	89
Instance Specific Configuration	91
Data vs. Code	93
Data as Code	96
Guidance on Script Includes	97
Managing Translations	98
Test-Driven Development	99
Semantic Versioning	102
Determining Minimum Version Requirements	104
Backporting	106
Managing Unfinished Work	108
Reducing Unfinished Work	110
Treat Evaluator Warnings as Errors	111
Defect Correction and Root Cause Analysis	112
Personas and Roles	113
Persona Access Matrix	116
Technical Users	117
Application Manuals and Release Notes	118

Managing Multiple Development Streams	119
Decommissioning Applications.....	122
Organizing Documentation	124
High Performance Background Jobs.....	126
Refactoring Customizations into Applications	128
DevTools.....	128
Four Tactics.....	129
Highlander Records.....	130
Copy-And-Deactivate-Approach	131
Hard and Soft Cross Scope Limitations	132
Record Transporter	133
Decision Tree	134
ACLs	135
Business Rules	136
Choices.....	137
Flows	137
Form and List Views	138
Script Includes	139
Service Portal Widgets	144
System Properties.....	145
Tables, Dictionary Entries, Dictionary Overrides and Attributes	146
UI Policies and UI Policy Actions.....	147
If All Else Fails – Defaulting to Update Sets.....	148
Making the Change	149
Pills and Surgery vs. Life Change	149
Getting out of the Comfort Zone	149
Zero-Tolerance Mind-Set.....	150
The Process Evangelist	150
Aligning the Organization and Technology	151
Refactoring existing Code and Customizations into Applications.....	152
Developer Onboarding	154
Communities of Practice	155
Moderating Community of Practice Meetings.....	156
Proof-Of-Concept – Pilot – Production Readiness.....	157
Backlog.....	158

Education Program	161
Daily Stand-Up Meetings	164
Platform Capabilities	165
Classic Studio	165
Source Control Integration (Studio)	165
Automated Testing Framework	165
Instance Scan	166
Impact Health / Scan Engine.....	167
Impact Health / Proactive Code Check.....	167
Build Agent	168
ServiceNow Studio.....	168
ServiceNow IDE	168
Sandboxes.....	168
Open-Source Applications	169
DevTools.....	169
Deployer	169
CodeSanity.....	169
DataSanity	169
Runbook.....	169
Agile	169
ArtificialDeveloper	170
R-IDE	170
Installing or Updating open-source Applications from GitHub	171
Maintaining Open-Source Applications	174
Risk Assessment	176
Memorandum of Understanding.....	177
Undo Open-Source	178
Family Upgrades	181
Impact on Family Upgrades.....	181
Procedure for Family Upgrades	182
Cloning.....	183
Do you still need to clone?.....	183
Activities when Cloning	183
Maturity Model.....	184

Disclaimer

This document is NOT released or published by ServiceNow. It does in no way express official ServiceNow's opinions nor shall any of its contents be interpreted as a statement made by ServiceNow.

This document expresses the individual opinions of its author which are based on observations made and work conducted with and for a limited number of ServiceNow customers.

Neither the author nor ServiceNow shall be liable for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising from following the guidance contained in this document.

This version of this document should be considered as work in progress. It is knowingly incomplete and subject to change and refinement.

License

Copyright 2023-2026 by Sascha Wildgrube

Licensed under the Apache License, Version 2.0 (the "License")

You may not use this document except in compliance with the License.

You may obtain a copy of the License at: <https://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, work distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Change Log

1.8 – 2026-05-29

- Updated the title to “How to sleep well when your 200 developers (and AI agents) deploy apps every day?”
- Updated chapter “Purpose of this Document”
- Updated and renamed chapter “The Problem – Increasing Complexity and Speed”
- Updated chapter “The Solution - From Update Sets to Source Control”
- Updated chapter “Source Control System” to explain how to respond to the OAuth requirement
- Added chapter “Application Naming Convention”
- Added chapter “Impact Health / Scan Engine”
Updated chapter “Impact Health / Proactive Code Check”
- Added chapter “Build Agent”
- Updated chapter “ServiceNow Studio”
- Updated chapter “ServiceNow IDE”

1.7 – 2025-12-09

- Added the citation of the “Hock Principle” to chapter “Backlog Management”
- Added the terms “Runbook”, “Downstream Instance” and “Upstream Instance” to chapter “Terminology”
- Added chapter “High Performance Background Jobs”
- Updated chapter “Copy-And-Deactivate-Approach”
- Added chapter “Flows”
- Added chapter “Script Includes”
- Added chapter “Service Portal Widgets”
- Added chapter “UI Policies and UI Policy Actions”
- Added chapter “If All Else Fails – Defaulting to Update Sets”
- Updated chapter “Procedure for Family Upgrades”
- Updated chapter “Maturity Model” to contain the new dimension “AI for development”

1.6 – 2025-07-18

- Added chapter “An honest debate about pros and cons”
- Added chapter “Guidance on Script Includes”
- Added chapter “Determining Minimum Version Requirements”
- Added chapter “Defect Correction and Root Cause Analysis”
- Added chapter “Refactoring Customizations into Applications” including specific guidance for some Application File types
- Added chapter “Impact Health / Proactive Code Check”
- Added chapter “Risk Assessment”
- Added chapter “Memorandum of Understanding”
- Added chapter “Undo Open-Source”

- Updated chapter "Backlog" to include the "Nightly QA" (a nightly run of all ATFs and Instance Scan checks for all apps)
- Updated chapter "Installing or Updating open-source Applications from GitHub" since the url in a sys_repo_config record cannot be changed in the GUI
- Removed chapter "What to expect from this Document" as it was redundant and merged some of its content into chapter "Purpose of this Document"
- Modified various chapters for clarity

1.5 – 2025-02-14

- Added chapter "Proof-Of-Concept – Pilot – Production Readiness"
- Updated chapter "Source Control System" to list steps for accessing the Source Control System
- Updated chapter "Backlog" to consider the PoC, Pilot and Production Readiness phases
- Changed chapter "Education Program": Improved the expert topics and introduction session topics and updated terminology: The "Homework App" is now the "Masterpiece" and "Learning Groups" are now "Study Groups".

1.4 – 2024-12-24

- Added chapter "Back to Baseline"
- Added chapter "Managing Multiple Development Streams"
- Added chapter "The Process Evangelist"
- Added chapter "Education Program"
- Added chapter "Daily Stand-Up Meetings" to make a clear statement AGAINST doing them
- Updated chapter "Open-Source Applications" to reference the R-IDE application
- Chapter "Handling Unfinished Work" has been renamed to "Managing Unfinished Work"
- Removed chapter "Self-Organized Learning"

1.3 – 2024-11-11

- Added chapter "Generative AI"
- Added chapter "Maintaining Open-Source Applications"
- Updated chapter "Governance" to add the new governance stakeholder for Artificial Intelligence
- Updated chapter "Open-Source Applications" to reference the ArtificialDeveloper application
- Updated chapter "Application Architecture and Dependency Management" to describe why it is necessary to refactor applications if circular dependencies are identified
- Updated chapter "Application Version Baseline Procedure" to reflect the new capability of DevTools and Deployer to protect branches automatically
- Updated chapter "Deployment Automation" to mention that ATF tests are only executed on non-production instances
- Updated chapter "Cross Scope Scripting" for Xanadu compatibility

- Updated chapter "Moderating Community of Practice Meetings": Replaced the term "Backlog" by "Next Action" for disambiguation
- Updated chapter "Platform Capabilities" to reference "Source Control Integration", "ServiceNow Studio", "ServiceNow IDE" and "Sandboxes"
- Updated chapter "Maturity Model" to add the dimension of governance maturity to the model
- The term "CICD Instance" is replaced by the term "CONTROLLER Instance" for consistency and clarity

1.2 - 2024-08-07

- Updated chapter "Personas and Roles" to include guidance that persona roles are best defined in the platform app
- Added chapter "Decommissioning Applications"
- Added chapter "Persona Access Matrix"
- Added chapter "Deployment Variants - Tankers, Speedboats and Lifeboats"
- Added chapter "Maturity Model"

1.1 - 2024-06-28

- Added chapter "Considerations for the (missing) CICD instance"
- Added chapter "Aligning the Organization and Technology" which discusses what we can learn from Conway's Law to make this process work
- Added chapter on "Refactoring existing Code and Customizations into Applications"
- Added data quality management aspects to various existing chapters and added reference to the DataSanity app
- Updated chapter "Installing or Updating open-source applications from GitHub" because the "Baseline Application" button (provided by DevTools) should not be used when ingesting an open-source application into the organization's source control system.
- Updated chapter "Personas and Roles" to clarify that personas should be designed with the production environment in mind
- Improved chapter "Test Driven Development" (e.g., referring to the "CodeSanity" app and its Instance Scan checks that support test coverage)

1.0 - 2024-04-15

The first version tracked in the change log.

0.1 – 2023-05-05

The first published version.

Introduction

Who should read this Document?

This document is for

- Platform Owners
- Architects
- Development Team Leaders
- Developers
- Everyone who wants to modernize their development and deployment process
- Everyone who consults and guides organizations on the topic
- Anyone who suffers from sleep deprivation caused by deployments that have gone terribly wrong

Purpose of this Document

This document aims to provide specific guidance on how to introduce and establish a robust, reliable, fast, AI augmented, and automated development and deployment process.

It contains guidance on the process, its building blocks, the techniques required to make it work on a technical level, the technical capabilities required (which go beyond OOTB), a proposed training schedule and on how to introduce the process into the organization.

Two of the main building blocks of the process are based on automated quality control – the verification against the Coding Guideline and the use of Automated Tests are instrumental. These and all other testing activities are part of quality control activities, which are described in detail in chapter “Testing”.

This document proposes three different variants of instance setups – and further guidance on how to choose from these variants.

The readers can use it as the foundation for a transformation project that takes the organization from where it is today towards a path of process maturity – for new and long existing ServiceNow instances.

The document also describes practical advice on how to introduce the process and to manage organizational change and enablement.

It should be noted however that the proposed process is ONE way to solve a specific set of problems. It is not the ONLY way.

With that being stated, the purpose of this document is not only to help organizations to solve problems but also to inspire and contribute to the discussion on how application development and deployment should be supported in the ServiceNow Platform in the future.

The Problem – Increasing Complexity and Speed

The reasons why you are reading this might vary. Maybe there are specific problems to be addressed, maybe there is just a foggy sense of things just do not work out as they should and maybe you are just interested.

Here is a list of observations the organization might have made:

- An increasing output of implementations due to the use of AI coding agents
- Long time-to-market / rare and huge releases
- Many roundtrips between test and development
- Time-consuming manual tests
- As more customizations are introduced over time, side-effects are more difficult to foresee – resulting in less accurate estimates despite increasing team experience
- Difficulties in responding to last-minute Product Owner decisions
- Difficulty to track the exact status of “story” delivery
- Inconsistency between development, test, and production instances
- Deployments don't produce the exact state in test environments as delivered (or intended) by the development team
- Deployments don't produce the exact state in production as was previously tested
- Incident-driven fixes (in production) don't make it back into development – and reappear after subsequent releases
- Difficulty with compliance requirements
- Talented and experienced team members leave and seek other opportunities quickly

The last bullet point is a tricky one. The observation might have been made, but the link to the existing development and deployment process might not yet have been established. Someone leaving the team has always multiple reasons and it is impossible to blame one single factor. The author of this document can't provide any evidence of causality but believes that process maturity is a factor in staff acquisition and retention.

No matter how bleak the situation may appear and how distraught the team and their management may be – this document provides specific guidance beyond the trite recommendations we all read and hear so often.

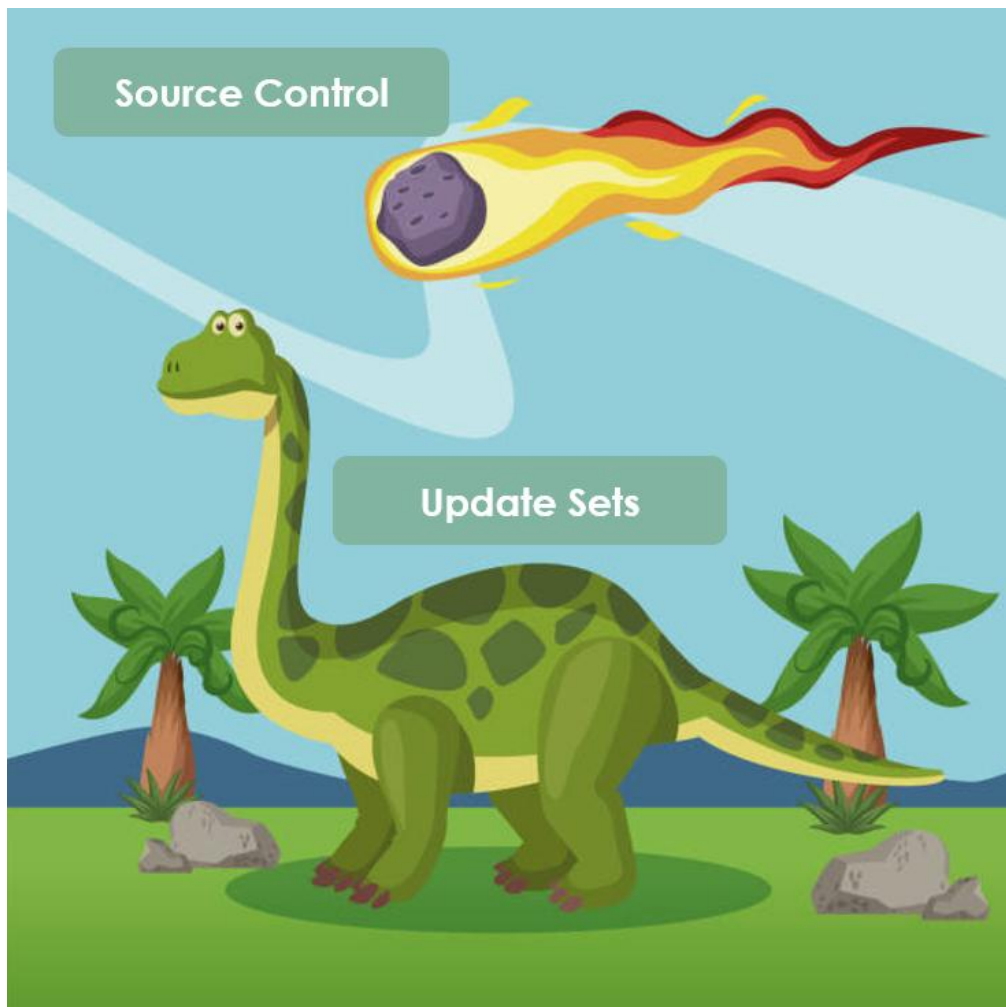
AI has finally arrived in mainstream software development work. This does not only increase productivity - this also requires an even more robust, fast and adjustable QA and deployment process.

The guidance contained in this document has never been more relevant and its implementation has never been so urgent as of the current day.

The Solution - From Update Sets to Source Control

The Paris release was a gamechanger. Source-control based application installations became delta-aware. This means, that if an application already exists on an instance and a new version of that same application is installed through source-control, existing database tables and fields are preserved. Prior to Paris, an application was completely uninstalled, and all data deleted, then the new version was installed – which was ok for sub-production (if a proper test data creation process was in place) – but obviously not ok for production.

Paris pushed the door wide open for source-control-based deployments on production.



If ServiceNow was a planet, the Paris release was to Update Sets, what the meteor hitting earth about 66 million years ago was to dinosaurs – an extinction event.

Current scientific research indicates that the dinosaurs did not die from the impact itself – the process took decades – and so Update Sets are still in action in many if not most ServiceNow instances.

The technical capability alone doesn't drive change. Many people, with their belief systems, their routines, their habits, and comfort zones are inclined to ignore these opportunities as long as they can.

The solution proposed in this document may appear radical. It requires unlearning quite a few things, it requires adopting new habits, learning new techniques.

The three guiding principles:

1. Always deploy applications – instead of selected records captured in Update Sets
2. Zero-tolerance mind-set regarding failed tests or Instance Scan findings
3. Application-centric architecture instead of individual customizations

If you cannot buy into these principles personally, or you believe that the organization you are working for will never ever adopt these principles, you might just stop reading here. In that case, the guidance you will find in this document is not for you or your organization.

The idea to ship everything as applications has been subject to criticism in the past – it was considered by many as an approach that is too far away from standard ServiceNow practices.

With the new ServiceNow IDE, SDK and AI Build Agent getting more traction and slowly but steadily moving into the ServiceNow mainstream – these critics have finally been proven wrong.

Whether you like it or not, applications everywhere are the new normal.

The implementation of the guidance provided in this document yet requires adamant action despite a daunting task – to change people's habits. That must not be underestimated.

Back to Baseline

"Insanity is doing the same thing over and over again and expecting different results."

Albert Einstein

Many have tried to solve problems by "going back to baseline". Many observations indeed may lead to the conclusion that going back to baseline may be a good idea.

Do some of these observations sound familiar?

- The performance of the instance gets worse
- The pace at which new requirements are implemented and rolled out slows down
- Adoption of new platform capabilities slows down or is prevented because no one can make sense of the "skipped records" any more and new platform features require updating OOTB records that have been customized
- Each roll-out produces more regression defects
- The team complains about "too many customizations" which are difficult to handle, and no one really knows how the different pieces work together

It appears straight forward to blame the sheer number of customizations as the root cause of the situation. And it is more than tempting to conclude that starting all over again with a clean slate would fix the situation.

The situation may sound familiar, and you may even have participated in a back to baseline project in the past.

If that is the case, the following aspects should sound familiar, too:

- Initially everyone agreed that going back to baseline is a good idea, but once process stakeholders realized that this implies changing their processes, they opposed or even derailed the project
- Not understanding the existing customizations in the first place led to uninformed and sometimes problematic decisions on what to transfer on the greenfield instance and what to throw away
- After the conclusion of the greenfield project the instance experienced heavy customization again after a few years

Let us face the ugly truth: The idea of solving problems by going back to baseline is a myth.

And Einstein might have said: I told you so.

The root cause of these observations is not the customization itself, nor their complexity, nor their amount. Hence the solution to the problem cannot be to wash them away.

Going back to baseline may conceal the underlying problem for a while before the same observations will re-surface. And all of that at a terrible cost: Going back to baseline projects are expensive – mainly due to the implicit process re-engineering that they imply – which is often not anticipated while planning.

Following Einstein's insight, going back to baseline is downright insane – if the process remains the same. The solution to the problem is to fix the process.

This is what this document is all about.

Terminology

A few notes on terminology used in this document:

- “Application” refers to a scoped or global application represented by a sys_app record in the ServiceNow instance
- “Backlog Item” is a statement of demand, usually represented by a single record in a ticket system (e.g., a “Jira issue”, a “Story”, a card on a Kanban board or something similar)
- “Release Package” is a specific set of application versions to be deployed on a target instance including any additional information required to perform the deployment
- “Dependency” refers to the relationship between two applications where one application depends on another when the first application in any form or shape needs the other application to function or even to be installed in the first place.
- “Baselining” is the process of creating a version from the dev branch of an application at a specific point in time. This “Baselined Application Version” will then remain static and unchanged forever.
- “Runbook” refers to the operations handbook or operations documentation. It take the form of a document, a set of pages in Confluence or Knowledge Base Articles or an auto-generated page directly on the platform.
It is NOT the checklist, or the step-by-step instruction set for a deployment.
- “Downstream Instance” refers to an instance that follows suite in the pipeline. E.g. the TEST instance is a downstream instance of DEV. The PROD instance is a downstream instance of TEST. Applications follow the pipeline stream down to PROD.
- “Upstream Instance” refers to an instance that precedes an instance in the pipeline. E.g. the DEV instance is an upstream instance of TEST. The TEST instance is an upstream instance of PROD.

A note on Open-Source Applications

Some of the proposed techniques require technical capabilities that are not available OOTB as of Australia. There are some open-source applications available that can be used to fill in the gaps.

References to these open-source applications are made in various chapters in this document.

The process and the techniques described in this document can be implemented and adopted with or without using the open-source applications referenced in this document. These applications have been developed to support the process and the techniques and re-implementing comparable capabilities on top of the platform will require significant time and effort.

Of course, it is at the organization's discretion whether these applications are installed and used or extended or re-implemented or used as inspiration.

A key element to the process is pipeline automation. The Deployer open-source application supports a fully automated deployment process. Countless developer hours and real-world deployment experience made it what it is today. A make-or-copy-or-buy decision should not be made unless the Deployer app has been evaluated.

An honest debate about pros and cons

The guidance in this document is radical. It may lead to radical efficiency, radical improvement of delivery quality, radical speed.

It may also lead to revolt, delays, fear, or project failure.

It worked for some clients. It does not work for all.

Before an organization chooses to go down that path, an honest debate about the pros and cons must be held – and conscious decisions must be made.

PROs	CONs
Deployments are fully automated – leading to less mistakes in the process and faster time-to-market.	Adopting the guidance in this document requires a new way of working. Some (experienced) ServiceNow developers and admis may not be willing to support it.
Code and implementation quality is enforced with a zero-tolerance mind-set.	The rigorous zero-tolerance approach forces developers to invest more in quality early in the process and hence may slow down teams during adoption.
All code will be stored in an organization's source control system – making it easier (in some cases even possible) to meet compliance and regulatory requirements (e.g. NIS2).	Using source control in the ServiceNow context is not commonplace and requires specific skills and experience that many teams don't have (yet).
Open-Source apps are free of charge and published under the Apache 2.0 license which gives everyone the right to use and change them without any restriction.	Open-Source apps must be fully adopted and maintained by the organization's team. This implies effort and cost to learn and maintain knowledge. No one provides a support plan for them.
	Following the guidance implies deviating from official ServiceNow guidance – and hence may lead to ServiceNow or partners consultants to reject supporting it.
Family Upgrades become faster and easier to manage – through the copy-and-deactivate approach.	This document suggests copying OOTB records instead of modifying the original – a clear deviation from ServiceNow traditional guidance.
Cloning becomes obsolete in the long run.	Following the guidance that code must not depend on data is sometimes tricky and prevents using "traditional" configuration options, like setting up an Email Notification for a specific group etc. This can make implementing "simple" things more difficult.

10 Things You Can Do NOW To Get Started

We all love lists. They provide a sense of order and manage expectations. This list can help teams to get started adopting the ideas and techniques described in this document.

This list, however, does not replace the laborious work on the details. It is an entry point. Not a short cut:

1. Create a first scoped application
If you did not do it ever before, go to “Studio” and create a new scoped application.
2. Connect an application to Source Control
Reach out to the team that maintains your in-house source control system (or get yourself a GitHub account) and link your scoped application to a source control repository
3. Install the Deployer App
Navigate to <https://www.wildgrube.com/servicenow-deployer> and follow the installation instructions. Set up a pipeline and perform your first fully automated deployment.
4. Write the first Automated Test
Create a Test Suite and create a first automated test.
5. Explore Instance Scan
Make yourself familiar with Instance Scan - ServiceNow's static code analysis tool.
6. Start a Coding Guideline with the CodeSanity App
Navigate to <https://www.wildgrube.com/servicenow-codesanity> and follow the installation instructions. Then start checking your applications using the “CodeSanity” Instance Scan suite.
7. Define your App Architecture
Draw a potential architecture design where each box represents an application. Which team would then be responsible for which app?
8. Organize a CI/CD Workshop
Plan for two or three days with the complete team to gather their insights, come up with a clear and precise problem description and then evaluate how the guidance in this document can solve the problem. Define a backlog of action items.
9. Setup the Community of Practice
Organize a recurring meeting with the development team where everyone is heard and decisions on the development process, coding practices, and deployment automation are made and documented.
10. Share this Whitepaper with your Team
Take this document and share it within your organization to get the conversation started.

Process Overview

As for every process, it requires three factors to play well together: people, technology, and organization.

In the following chapters we will provide a high-altitude overview of the process, describe in detail what the building blocks for the process are, which quality gates and procedures need to be put in place and which techniques the development teams need to learn and apply.

Further, we will outline an approach to introduce the process into an existing or new organization.

When thinking in terms of a process, the most obvious question is: what represents one single “instance” of the process? (The process “Instance” should not be confused with a ServiceNow “Instance” in this context).

The process ranges from end to end: from the documentation of an idea or demand stated by a Product Owner representing the business to the moment in time when a new or changed application capability can be used in production.

The process has 3 phases:

1. Implementation
2. Application Delivery
3. Test and Production Deployment

The process “instance” is a different one in each of the phases:

1. During “Implementation”:
A single backlog item
2. During “Application Delivery”:
An application version
(which embodies changes resulting from one or more backlog items)
3. During “Test and Production Deployment”:
A release package
(which is a combination of one or more application versions and instructions on how to install them on a target instance)

The process has 6 quality gates:

During Implementation

- Backlog item is “Ready”
- Backlog item is “Done”

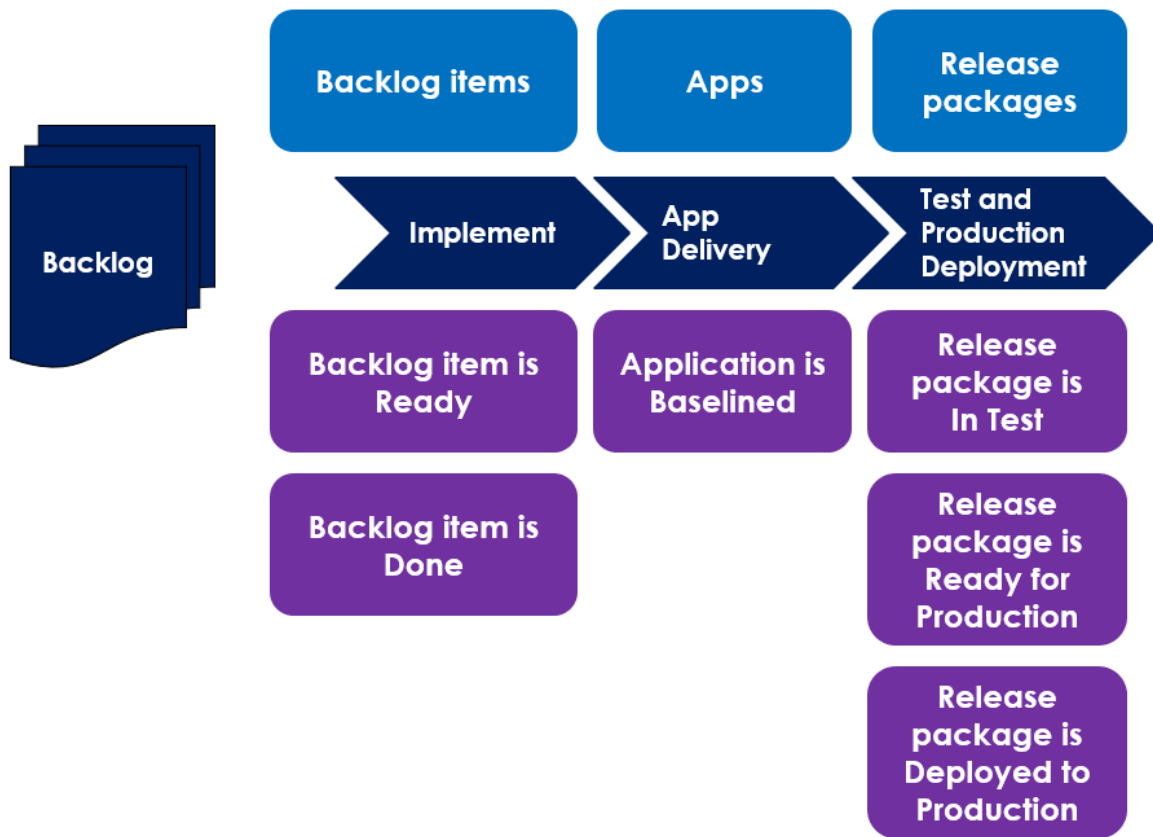
During Application Delivery

- Application version is “Baselined”

Test and Production Deployment

- Release package is “In Test”
- Release package is “Ready for Production”
- Release package is “Deployed to Production”

The following chapter provides more insights into the quality gates and what requirements must be met to pass them.



Process Overview with phases, instances, and quality gates

Quality Gates

Implementation: Backlog Item is “Ready”

To pass this quality gate a backlog item must be sufficiently documented, so that the development team can start working on it.

Sufficient documentation must meet all criteria defined in the “Definition of Ready”. However, not only must the “user story” be told, but there are also architectural decisions to be made.

Which applications need to be created or changed? Which dependencies must be considered? Should existing functionality be changed or new components be created?

Another important aspect is the relevant personas, which ideally map roles and access rights.

As always, a fair balance needs to be found between excessive precision upfront and flexibility and creativity in the process. Asking these questions is never wrong – but in some cases they may get answered later in the process.

The question whether any kind of estimate (in hours, story points, or t-shirt sizes) should be done before or after a backlog item is considered “Ready”. One can argue that an estimate is only possible IF the backlog item is ready. One may also argue that once enough information is available for an estimate, a requirement to include the result of an estimate into the definition of ready right away. As many organizations develop their own agile style, this document does not make any suggestion when estimation takes place or even if estimations should be done at all – although the author leans toward avoiding tactical estimates at all.

Checklist

- The documentation of the backlog item is compliant with the Definition of Ready
- The impacted applications have been identified and documented
- The expected dependencies have been identified and documented

Relevant Building Blocks

- Application Architecture and Dependency Management
- Definition of Ready

Gatekeepers

- Lead Developers
- Architects

Implementation: Backlog Item is “Done”

To pass this quality gate a backlog item must be completely implemented according to the requirements documented in the backlog item itself AND the additional requirements documented in the “Definition of Done” – which should specify that all tests must pass, and the delivered code is fully compliant with the Coding Guideline.

The completed code must be committed to the Source Code Repository.

“Done” does not mean that the capability described in the backlog item is available to users in production.

Checklist

- All activities defined in the Definition of Done are completed
- All related changes have been committed to the development branches of the impacted applications
- All related application files are compliant with the coding guideline

Relevant Building Blocks

- Definition of Done
- Coding Guideline
- Automated Tests
- Source Code Repository

Gatekeepers

- Lead Developers
- Product Owners

Application Delivery: Application Version is “Baselined”

To pass this quality gate an application version is available as a version specific branch (or tag) in the source code repository.

All activities described in the Application Version Baseline Procedure must be completed. That includes that any unfinished work is configured in a way that it does not impact the application's behavior and it is (like all other technical debts) documented as outlined in Technical Debt Management. The development team must have run all automated tests and verified their compliance with the Coding Guideline – verified through Instance Scan checks.

Checklist

- All activities defined in the application version baseline procedure have been performed
- The version branch is locked and protected against future changes

Relevant Building Blocks

- Application Version Baseline Procedure
- Technical Debt Management
- Source Code Repository
- Branching Strategy
- Coding Guideline
- Automated Tests

Gatekeepers

- Lead Developers
- Architects
- Product Owners

Test and Production Deployment: Release Package is “In Test”

To pass this quality gate a release package is successfully installed on a TEST environment.

To get there, a set of application versions – which typically relate to each other through their documented dependencies – must be identified. All steps required to install these application versions must be documented, reviewed, and performed in the TEST environment. As part of the deployment, all automated quality assurance activities should be conducted – ideally even before installing the release package on the TEST environment. These automated activities should at least include running all available and relevant ATF tests contained in the applications in the release package and performing coding guideline compliance checks using Instance Scan checks on these applications.

Checklist

- Deployment instructions do NOT contain update sets (all changes must be shipped as part of a baselined application version)
- Deployment instructions do NOT contain manual tasks – everything that needs to be “done” should be scripted in installation scripts
- All applications contained in the release package have been deployed to the TEST environment (in the required versions)
- All applications are fully compliant with the coding guideline – i.e., there are no Instance Scan findings
- All installation scripts of all applications have been executed
- All ATF tests contained in all applications have passed
- All steps defined in the deployment instructions have been performed – this may include activities to be performed in external systems or communication to stakeholders
- Smoke tests have passed – these are any quick manual tests

Relevant Building Blocks

- Source Code Repository
- Coding Guideline
- Automated Tests
- Deployment Automation

Gatekeepers

- Test Managers
- Operations

Test and Production Deployment: Release Package is “Ready for Production”

To pass this quality gate a release package is successfully tested on a TEST environment.

This document does not make any statement regarding manual testing – be it scripted or explorative – nor does it contain guidance on automated testing performed by tools outside ServiceNow. Such activities for sure play a significant role to verify quality and should be conducted with great care and craftsmanship.

If any of these tests fail, however, defects must be reported to the development team. The Defect Management and Backporting procedures specify in detail what – besides fixing the defect – needs to be done and where.

The most important aspect in this quality gate is that only if all tests pass, only then the release package in the tested form can be deployed to production and considered as “Ready for Production”.

If there is at least one show-stopping fail, the applications responsible must be fixed and provided as new versions. This constitutes a new release package, and the tests must start over.

Checklist

- All tests have passed OR all failed tests are not considered showstoppers
- All identified defects have been documented according to Defect Management and Backporting

Relevant Building Blocks

- Defect Management and Backporting

Gatekeepers

- Test Managers
- Operations

Test and Production Deployment: Release Package is “Deployed to Production”

To pass this quality gate a release package must be installed on the production instance. This may include making the newly introduced capability available to users, but it does not have to. In some cases, new capabilities are introduced in a silent go-live without visible impact on users.

Checklist

- All applications contained in the release package have been deployed to the production environment (in the required versions)
- All installation scripts of all applications have been executed
- All steps defined in the deployment instructions have been performed
- Smoke tests have passed

Relevant Building Blocks

- Source Code Repository
- Deployment Automation

Gatekeepers

- Operations
- Product Owners

Process Roles

The following roles are required to execute the proposed process. Multiple roles may be assigned to a single person, and some roles may be assigned to multiple individuals – depending on organization structure, culture, and team size.

Platform Owner

The platform owner is responsible for the platform and its operations. The platform owner should have decision making power with regards to staffing and budgets.

The platform owner is the highest escalation instance for any conflict that cannot be resolved by other team members.

Developers

Developers convert requirements documented in backlog items into new and changed applications.

Lead Developers

Lead Developers are responsible for baseline applications and to oversee the compliance to the Coding Guideline.

Architects

Architects research, advise and decide on solution approaches. They are responsible for the application architecture – they own the optimization problem to balance the amount, and size of applications and the reduction of dependencies between applications.

Deployers

Deployers conduct deployments of release packages to downstream instances. They own the process of deployment and its automation.

Operations

Operations are responsible for the day-to-day administration of the platform. In case of production incidents, they are responsible for providing solutions. They are responsible for monitoring the production platform and can respond to monitoring events. They perform configuration changes that are independent from deployments.

Testers

Testers confirm compliance of release packages to set expectations.

Governance Stakeholders

Governance stakeholders are responsible to define checks and procedures to be applied at the quality gates – and to provide insights into process quality from their area of expertise. They may also add backlog items to support this. Every governance stakeholder has a say in backlog item prioritization.

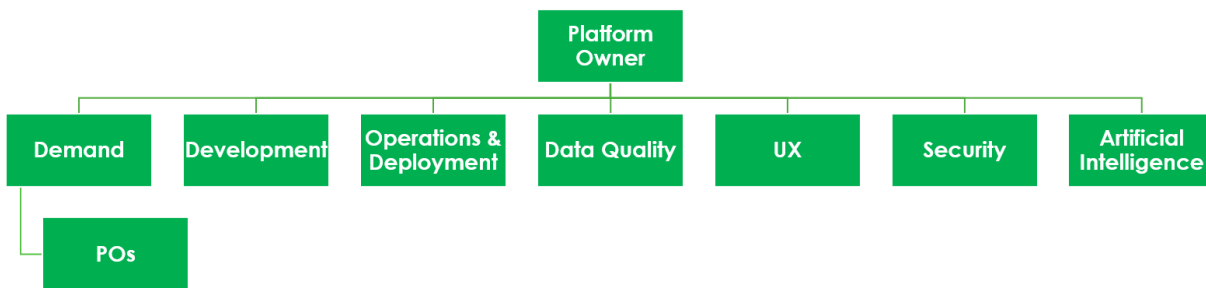
Refer to chapter “Governance” for details about the various governance stakeholder roles.

Governance

A ServiceNow platform requires an effective governance body. Of course, every organization has its own structure, staff, and organization for its governance body. This chapter outlines activities and responsibilities of the constituents of the governance body. Nine stakeholder “roles” are required to oversee the operations of a ServiceNow platform. How and by whom these roles are taken is for the organization to define.

Each of these stakeholders can add backlog items to the backlog and should have an equal say on how to prioritize these backlog items.

Each stakeholder can also define criteria by which the gatekeepers should consider and verify when making decisions and letting “process instances” pass their quality gate.



Platform Owner

As the name suggests the platform owner “owns” the platform. The platform owner has ultimate decision power and is the highest escalation instance for any conflict or decision to be made.

In the context of the governance body, the Platform Owner is responsible that each stakeholder role is staffed.

Demand

The “Demand” stakeholder is responsible for the identification and the prioritization of requirements and the approval and testing of new and changed capabilities of the platform. Usually the day-to-day work of identifying, prioritizing and documentation of requirements is delegated to one or more “Product Owners”.

Product Owners

“Product Owners” are responsible to identify, document and prioritize requirements related to a defined area of the platform or one or more business or IT processes.

Development

The Development stakeholder is responsible for the output quality of the work of all developers. They own the Coding Guideline and those automated tests that are created and maintained by the development team.

Typically, the Development stakeholder demands criteria for the documentation of requirements before they are worked on by the development team. These criteria are documented in the Definition of Ready.

Operations and Deployment

The Operations and Deployment stakeholder is accountable for all activities that administrators perform on all instances, and the deployment of applications to non-development instances.

The main concern of the stakeholder is operational stability. A key element is that what is delivered by the development team is deployed to target instances exactly as intended. Only then can testers verify shipped release packages and confirm that they are fit for purpose and that these can be deployed to production.

Data Quality

The “Data Quality” stakeholder is accountable for correct and complete data on the production instance. They also oversee that any data that should no longer be present on the production instance is archived and removed.

Each table of the ServiceNow platform that contains data (as defined in chapter “Data vs. Code”) should be associated to a stakeholder. It is unlikely that one person can manage that responsibility alone – so this stakeholder role should be shared between multiple stakeholders depending on the data.

Data Quality stakeholders may demand new or updated Instance Scan checks to be developed so that data quality can be monitored and managed on the production instance.

User Experience

The “User Experience” stakeholder’s main responsibility is to ensure that features are relevant, useable, and accessible to users. This stakeholder may have a say in the way new requirements are identified and described and how the user interfaces for these new capabilities are designed.

Once made available to users these capabilities can be checked through various means – from user interviews to statistical analysis of usage behavior. The “User Experience” stakeholder oversees these activities.

User satisfaction is their main concern – as adoption is driven by user experience and the feeling that an application is providing value.

Security

The “Security” stakeholder oversees all activities targeted to improve security on the platform. This starts with proper documentation of requirements considering what specific persona can or must not be able to do or access. It involves automated and manual testing activities and corresponding monitoring in production.

Security stakeholders may demand new or updated Instance Scan checks to be developed so that secure configuration can be monitored and managed on the production instance.

The Security Stakeholder should also oversee which data from production may be transferred to upstream instances for testing purposes.

Artificial Intelligence

The “Artificial Intelligence” stakeholder is responsible for all AI-based activities on the platform – in development and production. They oversee, analyze and monitor the results produced by AI.

Testing

Testing is a fundamental part of the process. Every single quality gate, building block, and every technique has some kind of testing built into it. This chapter provides an overview of the terminology and the different intentions (why we test), scopes (what we test), methods (how we test), and tools used for testing.

Testing is so important that it is covered by a separate chapter – it is a binding element across all aspects of the process.

For a team to communicate effectively about testing, the terminology must be aligned. This chapter aims to provide a common language for testing.

Doing the right thing and doing things right

Validation is to verify that we are doing the right thing.

Verification is to verify we do things right.

These are different aspects of testing, they require different approaches, different techniques and take place at different times.

Validation is about creativity, empathy, understanding, anticipation, investigation – it is part of a design process that never ends.

Verification is about precision, details, the happy path and the potential errors, the thinkable edge cases, the nitty gritty stuff, looking under every stone.

Verification is – for the good or the bad – agnostic to whether what has been built is easy-to-use or makes sense from a business perspective. Verification is about making sure things work as described and designed.

It is validation that helps a team to understand whether the intended outcome is likely to be achieved, whether users will understand, adopt, and make the best use of provided technical capabilities.

This distinction has implications on the approach, tools, and techniques – and eventually on the personnel that conducts the corresponding tests.

In an ideal world, verification is done BEFORE a backlog item is considered ready. Then AGAIN during the formal test conducted on a test environment. And continuously AFTER applications are deployed to production and exposed to users.

Validation on the other hand starts with the implementation and is performed continuously throughout the delivery process – and – in the form of operations monitoring – continues after go-live.

Intentions – why are we testing?

- Validation of designs and requirements
- Validation of the intended outcome when exposed to users
- Verification of functional requirements
 - Narrow scope technical capabilities ("units")
 - Use cases (Sometimes expressed as user stories)
 - Full end-to-end processes
 - "Business Capabilities" that may contain multiple processes, integrations, etc.
- Verification of non-functional requirements
 - Performance
 - Security
 - Maintainability
 - Portability
 - Scalability
 - Coding Guideline compliance
 - Documentation
- Protection from Regressions
- Verification of defect resolution

Scopes – what are we testing?

- Unit
(the smallest possible technical element)
- Capability
(A combination of technical units that provide some distinct value to users or stakeholders)
- System / Integration
(A combination of capabilities and external systems – real or simulated)

Methods – how are we testing?

- Manual
 - Ad-hoc / Exploratory
(Testers interact with the system without a clear script or path to follow – in some cases testers may be given a goal but no guidance on how to achieve the goal)
 - Scripted
(Testers interact with the system following a clearly defined path and given a clear set of quality criteria on when the test is to be considered failed or successful)
 - Code review
(Peer developers look at code and join a discussion about what they see and understand)
- Automated
 - Whitebox
(“Whitebox” means that the test tool has both access to the defined interface AND to the underlying structures, e.g., the database tables being modified during the test – so the result of the test may not only depend on the output provided by defined interfaces but also by assessing the actual changes to the database or external systems)
 - Simulation
(Test scripts execute specific parts of the system and compare actual and expected results)
 - Static code analysis
(Test scripts are used to analyze the source code artifacts directly without executing any of the analyzed code)
 - Blackbox
(“Blackbox” means that the test tool has ONLY access to the system via defined interfaces – and the test result is assessed based on the responses provided by these defined interfaces)
 - Via API
 - Via GUI
 - Via data import/export interfaces (e.g., SFTP, file shares, database connections, etc.)

Tools – what do we use for testing?

- **Automated Test Framework**
The ATF is the ServiceNow test automation tool that enables automated scripted white box and black box testing and to ship the tests as part of applications.
- **Instance Scan**
Instance Scan is a ServiceNow platform capability that can be used to perform white box testing on code and configuration artifacts at rest – that is to check the source code and the configuration instead of executing it.
- **Test Management 2.0**
A ServiceNow platform capability to manage and document manual scripted tests.
- **External tools (E.g., Selenium, RoboClient etc.)**
Any other tools for automated or manual black box or white box testing or the documentation of such tests.

Test Suites that yield Value over Time

Testing is effort. Yet, a well-balanced and executed test strategy can reduce overall efforts, costs and increase trust of all stakeholders over time.

A **Test Suite** is the superset of repeatable tests being performed as part of the process.

This does not include any ad-hoc or exploration tests.

While all tests that are conducted manually directly translate into effort each time the tests are executed, the long-term effects on efforts of automated tests are more subtle.

Obviously, the initial creation of automated tests is associated to some effort – which if done very early in the process (during or even before actual development) is not only hard to measure but – that's the good news – almost insignificantly low.

The tricky part is the maintenance of a Test Suite over time. How much effort is required to adapt an existing Test Suite to modified components? That is the key question.

The better a Test Suite is designed – the less costs it will require over time – and the more value it will yield – by detecting introduced regressions early in the process – when their remediation is least expensive.

What can be said about the long-term costs of a code base can also be applied to a Test Suite.

The more code there is, the higher the maintenance cost. If less code (in a wider sense) is required to implement the required capabilities, more value is produced.

The exact same is true for tests in a Test Suite.

When talking about code, the most obvious pattern to look after is duplication. The more often the exact same code pattern, logic or sequences repeat themselves in a code base, the more code is needed. Duplication, however, may come in very subtle forms. Exact copies of functions or even classes may be spotted here and there, but most cases of duplication are more subtle and often not that easy to detect – even for the experienced eye.

It requires intimate knowledge and oversight over large parts of a code base to spot the more subtle repeating patterns for the human eye – and so far, even very advanced static code analysis tools were not convincingly good at that.

Rather it requires a mindset shared by the whole team to always seek logic that can be refactored into re-usable components. This will shift the task from spotting duplications in hindsight towards making code as re-usable as possible all the time – which will eventually lead to code being moved away from a very specific context into places where the team agrees to put re-useable components – where it is then easier to see if a comparable component already exists and whether – maybe with slight modifications – it can be used for the specific case a developer currently works on.

This same principle should be applied to tests.

When done so, the resulting Test Suite is likely to consist of many smaller, narrow scoped tests, and fewer larger, broad-scoped tests. It will consist of tests that verify the functionality of single components, rather than many components at the same time. Tests that do not depend on each other, data, the environment, the time of day.

If most tests are of that nature – and that applies to automated and manual scripted tests – the resulting Test Suite will be easier to change and adapt and hence to maintain.

Test Data

To perform tests in many cases test data is required. To test different scenarios and use cases different combinations of data are needed during the execution of different tests. And some of the data combinations may not exist at the same time.

So, the creation and maintenance of test data on different instances to support different test scenarios and different testing methods, intentions and scopes is all but trivial.

Test data - as opposed to production data - is any data (not code) that is used to conduct tests. In rare cases, test data may even exist on a production instance and that data may be stored in the same tables as production data.

With that being stated, it should be obvious that test data must be created, managed, maintained, and eventually be removed consciously and according to an agreed and defined process. Ownership should be defined for test data on different instances.

Consider the following guide on test data:

- Test data must be obvious – both the human eye and software should be able to tell if a given record is test data or not
- Automated Tests must not depend on any existing test data on an instance: all data required by the automated test must be created by the automated test
- Test data should be generated through code – not manually
- Test data ownership must be clearly defined per instance and table

Test Users

Test users are test data artifacts. However, they are special. They are not only part of a test data set that is required to test specific scenarios or use cases, but they are used to perform the tests – as test users are impersonated by manual testers or automated tests.

Test users should hence be as close to real users (human or technical) as possible to produce high fidelity test results.

As outlined in chapter “Personas and Roles” users should always be associated to a persona role – the same can be said about test users for end-to-end tests – but in some cases only specific technical roles should be assigned to test users to verify the exact behavior and access defined for specific roles.

The platform does not have a specific representation of test data – including test users in its OOTB data model. There is no flag by which test data and test users could be identified.

This calls for a naming convention to identify test users.

The following convention worked well for several teams for several reasons:

- A test user's “First name” is ALWAYS “Test”
- A test user's “User ID” ALWAYS starts with “test.”
- A test user's “Last Name” contains a hint on the persona or role which the test user has (e.g., “Service Desk Agent”, “Report Admin”, or “ITIL”) If applicable the application name that defines the role should also be used in the “Last Name” (e.g., “Agile Agilist”, “Deployer Manager”)
- A test user's “User ID” ALWAYS ends with the lowercase, dot-separated equivalent of its “Last Name” (e.g., “test.servicedeskagent”, “test.report.admin”, “test.itil”, “test.agile.agilist”, “test.deployer.manager”)
Note how the dot is used to express a hierarchy in the naming scheme.

By following that convention, developers and testers can

- Filter for test users
- Identify a test user as the source of a log entry or any kind of errors

The Definition of Ready ideally defines some test cases which should be created as part of the backlog item. All test cases require a test user; hence the description of test cases should specify which personas (or roles) should be used to perform the corresponding test cases.

Generative AI

AI and the Process – How does this fit?

How does AI impact the recommendations in this document? Does it matter at all? Does it make a difference? Is AI even rendering the whole thing obsolete?

All fair questions that need to be asked. And answered.

First things first: With AI in mix, it is more important than ever to establish a robust development and deployment process.

The following chapters provide guidance on how AI plays a role in the process and the outcomes that the process delivers.

Developing and Deploying AI-based Capabilities

With more ready-to-use AI capabilities on the platform, more such capabilities are configured and prepared for production use. Such configuration or even the development of new features based on AI capabilities should be subject to the same rigorous structured and automated QA and deployment mechanisms - like any other (non-AI-based) feature.

One may argue that the use of generative AI – from a strict technical perspective – is just yet another REST API that is requested by an app running on the platform.

Of course, it is not just that.

With its non-deterministic, non-binary responses, AI-based capabilities require new testing and validation strategies that go beyond testing the data flow and technical integration. They also require a new form of governance, monitoring and refinement.

Many choices must be made, starting with which LLM to use, where to operate it, whether to feed it with sensitive data, how to protect such data, what data to be used for training, how to choose training data and how to validate the success of all these choices.

Answering all these questions goes beyond the scope of this document. It should however be obvious that with the integration of artificial intelligence complexity increases and effective guardrails are crucial for success.

Coding Guidelines must evolve to cover lessons from mistakes including AI integration and configuration.

Automated Testing strategies must cover AI generated output.

The process described in this document provides a solid foundation to address these challenges, too. Without a robust process, AI might easily overstretch any organization's capabilities.

AI augmented development

Besides the many uses of AI in production where the output of LLMs is directly impacting users and processes, the aspect of AI augmented development is equally important.

Until the general availability of AI tools, it was mainly humans that defined machine behavior. Humans are principle, the computer their agent. We wrote code, computers executed it.

As of the release of this document this relationship has not changed yet.

However, as developers delegate more tasks to AI the result is no longer just prone to human error – it increasingly becomes prone to machine error, too.

With increasing AI capabilities to produce code and configuration, so does human confidence in these capabilities over time – and so our ability to detect such mistakes may decline.

A robust development and deployment process is essential to guardrail work performed not only by humans – but by machines alike.

Most processes that exist are aimed at and are optimized for detecting human error (hopefully early) in the process.

Now, processes must be adapted and extended to aim at detecting machine errors, too.

The development and deployment process proposed in the document serves as a rock-solid foundation.

Developers may have a degree of freedom to choose their tools and methods – including AI companions for writing code – as long as the result match the same high standards.

AI generated code must be covered by automated tests and comply with the Coding Guideline. No exceptions to be made.

As soon as anti-patterns emerge from AI augmented code, such anti-patterns must be made subject to Coding Guidelines, too.

Test-Driven Code Generation

LLMs have made considerable progress since ChatGPT was made available to the public for the first time. And the quality of generated code improved greatly. Yet, the code created by far does not yet meet production standards and is not fit-for-purpose in many cases.

The core challenge is that code - to function properly - must be an expression of an algorithmic idea - and not the random summary of something that someone (or something) read before. The latter may be the starting point, but it is not sufficient for a final version.

Whether tests are written as code or executed manually by a developer, in most cases a (human) developer runs in cycles of building a hypothesis, then writing code, testing the resulting program and analyzing the result of the test - and then starts over.

Test-Driven Development is the practice of automating parts of that cycle by writing test code early in the process and shortening the feedback loop between the hypothesis and the analysis, allowing for greater productivity and more robust results (see chapter "Test-Driven Development").

When the quality of generated code is assessed, we compare apples and oranges. Generated code usually results from a single prompt, containing a description of the requirement, eventually the current version of the code. Do we really expect AI to deliver a result that is comparable to what an experienced human developer would provide after running through the cycle several times?

The relevant question is: are we asking AI the right way?

Test-Driven Code Generation (also referred to as "Test-Driven Generation" or "TDG") is to integrate the generative power of AI in the same procedure that the human developer would go through. Where Test-Driven Development automates the testing part in the cycle, Test-Driven Code Generation aims at automating the part where code is written.

1. Humans write boilerplate code (that defines the function name, parameters and return values) and describe the outcome.
2. Humans write tests.
3. The test is run.
4. If the test passes, end here or return to step 2 to add more test cases.
5. If the test fails, ask AI to implement the code based on the description, the interface, the current version of the code, the test code and the latest test results.
6. Return to step 3.

For disambiguation: The term "Test-Driven Generation" has sometimes been used to describe the process of creating test cases (not the code to fit these tests) using AI. However, more recent publications use the term consistently for code (not test) generation.

Developer Efficiency Gains

What to expect from AI augmented coding? As always, the answer is: it depends.

It depends on the LLM, the specific prompt and the difficulty of the problem to be solved. To make reasonable statements about solution quality, the following problem taxonomy is proposed:

Class 1 Problems

These are low complexity non-domain specific problems, where many fit-for-purpose solutions are available on public sources. Solutions do not depend on additional data sources or the system environment. The resulting code delivers fully deterministic results.

Examples are simple math problems (linear algebra, geometry, Newton physics) like calculating the volume of solid bodies or gravitational vectors, or well-known algorithms for sorting or searching.

Class 2 Problems

These are high complexity non-domain specific problems or low complexity domain specific problems. Solutions are available on public sources, but not all of them are fit-for-purpose.

Examples are complex math problems with partially non-deterministic results, optimization problems, data validation that require implicit knowledge (e.g., emails, URLs, ISINs, IBANs), problems that require data retrieval via database interfaces or REST APIs, low complexity domain-specific problems that involve ServiceNow specific APIs like retrieving records based on filter criteria.

Class 3 Problems

These are high complexity domain specific problems. Fit-for-purpose solutions are not available on public sources or are difficult to find due to varying terminology or context.

An example is to retrieve an API token from a ServiceNow connection alias and then to use it with a REST API that requires multiple requests to perform a task.

As of late 2025, Test-Driven Code Generation as implemented in the ArtificialDeveloper open-source app, using the GPT-5o model works very well for class 1 problems, works most of the time for class 2 problems but does not excel on class 3 problems in many cases - although provides code that can serve as a basis for further manual work.

As the Test-Driven Code Generation approach requires developers to split capabilities into functions and produce automated tests, the efficiency gain for class 1 problems may be neglectable but for class 2 problems it can be significant. One could argue though that enforcing the refactoring of code into Script Include functions and creation of automated tests is per se a design pattern that saves costs on the long run. As a result, the adoption of TDG and hence indirectly adopting TDD already represents an efficiency gain - even if a particular problem cannot be fully solved through TDG.

Building Blocks

Application Architecture and Dependency Management

In the past, individual changes (or “customizations”) were done and captured in update sets. These update sets were then deployed selectively. And that was ok, if there were only a few dependencies and their sequence either did not change or did not matter.

Some clients have built their own methods, processes, and tools for handling the “update set hell” in more complex scenarios. Kudos for that.

This document proposes a fundamentally different approach: Everything is shipped as an application. That can be

- a scoped application (preferred)
- a global application (should be the exception)
- a customization application (if there is no other alternative to change a plugin's or store app's behavior)

Scoped applications are always the first choice. From an architectural point of view, these applications are the modules that constitute the application architecture.

Architects have several decisions to make, and to make this process work, the most crucial decision is how to split the business capabilities into technical capabilities and how to slice and dice these technical capabilities into one or more applications.

That is the primary task for architects. The business describes a demand, the development team and the architects draft a solution, and decide:

- Which applications should be changed?
- What new applications should be created?

The superset of these decisions is what we call the “Application Architecture”. Ideally there is an up-to-date visual representation and documentation of that architecture for reference and for new developers to understand what is going on.

When we talk about applications as modules, some of these modules might relate to each other. More precisely, they depend on each other.

This is where “Dependency Management” comes into play.

As important as the decision on which technical capability should be implemented into which application, so is to keep track about the dependencies between these applications.

First things first: What are dependencies?

In general terms application “A” is dependent on application “B” if

- “A” calls a function in “B”
- “A” extends something in “B”
- “A” just only works if “B” is also installed

In ServiceNow terms, "A" is dependent on "B" if (for example)

- "A" uses a Script Include contained in "B"
- "A" extends a table defined in "B"
- "A" adds a field to a table defined in "B"
- "A" reads or writes records from or to a table that is defined in "B"
- "A" requires a business rule, flow or other logic contained in "B" to work properly

After establishing the existence of a dependency between two applications, we also need to consider their versions. The dependency of one application on another may change over time and not all versions depend on each other in the same way.

The statement: Application "A" depends on application "B" is not sufficient – in fact it is a specific version of application "A" that depends on one or more versions of application "B".

Imagine a case where application "A" version 1 depends on a function that has been introduced in application "B" version 2.

Application "A" version 1 depends on application "B" version 2 (or later) – you may refer to chapter "Semantic Versioning" for more details on versions, their compatibility and meaning.

To know the dependencies between all applications serves multiple purposes:

- We can derive the exact order in which these applications must be installed during deployment
- We can determine which other applications must be installed (or even baselined) before an application can be deployed
- We can assess the impact of potential backward incompatible changes of an application on its consumers (the applications that depend on the application to be changed)

One can think of the superset of all applications in an instance as a forest of one or more dependency trees. Some applications are roots as no other applications depend on them. But these root applications may depend on other applications, and these applications may then depend on even other applications – forming branches and leaves of the dependency trees.

To simplify this, one can transform the forest into a single tree.

Enter the **"Platform" scoped application**. Each instance should have its own platform scoped application that serves as the one and only tree root for all other applications. The version of the platform application could be interpreted as the version of an overarching release. The dependencies of the platform application define which versions of all other applications must be deployed as a release package.

The Platform application really is a scoped application with its own repository in the Source Control System – which needs to be maintained by the development team and baselined whenever a new release is prepared.

Once Deployment Automation has reached the highest grade, a deployment can be as simple as "just" installing the platform app (which will then lead to the installation of all its dependencies in the required versions).

The metaphor of the tree points to another detail which is important: Dependencies must ALWAYS be unidirectional.

If application A depends on application B, application B MUST NOT depend on application A.

This is also true for indirect dependencies: If application A depends on application B and application B depends on application C, application C MUST NOT depend on application A.

Such a scenario would form a circular dependency which is forbidden.

An application and its dependencies can be represented as a **dependency tree** where the root is the application on top and its dependencies are the branches, and their dependencies are the smaller branches, and their dependencies are the leaves and so on. Some applications may appear multiple times in the tree.

An example:

- App Alpha
 - App Beta
 - App Epsilon
 - App Gamma
 - App Zeta
 - App Gamma
 - App Zeta
 - App Delta
 - App Eta (Not a typo, refer to https://en.wikipedia.org/wiki/Greek_alphabet)
 - App Theta

The example should be read as "App Alpha" depends on "App Beta". "App Beta" depends on "App Epsilon" etc.

Also note that in the real world each of the dependencies must also include a version requirement.

This dependency tree can also be represented as a **dependency list** which indicates the order in which application must be installed:

1. App Theta
2. App Eta
3. App Delta
4. App Zeta
5. App Gamma
6. App Epsilon
7. App Beta
8. App Alpha

Each dependency tree may be transformed into multiple valid dependency lists. In this example, the following list is also valid:

1. App Zeta
2. App Gamma
3. App Epsilon
4. App Beta
5. App Theta
6. App Eta
7. App Delta
8. App Alpha

Note that “App Gamma” appears at different positions in the two variants of the dependency list, but “App Zeta” is always installed BEFORE “App Gamma” because “App Gamma” depends on “App Zeta”.

In any case an application can only be installed after all its direct and indirect dependencies are installed in the required versions.

This rule must be strictly followed. The most important reason is this: Given an application Alpha adds a field to a table that is defined in application Beta, application Alpha depends on application Beta. If application Alpha were installed before application Beta is installed, a new table would be created in Alpha’s scope containing only the added field. A subsequent installation of application Beta would fail and only database administrators can fix the issue.

If a circular dependency is identified at any time during development, both involved applications must be refactored to resolve the issue. The team must apply zero tolerance to circular dependencies.

To perform Dependency Management on a day-to-day basis, the platform must be extended with the following capabilities:

- Document dependencies from one application to another allowing multiple (major) version requirements
- Such dependencies must be part of an application so that they are also stored in the application's source code repository
- The dependencies must be stored in a way that they can be retrieved and processed from a source code repository
- The dependency tree must be visible to developers
- The dependency list shows all required dependencies in the exact order in which they must be installed

Any issues (like circular dependencies) must be detected automatically

These requirements are not met by the OOTB dependency table provided in ServiceNow – hence this additional capability must be implemented.

The open-source application “DevTools” supports these capabilities.

As in most cases, by far the most relevant factor is people. Without a team of architects – or developers who perform that role and take the responsibility – any technical tool is pointless.

Setting up a team of people who really care for the application architecture, drive its development, create a vision, and pursue a long-term strategy is the key success factor.

Unsure about the Greek Alphabet? Refer to https://en.wikipedia.org/wiki/Greek_alphabet.

Definition of Ready

The Definition of Ready is the central building block for the first quality gate. It is important that the team has a say in what should be contained in the Definition of Ready. However, it is strongly recommended to include the following points:

- Description of desired outcome / Use case
- Affected applications
- Affected personas and technical users
- Affected process(es)
- Affected data entities
- New and modified fields (including labels)
- Affected GUIs (Workspaces, Classic, Portal, Mobile, Custom)
- Relevant GUI languages
- Affected notifications
- Affected technical interfaces
- Required access restrictions (per persona)
- Data Protection/GDPR/Compliance implications
- Required documentation
- Proposed test data
- Proposed test cases
- Proposed data quality checks

When presented to teams for the first time this list may cause some concerns – especially amongst product owners or other stakeholders representing the business. They might argue that a good portion of these topics cannot be provided by them.

But passing the first quality gate is NOT a handover from product owners to developers!

It is important to understand that it remains a team effort to describe a backlog item in a way that it is compliant with the Definition of Ready. It requires the business to describe the need, the architects to make decisions on which applications to change and the development team to fill in implementation details where needed.

Definition of Done

The Definition of Done is the central guidance and checklist for the second quality gate: Backlog item is “Done”.

In many agile organizations, both the “ready” and the “done” state are seen as hand-over points where the work (or responsibility for something) is handed over from one team to another. This perspective might seem comforting, but it is not helpful.

This process will not work well when things are thrown over the fence and then thrown back if they do not comply with a set of rigid rules.

At all stages it remains a team effort – to define the rules, to use them, to apply them, to review and change them.

The Definition of Done is often used for protection by a development team – where a team tries to defend a way of working which they perceive as good quality against business stakeholders who want to get new features out of the door as fast as possible at the lowest possible cost.

Deployment Automation and the embedded use of automated quality checks are intended to bring some relief to this conflict. Once there is a baseline of quality requirements which are checked as part of every deployment, both the developer team and product owners know that if they do not deliver according to the agreed quality levels – nothing will be shipped. Period.

The Definition of Done is a pre-check after the completion of a single backlog item – before the feature is shipped as part of an application version. The line items in the following proposal should hence be considered as non-negotiable; others can be added at the team's discretion. The question is less “if” these checks should be performed, the question is rather “when” they should be performed:

- All the features described are implemented
- Implementation is peer reviewed
- All described (and relevant) test cases covered in ATF tests
- All related (new or changed) ATF tests are peer reviewed
- Compliance with the Coding Guideline (i.e., no Instance Scan findings)
- All related ATF tests pass
- All technical documentation written
- All changes have been documented in the Release Notes
- Validated by Product Owner (explorative test)
- Application dependencies are verified and updated if needed

Coding Guideline

It may appear self-evident to make a coding guideline part of a mature development and deployment process. It is not. Only a few teams or organizations have coding guidelines, and even fewer actively use them. But why is this the case?

Good coding practices are expected to lead to better code, which is easier to understand, easier to maintain, easier to analyze, easier to test, easier to change. Consistency across a code base produces value by itself, since developers can then navigate the code base better and are able to make changes to different parts of the code – those parts they were not actively working on themselves.

A coding guideline may not only describe how specific code details should look like, but it can also describe which automated tests should exist and how code should be structured – which types of functionalities should be implemented where and thus may even go beyond being a guideline for code – but even be a guideline for the architecture.

ServiceNow provides many low-code and no-code capabilities. The outcomes produced with these tools should also be covered in the Coding Guideline. The “Coding” in Coding Guideline should be interpreted as broadly as possible.

A Coding Guideline may cover:

- The use or non-use of specific coding patterns (or anti-patterns)
- Error handling
- Naming conventions
- In-code documentation
- Test coverage
- Modularization and how to split things
- ...many other aspects

A coding guideline is a set of rules – to which a team has agreed. That points to a very important aspect: Participation. If the Coding Guideline is a law imposed by a privileged small group to control the work of a larger developer population, we should not expect any positive impact.

The coding guideline should be the result of a collective experience of failures and mistakes. The reason why a team uses Coding Guidelines is to avoid making the same mistakes and experiencing the same failures again. The Coding Guideline should be an expression of collective learning.

To make this happen the whole team must have the opportunity to participate and contribute, to have a say in which rules should be applied, and which are not and which exceptions to the rule should be allowed in which cases. For sure, consensus cannot be achieved in all cases – so some form of a decision-making process is needed. Please refer to chapter “Communities of Practice” for guidance.

Once a team has agreed on a coding guideline (and on a process to change it going forward) it needs to be applied. Just writing down what the team intends to do is not good enough. To be straight: the coding guideline document alone will not have any significant impact on the code base. The individual rules described in the guideline must be checked – automatically – by the developer when writing code and as part of every single deployment.

The OOTB platform feature “Instance Scan” provides the framework to build checks that can investigate every single record (of an application) and assess whether it is compliant to the Coding Guideline. Further, the checks and their documentation can be used to generate the

Coding Guideline document – making the checks the actual single source of truth of the Coding Guideline. Instance Scan essentially is turned into a static code analysis tool within the ServiceNow platform.

With that being said, the recommendation is to limit the Coding Guideline to statements that can be checked automatically – and hence produce a binary output of either compliance or non-compliance with no room for interpretation.

This is important for two reasons: running an Instance Scan check as part of the deployment pipeline requires a clear decision on whether the deployment should proceed (no findings produced by any check) or not (at least one finding was produced by a check). A developer should get clear guidance on whether the code they wrote is compliant or not – any room for interpretation will lead to ignorance in the long run.

If, however, a check reveals a finding and the developer is yet convinced everything is fine and as it should be, there should always be the following options:

- The intention of the check is valid and agreed by the team, but the implementation of the check does not match that intention, the check must be improved
- The team does not agree with the intention and decides to remove the check
- The team confirms the intention and the correctness of the result, and the developers get all the support needed to refactor the code and the time and support needed to understand the intention and solution

This feedback loop is important for organizational learning and to build awareness and support in the team.

A team does not need to start from scratch with a blank page, there are numerous resources available that can serve as a basis for the team's Coding Guideline. There are several publications on ServiceNow good practices (not to say "best practices"), numerous publications on JavaScript coding practices in general and even several open-source contributions for check implementations to start with.

However, to integrate the Coding Guideline related Instance Scan checks into the developer workflow and into the automated deployment the following technical capabilities are required (which are not available OOTB):

- Option to run an Instance Scan check on an application that only uses checks related to the Coding Guideline
- Option to run an Instance Scan check on a single application file that only uses checks relevant for the Coding Guideline
- Creation of the Coding Guideline document based on the relevant checks (to be published on a knowledge base, a corporate wiki, or any other knowledge management system)
- Mechanism to ignore specific checks on specific applications (blacklisting checks)
- Mechanism to use specific checks only on specific applications (whitelisting checks)

The open-source tool "CodeSanity" has all these technical capabilities and contains numerous implemented checks which embody the learnings of several large teams working on highly complex application eco-systems.

When a new set of checks are applied to an existing code base, the new rules will inevitably clash with the existing code. The team needs to figure out what fits, what they need (and like), where exceptions need to be made for the existing code and where investments should be made to refactor code.

Automated Tests

The key to shorter release cycles is automated tests.

The key to fewer regressions is automated tests.

The key to fewer defects is – again - automated tests.

Automated tests are crucial for a robust, reliable, and mature software development and deployment process.

Titus Winters, one of the authors of *Software Engineering at Google* stated: “One of the broad truths we've seen to be true is the idea that finding problems earlier in the developer workflow usually reduces costs.”

This underlines the idea – often referred to as “shift left” – to establish quality as early in the process as possible.

There are different forms of automated tests, and such tests may be executed for different reasons and different stages of the process.

Of the many forms of automated tests (and tools) which may be helpful to verify quality in a specific organization, the automated tests that can be developed and deployed as part of the application and run within the platform are most relevant. In ServiceNow these are tests based on the Automated Testing Framework (“ATF”). The ATF is an OOTB platform feature.

ATF – or to be precise – the many ServiceNow plugins and products come with a set of ready-built tests. These quick-start tests validate the OOTB features in their unaltered non-customized form. However, most relevant for this development and deployment process are the tests that the development and test team create by themselves to verify the result of their work.

The recommendation in this document is to

- Practice Test-Driven Development – and create tests as early as possible by the development team (refer to chapter “Test-Driven Development” for details)
- Create tests as part of the applications to be shipped
- Execute tests as part of every deployment
- Maintain zero-tolerance regarding failed tests

Executing tests is an integral part of the automated deployment and only if all tests pass an application should be deployed to a downstream instance. That requires discipline regarding the fidelity of all tests contained in an application regarding its behavior. A zero-tolerance mindset can only be applied if tests are kept up to date with the application in the long run.

To be executed continuously and successfully as part of the deployment and to be maintainable in the long run, tests should meet the following requirements:

- Tests are self-contained – that means they are agnostic regarding the data that already exists on the instance. Required test data must be created by the test – that includes users and groups and any other record that is relevant for the outcome. This is very important yet sometimes difficult to achieve.
- Tests must be executable in headless mode. This may be achieved using the Cloud Runner – for tests that make use of client-side test steps or by using server-side test steps only.
- Tests should not be redundant – every test should verify a specific aspect of the application's behavior; any duplication should be avoided. The scope of a single test

should be as narrow as possible. Lengthy end-to-end tests are more difficult and costly to maintain.

- Tests are part of an application and verify the behavior of that application – by that these tests indirectly verify the behavior of the application's dependencies. However, the results of the tests must not depend on the behavior of applications that depend on the application being tested. As a result, tests contribute to the dependencies of their application and like any other application component bi-directional (i.e. circular) dependencies must be avoided.
- All tests that are intended to be executed as part of the automated deployment should be contained in a test suite that is named exactly after the application – so that the tests can be clearly identified – and of course be contained in that application.
E.g. an application named "My App" should contain a test suite called "My App". This test suite is to be called during deployment.
The application may have any number of additional test suites (e.g. for GUI based tests that cannot be run during deployment – the recommended name for the test suite containing GUI-based tests is "My App GUI")

"Tests are not outside the system; rather, they are parts of the system that must be well designed if they are to provide the desired benefits of stability and regression."

Robert C. Martin, Clean Architecture.

Writing automated tests has an impact on how software must be designed. Teams that start writing tests late in the process will face all sorts of challenges. Obviously this insight is of little help when a development team is faced with already written code and applications with little test coverage.

However, that must not be accepted as an argument against making the first step and starting the journey.

The increase of test coverage has always a positive impact on code structure, architecture, and overall maintainability of code – as there is a natural incentive to refactor (existing) code to allow better testability.

Sometimes stakeholders may oppose the idea of automated testing as there is a widespread – and false – belief that maintaining a suite of automated tests implies higher costs and prolonged time-to-market cycles. While this observation may be true for the first iterations after a development team started the new practice, even short to mid-term effects are the opposite. A well-crafted test suite increases velocity, reduces roundtrips between development and test, reduces production issues and results in better maintained code.

Never ever argue about the costs of automated testing. Never adjust estimates by skipping automated tests.

Defend that ground at all costs!

Source Control System

No matter which variant of the process you choose, which level of automation, how broad the test coverage or how detailed your Coding Guideline is – in any case you will need a Source Control System.

This can be

- Private or even public repositories on GitHub
- A self-hosted or private instance of GitLab
- Any other source control system supported by ServiceNow

Many organizations have requirements to store code and/or all deployed application versions in their self-operated Source Control System (e.g., GitLab) or definitive software library (e.g., Jenkins).

AppRepo – the ServiceNow hosted “Application Repository” – however does NOT qualify as a Source Control System. AppRepo can deploy published – that is “baselined” applications to downstream instances. Think of AppRepo as the ServiceNow equivalent of a definitive software library (in ITIL terms) (e.g., Jenkins) – not more, not less.

To make this process work, we will need both: a system to safely store ServiceNow applications while they are being worked on and to reliably deploy these applications to downstream instances.

Whether a Source Control System should be combined with the ServiceNow hosted AppRepo is debatable.

ServiceNow official documentation – at the time of the publication of this document – may be read as if it advises against the use of a Source Control System to perform deployments to production instances. The main argument is that deployments via source control may unwittingly delete tables and columns and the contained records – if such tables and column definitions are not (or no longer) part of the application stored in the source control branch being deployed. This however is exactly the expected and intended behavior and perfectly aligned with the process proposed in this document.

The customer environments where the insights were gathered that led to this document all did not include the use of AppRepo.

Here are some reasons that resulted into the decision not to use AppRepo:

- On-premise hosting and air-gapped – and hence no access to ServiceNow AppRepo
- Limitations regarding the amount of stored application versions in AppRepo – this deficiency is reported to be fixed in the meantime
- Limitations regarding the total size of application file data to be stored in AppRepo
- Additional risk and maintenance work for pipeline automation using two different deployment methods (via source control system AND AppRepo)
- Inability to publish hotfix versions with a lower version number than the latest published version – this deficiency is reported to be fixed in the meantime

For these reasons, the guidance in this document is to use a Source Control System both for development and for deployments – including deployments to production.

The process and all technical and operational details are agnostic to the use of AppRepo. To leverage the benefits of this process its use is not required.

Accessing the Source Control System from an instance may be a challenge of its own. Consider the following steps:

1. Personal access credentials may have to be set up to access the web GUI of the Source Control System for those users that operate the deployment process – to create new repositories, manage branches and create or renew access tokens for API access
2. Within the Source Control System, a project, group or team hierarchy may exist. So before being allowed to add new repositories for ServiceNow apps, a new “space” may have to be created
3. To interact with a Source Control System's REST API, an access token is required. This access token should NOT be associated with a specific user account to avoid it becoming invalid if the person leaves the organization
4. Access to the Source Control System's REST API by the various ServiceNow instances may have to be configured and allowed by firewall configuration

Many organizations impose strict security requirements when accessing their Source Control System – for good reasons. The Source Control System literally stores the DNA of the organization, the specific software and hence the business logic that underpins the success of an organization in the market or their field of practice.

Some organizations explicitly demand personal accounts when accessing the Source Control System – often preferably (or demand) using OAuth.

As of the release of this document, the ServiceNow platform does not support OAuth access to Source Control integration (via Studio).

When the security and compliance stakeholder demands OAuth and the platform does not support it, would this rule out the use of a Source Control System as a deployment mechanism?

Not necessarily!

It is common knowledge that many bad actors act from within an organization, actors who have some level of access to some systems or digital assets controlled by the attacked organization. This is the kind of attack vector that these security requirements aim to manage and defuse.

To mitigate this incompatibility, let us consider the whole development environment (i.e. the DEV instance) as a zero-trust environment and let us also consider the “dev” branches of all applications as zero-trust environments.

After an application version is baselined, and the new version branch is turned into an immutable read-only digital asset, testing is performed – for functionality and security.

Only after a successful test is the application version considered safe for production use.

When applying this process with a zero-tolerance against failing automated and manual tests, static code analysis and security findings – the actual account that was used to make a commit to a dev branch becomes less of a concern.

Hence access tokens associated to service accounts may be accepted by the organization's security stakeholders.

Branching Strategy

Most experienced developers have at time point in time worked with a source control system and many of them have been exposed to widely adopted standards like gitflow – which essentially proposes the use of feature branches.

The main idea of feature branches is that any separable feature is developed in its own branch and once the work on that feature is completed, that branch is merged back into a master (or default) branch.

This approach works under the assumption that source code can be merged by the Source Control System. Merging involves a side-by-side comparison of source code files and the possibility to make decisions about how to inject the proposed changes taken from the feature branch should be ingested back into the master branch.

That works fine with traditional source code files, config files, even xml files to a certain extent. And here is the problem: branch merging does NOT work with all ServiceNow application files. Here is why: Traditional source code is text only.

ServiceNow application files are stored as records in the database and once they are committed to the Source Control System, these records are represented as xml files – which do not only contain source code but also meta data and – this is the tricky part – references to other application files.

For some Application Files, like Script Includes and Business Rules – which consist of exactly one record, the process of merging – even when represented as xml files – is working.

ServiceNow however provides a growing toolset of low-code or even no-code mechanisms to design and implement application behavior – Flow Designer and UI Builder just to name a few.

A Flow is not just a single record, it is represented by a vast network of records, which are tightly coupled and are neither human readable as xml files nor can they be merged by a Source Control System in any meaningful way.

Another property of the ServiceNow platform is that an application is always connected to one source control repository branch at any given time. So, if two developers work on the same application in the same instance, they make changes to the same set of records that constitute that application.

One may argue about the design decisions made – but that won't change the fact that

- Many Application Files just CANNOT be merged in a Source Control System
- Multiple developers cannot work on multiple branches of an application on one development instance at the same time

To cut this short: **forget gitflow, forget feature branches, forget merging!**

With that being said, the questions are

- How can developers work on separate features?
- How can features be released selectively?
- What to do if the work on a feature is not yet finished when an application version should be shipped?

The answers to these questions can be found in chapter "Technical Debt Management" and – in more technical terms – in chapter "Managing Unfinished Work".

If feature branches are not an option – how should developers now work with branches? What value do we get from the Source Control System after all?

The Source Control System serves three main purposes:

- To keep a reversible trail of all changes deemed relevant by the development team
- To store all baselined application versions that were delivered by the development team
- To allow switching between different application versions – that may be multiple versions in development – and all versions that were ever baselined

This is how it works:

- Each application has its own repository (or “project” in GitLab’s terms) – and this will stay that way even if ServiceNow ever decides to support storing multiple applications in one repository in the future.
- The default branch in all application repositories is to be named “dev” (most Source Control Systems start with a branch called “master” or “main” when creating a new repository – these branches need to be deleted).
- The “dev” branch is always the active branch in the development environment.
- Whenever an application version is baselined, a new branch is created based on the current state of the “dev” branch which is to be named exactly like the version to be baselined (e.g. “1.5.0”)
- This new **version branch** is to be locked immediately after being created – no further changes to this branch should be allowed.
- In case of a hotfix, the existing version branch is installed on an instance, the application is modified, and the new state is stored as a new version branch (which possibly represents a lower version than the latest version already stored as a branch).
The instance used to develop a hotfix depends on the instance setup used. Ideally there is a dedicated HOTFIX instance reserved for ad-hoc hotfixing. If such an instance is not available, hotfixes can also be created on DEV or TEST instances (see chapter “Instance Setups” for more details).

By applying these rules, the read-only version branches form the audit trail of delivered application versions by the development team and the single source of truth of anything that will ever be deployed on a downstream instance. Note that applications can only be installed from a branch (not from a tag) hence branches must be used.

Developers will always commit their changes to the “dev” branch during regular development. The guidance should be that developers commit all changes made on the development instance as soon as possible – even if they are yet unfinished and not yet ready for deployment. In theory, a team could also commit only finished work and stash any uncommitted changes during application baseline – when a new branch is being created – and then reapply the stashed changes after switching back to the dev branch. Real world experience with two larger developer teams revealed that – although technically feasible – the approach is more difficult to manage and more error prone when executed.

For hotfixes, the process is slightly different – please refer to chapter “Hotfixing and Backporting”. If a development team currently works on a version of an application that is soon to be released (e.g., version “1.x”) and at the same time the next – and backward incompatible version should be worked on (e.g., version “2.x”) – a second development instance is required. The work on this future version would then be stored in a second “dev2” branch – please also refer to chapter “Hotfixing and Backporting” for further guidance on that scenario.

The list of branches of an application could look like this:

- dev
- dev2
- 1.0.0
- 1.1.0
- 1.2.0
- 1.2.1
- 1.3.0
- 2.0.0
- 2.1.0

To summarize: ServiceNow application files cannot be merged in a Source Control System and hence any method involving feature branches (e.g., gitflow) is not an option. To allow for selective feature deployments and the ability to always provide new application versions from a “dev” branch, features must eventually be turned off and treated as technical debt.

The branching strategy can hence be described best as “commit at head” as all changes are to be committed to the “dev” branch as soon as possible – no matter if the related work is completed.

Application Version Baseline Procedure

After a sprint, or whenever a new feature has been completed – or a new version of a dependency application should be deployed – it may be a good point in time to release a new version of an application.

So, whenever the development team considers the state in the application's "dev" branch ready for a deployment, the following activities should be conducted:

1. Run the CodeSanity check on the app. Ensure the app has no findings. Fix findings if necessary and start over.
2. Run all available ATF tests – and fix issues if any tests fail and start over.
3. Insert or update technical debt records and adapt switches to turn features on or off to match the product owner's expectations and decisions regarding
 - a. Unfinished work
 - b. Features not (yet) to be released to users
 - c. Deprecated features
 - d. All other forms of technical debt
4. Insert or update dependencies of the app (i.e., update the minimum version requirements).
5. Document all changes relative to the previous version in the application's release notes.
6. Review and update feature descriptions in the application manual
7. Set the application version in the application settings (if not done already).
8. Commit all uncommitted changes to the "dev" branch.
9. Click on the "Baseline" UI action on the application form (if the DevTools open-source application is installed, otherwise the following tasks must be performed manually):
 - a. Repair several Application Files which may be modified by the users or even by the platform by mistake.
 - b. Commit all resulting changes to the "dev" branch.
 - c. Create branch and name the branch exactly like the version of the application (e.g., "1.5.0").
 - d. Switch back to the "dev" branch.
 - e. Add the postfix "WORK IN PROGRESS" to the application name (to remind developers that the state of the code they work with is NOT YET the version currently associated with the application)
 - f. Create a new section in the application's release notes (if they exist in the application manual UI page)
 - g. Increase minor version number (and optimistically assume that the next version will deliver a backwards compatible new feature)
 - h. Commit all resulting changes to the "dev" branch.
10. Lock the new branch in the source control system

Performing all these steps manually is obviously laborious and prone to human error. It is recommended to automate as many of the steps as possible. To do so, the following technical capabilities need to be build – as these are not provided OOTB:

- Provide a user interface to trigger the baseline procedure
- Ensure that the application is ready for baseline
- Clean up application files and remove any unintended files
- Commit pending changes, create the new version branch and switch back to the “dev” branch
- Increase the version number
- Add a section to the release notes
- Lock the new version branch to prevent further changes

The open-source applications “DevTools” and “Deployer” support all the required technical capabilities.

Deployment Automation

Each deployment of an application version to a downstream instance should be conducted exactly according to the following procedure in exactly this order:

Automated steps:

1. Establish the dependency list based on the requested application version
2. Check the availability of the requested application version branch in the Source Control System and all application version branches of the dependency applications
3. Check which application versions need to be deployed to the target instance
4. Check if there are any version incompatibilities between the to-be-installed versions and the already installed versions on the target instance
(e.g., if a version 1.5.0 is to be installed but version 2.0.0 is already installed – this would imply a version incompatibility, and the deployment cannot be done)
5. Deploy all applications (in the order of the “dependency list”) to the target system
6. Verify compliance with the Coding Guideline on all deployed applications using Instance Scan
7. Run installation scripts (in the order of the “dependency list”)
8. Run all automated tests of all deployed applications (in the order of the “dependency list”) – but only on non-production instances (if there is a “Platform” application available, run all automated tests based on the dependency tree of the “Platform” application. Refer to chapter “Application Architecture and Dependency Management”)

Manual steps:

1. Performing a smoke test
2. Optional: Clone production instance to HOTFIX instance

To perform these steps, the following technical capabilities need to be built – as these are not provided OOTB:

- Establish the dependency list based on version branches in the repositories
- Deny any deployments in case circular dependencies or missing version branches for required applications
- Identify all tasks necessary to conduct the deployment – including installing applications from source control, running Instance Scan checks, and running ATF test suites (on non-production instances) – and execute these tasks automatically
- Run all installation scripts remotely on the target instance
- Ensure that there is only one deployment taking place on one target instance at any given point in time (otherwise a mix of different or even conflicting version states may temporarily impair the results of automated tests being run in parallel)
- Warn users of potential data deletions if an application no longer contains tables or columns in other tables which are present on the target instance

The open-source applications “DevTools”, “Deployer” and “CodeSanity” support all the required technical capabilities.

Technical Debt Management

A technical debt is any state in a software module that has a negative impact on its maintainability and requires future work but does not prevent production use.

What exactly should be considered as technical debt is certainly subjective. Here are some examples that can be considered as technical debt:

- Dependencies to outdated or even deprecated libraries or components
- Duplicate code
- Dead or unreachable code
- Incorrect or misleading names of database entities (tables) or fields
- API naming inconsistencies (e.g., functions "GetInstanceURL()" vs. function "GetRecordUrl()" – note the different spelling of "URL" vs. "Url")
- Non-compliance with the coding guideline
- Insufficient or missing documentation
- Insufficient test coverage
- Deprecated features
- Unfinished work
- Fully operational features not yet to be exposed to users

For disambiguation: Sometimes the term technical debt is used in publications to criticize excessive customization or deviation from OOTB configuration. The definition used in this document is more specific (and less judgmental).

A lot has been said and written about the long-term costs and effects of technical debt. For the context of this process, there are two main take-aways:

- Anything that is identified as a technical debt must be represented by a backlog item and treated like any other item that represents future work - e.g., a business requirement – apply the boy scout rule – not all technical debt is known (or even considered as such) when it appears – the whole team should always document technical debt when they see it
- Technical debt should be documented as part of the application documentation which ideally is part of the application itself (e.g., as a UI page).

From the above list there are two special forms of technical debt: Unfinished work and features not yet to be exposed to users. One might argue that these have no negative impact on the long run. One may say: the opposite is true. But only after a feature is completed, or a completed feature is finally exposed to users, will it generate value to the business, and not just incur costs.

However, in the context of this process, it is pragmatic to treat such cases as technical debt, because they need to be managed in the same way. Unfinished work obviously requires future work and a completed feature that does not provide value to users still needs to be maintained until it is released – which again may imply additional costs – although hopefully not for a very long time.

The cases of unfinished work and completed features not yet to be exposed are specifically important. The guidance in this document is to commit all work into the development branch, as early and as often as possible. This leads to a state in which the development branch may contain unfinished code or application files at the time of the next version baseline.

Since the advice is not to use selective commits or feature branches (see “Branching Strategy”) we need a technique to manage unfinished work (see chapter “Managing Unfinished Work”).

The following outlines the life cycle of technical debt:

1. A technical debt is identified – either by the development team, a product owner, or any other stakeholder
2. The technical debt is documented in the application documentation
3. The technical debt is added to the backlog
4. If applicable a switch is introduced to render parts of the application inactive or at least invisible to users. The switch is set to whatever is appropriate at the time.
5. The affected application is baselined any number of times with the set switch state
6. Once appropriate the switch is flipped and the feature becomes visible (or invisible) depending on stakeholder’s decisions
7. The affected application is again baselined any number of times
8. The technical debt is resolved, the switch is (and eventually the feature) is removed from the application

To perform Technical Debt Management on a day-to-day basis, the platform must be extended with the following capabilities:

- Document technical debt of an application in a way that these records are part of the application so that they are also stored in the application’s source code repository
- The technical debt records must be stored in a way that they can be retrieved and processed from a source code repository
- Documentation on technical debt can be generated automatically based on these records
- Each technical debt record has a switch that can be turned on and off
- A function is available to request the state of a technical debt switch – to control application behavior depending on the switch state

The latter two requirements are crucial for cases of unfinished work and features not yet to be released to production.

These requirements are not met by the ServiceNow platform OOTB – hence these additional capabilities must be implemented.

The open-source application “DevTools” supports these capabilities.

Hotfixing and Backporting

A hotfix is the resolution of a defect which is applied to an application version that is already deployed to a non-development environment – i.e., to a test or even production instance.

Backporting is the process of implementing the same or a comparable resolution to the current state of an application in a development environment.

When looking at the quality gates described it seems as if the delivery of new applications is a one-way street: development of a new feature or a bugfix starts in a development environment. These new features and bugfixes are then shipped to test and later to production environments as part of a baselined application version.

But what if a defect is identified in production? The defect may be severe and may cause incidents. Users may be affected, and a business process may come to a halt.

A fast solution is required.

Two scenarios are possible:

1. The application has not changed significantly in the development instance since it was last baselined, tested and deployed to production.
Any changes made in development since the latest released version can all be set inactive (refer to chapter “Managing Unfinished Work”).
The application can be tested with minimal effort – and most of the relevant test cases have been automated.
Only few or no dependencies exist.
2. It's complicated

In scenario 1 a bugfix can be implemented in the development environment, a new application version can be baselined, deployed to the regular test environment, tested, and deployed to production – all of that within an hour or even less. However, this is not how the real world looks like most of the time.

Scenario 2 is the one we need to look at.

- What if the application has changed significantly since the last version baseline?
- What if multiple applications need to be changed to apply the bugfix?
- What if other applications depend on these applications?
- What if one or more affected applications are being worked on now and a lot of unfinished work has been produced and committed?
- What if one of these applications is even backward incompatible to the version in production?
- What if multiple versions of these applications have been baselined in the meantime and the state in development differs significantly from the version currently in test and the version in test differs from the version in production?
- What if resolving the issue in production would take days or even weeks if the regular process is followed.

This is when the **Hotfixing and Backporting procedure** comes into play:

Please note: the "HOTFIX" instance is the instance that is used to produce the hot-fixed application versions (refer to chapter "Instance Setups" for details).

1. A defect is identified in production
2. Optional: a clone is made from production to the HOTFIX instance (ideally this is done after each production deployment)
3. Verify all application versions installed in the hotfix instance match the application versions installed on production (just in case a deployment took place after the most recent clone)
4. The defect is reproduced in the HOTFIX instance
5. The affected applications are identified
6. A decision is made whether the defect can be resolved according to the regular process – in that case the procedure stops here, and the regular process is applied. If in doubt, continue with the "Hotfixing and Backporting" procedure.
7. For each of the affected applications, a new branch is created named "hotfix"
8. The defect is fixed in one or more applications in the HOTFIX instance
9. Optional: ATF tests are created to verify the resolution of the defect
10. Testers and other stakeholders test the fix and perform a regression test on the HOTFIX instance – including running all ATF tests
11. The resolution is confirmed
12. New application versions are baselined directly in the HOTFIX instance – however instead of the "dev" branch, the "hotfix" branch is used – refer to chapter "Application Version Baseline Procedure". This includes running all ATF tests and Instance Scan checks.
13. The new version branches are directly deployed to production (not to the TEST environment as it may be in a state that is too different from production to get reasonable results)
14. A smoke test is performed on production
15. Stakeholders confirm the resolution of the defect
16. The defect resolutions are backported to the current state of the affected applications in development (see chapter "Backporting")
17. ATF tests are also backported to the affected applications in development or new ATF tests are introduced to verify the resolution of the defect
18. Optional: New Instance Scan checks are introduced to prevent similar mistakes in the future if applicable
19. If the affected applications are being developed in multiple development environments – e.g., because an application must be worked on in the current and a future major version at the same time – the resolutions and ATF tests need to be backported to these future development instances as well

It is advisable to configure the Backlog Management system to support this procedure in addition to the regular process – as the steps are significantly different.

Further it is advisable to identify a few individuals in charge of managing this procedure, to conduct specific training and perform simulations. The procedure is time critical and will likely not be used very often – so the team's capability to perform this procedure must be kept alive proactively.

For technical considerations on backporting, please refer to chapter "Backporting".

In theory, the procedure may also be applied to defects identified during a regular test conducted on a test environment. However, if a team decides to apply the procedure in that case, this should be seen as a symptom of yet another severe problem: test cycles are obviously too long!

Release Notes

No one likes to write documentation. Or are you? Who reads that anyway?

Wherever the creation of documentation can be automated, based on the artefacts in the platform, that is great. However, there is one form of documentation that a human being will need to write – unless we have even better versions of generative AI – and these are release notes.

Release notes describe the changes made between the last baselined version and the current version.

Each application must have release notes. Within a development eco-system, release notes are ideally all at the same place or follow a common pattern so that they can easily be found by other developers, product owners, operations, and other stakeholders across teams.

But why are they so important?

Because others may depend on it. Literally. If application A has a dependency on application B, application A must specify the minimum version requirement for application B. If new features are added to application B on which application A depends on, the developers of application A must update their version requirements.

Some developers are lazier than others. The laziest developer might think: "I just update all the minimum version requirements to the current versions found in the development instance. This way I always depend on the latest versions, and I do not have to read release notes or communicate with anyone."

If everyone would behave that way, no application could ever be deployed after baselining. Any new baselined application version would then depend on various other application versions that are not yet baselined and hence it would not be deployed unless all other developers provide newer versions of their applications to the source control system.

When baselining an application version, the developers need to update the **minimum** version requirements to their dependencies, but they should do it wisely. To identify the minimum version requirement (and not just the latest) they need to know which capabilities already existed in versions baselined earlier – and the single source of truth for that are (or should be) the applications' release notes.

Developers are well-advised to write release notes immediately when they finish the work on a change to their applications – the text written for the release notes can also be used as part of the commit message – laziness may work in our favor after all.

See chapter "Determining Minimum Version Requirements".

Backlog Management

"Simple, clear purpose and principles give rise to complex and intelligent behaviour. Complex rules and regulations give rise to simple and stupid behaviour."

Dee Hock, the Hock Principle

Every development and deployment process requires a backlog management system – a tool where a team can track the requirements and the progress of their work. Chances are high that your organization already has one – or many.

Some use Jira, others use GitLab, Excel, paper sheets on a wallboard or ServiceNow. No matter which tool you use, to support this process, some configuration is required.

In some organizations, this is a challenge. If many teams are using a single instance of a backlog management system, the department that operates the system tries to standardize the workflows across the organization and avoids customizations for individual teams or projects.

Seen from a high altitude that makes perfect sense. From the perspective of a team that tries to get work done in the most efficient way - this is just a headache.

If you are one of the fortunate and you have a say in how your backlog management system should be configured, you may find this guidance helpful.

The backlog management system should support the following states for backlog items:

1. **Draft / Need more info:** The requirement description is being worked on or more information is required to implement the backlog item.
2. **Ready:** The author of the backlog item claims the item contains enough information to start development. A developer should be assigned to the backlog item to verify that sufficient information is contained in the backlog item.
3. **In Development:** A developer works or is asked to work on the implementation of the backlog item in the DEV environment.
4. **Done / Committed to repo:** The developer has completed their work and committed their changes to the source code repository - which makes the backlog item part of any subsequent deployment to the TEST environment.
5. **Baselined:** The team has baselined all related applications that were affected by the changes made for this backlog item - which enables the backlog item to be part of a subsequent deployment to the TEST environment.
6. **In Test:** The changes related to the backlog item have been deployed to the TEST environment and a tester is asked to test the result.
7. **Test Failed / Showstopper / Back to development:** A backlog item has failed the test and must be considered as a showstopper - so the current version on the TEST environment **MUST NOT** be deployed to the PROD environment. The backlog item is sent back to a developer who is now asked to fix issues. It is important to distinguish this state from "In Development" because the current deployment on TEST should not be deployed to the PROD environment if there are showstoppers. If a test failed but the issues are not considered as showstoppers, the version can be deployed to PROD, and the state should be set to "In Development" which indicates that they do not represent a reason to delay deployment to PROD but still need to be worked on.

8. **Baselined Showstopper:** A fix has been provided to resolve the showstopper. All relevant application versions have been baselined and are now ready for deployment to TEST.
9. **Test Passed / Ready for PROD:** The test has been passed, and the backlog item is ready for deployment on the PROD environment.
10. **Review on PROD:** Deployment to PROD environment has been completed. A final check should be done before the backlog item is marked as "Completed".
11. **Completed:** The backlog item is finally completed.
12. **Cancelled:** The backlog item will not be implemented or worked on any further.

Ideally each backlog item, once it enters the state "Baselined", should be linked to an application version – as some state transitions for one backlog item might imply the state transition for all other backlog items that are covered in an application version.

It is extremely helpful – although that is not mandatory – if the backlog management system also contains a list of personas that can be associated to backlog items. This allows us to search for backlog items that refer to specific personas and help testers to align their activities when testing multiple backlog items.

The backlog management system should also have a list of components which can be associated with a backlog item. A component can be a scoped application but also other things like the mid server, an external mail server, documentation assets, etc.

That makes it easier to group and filter for backlog items relevant to a component.

If your backlog management system is also able - based on default settings for each component - to assign developers and testers – either individually or as a team – depending on the state of the backlog item, that is even better.

The backlog management system should support dependencies between backlog items – to indicate that some work needs to be done, before other work can be completed, too.

These dependencies should be visualized so that the teams can oversee what work is yet to be done and in which order.

The backlog management system should also maintain a definitive order for each backlog item. This order should not allow to set any two backlog items at the same position – after all one backlog item must always be more or less important than another.

The order of backlog items must also respect their dependencies. If one backlog item depends on another, the backlog item that must be completed first, must always have a higher priority position than the one that depends on it.

All these capabilities can be built in systems like Jira or ServiceNow (e.g., using the "Agile Development" product) – but this requires significant changes to the OOTB configuration.

The open-source application "Agile" supports all these required capabilities.

However, the "Agile" app does not support sprints and epics – and this is a conscious choice.

Whether the idea of "sprints" really add value to a development process is debatable. If you read this, you are likely open to radical ideas, like organizing your development team not in sprints but based on a prioritized backlog only.

Even more debatable is the idea of a category system that distinguishes between epics and stories. Many teams struggle with the question on whether a backlog item is an epic or a story,

how large stories may be until they should be split into smaller stories – and whether this turns the original story into an epic and if they should have sub-epics or sub-stories or story-tasks. Many teams have come to the point where they gave up and realized that any multi-layer category system just follows the golden rule of category systems: No matter how it is designed, it turns out to be wrong at some point.

After all, it does not make any difference whether you call backlog items “backlog items” or “stories” or “issues” or “tasks”. The author of this document believes that the popularity of the term “story” results from a misconception about agile methodologies mixed with the idea of “user stories” to describe requirements - which influenced the terminology used in Jira.

As Jira became so popular amongst many software development teams, Atlassian (the creators of Jira) coined the term “story” as a category of tickets. This has led to a widely adopted industry standard – which is good – everyone knows what is meant if someone says “story”. But this also caused massive confusion and waste.

The waste is caused by the endless debates and negotiations about the right category of “something” – and even worse: if a decision is made to re-categorize a ticket this usually results in significant copy-paste work (e.g. from “story” to “epic” or from “epic” to “feature” etc.)

A backlog item is agnostic to that. Any backlog item in the “Agile” app can depend on other backlog items – and hence form a tree of related backlog items that can be put into a definitive order.

At the end of the day, we all do tasks one after the other. No matter what we call them.

The main takeaway is that the Backlog Management tool should be configured to reflect the states described to support the process in the best way possible.

Instance Setups

Any instance setup that consists of at least three instances can support the proposed process – however to allow parallel version development and independent hotfixes, a more complex setup is required.

But first things first: here is a taxonomy for instances:

DEV The main development environment where the next version of an application is being worked on.

DEV2 A secondary development environment where a future version of an application is being worked on. Working on DEV2 should be the exception. Changes made in DEV should be deployed as fast as possible – to reduce the amount of finished work waiting for deployment (which may even have a negative CAPEX effect).

However if backward incompatible changes are required, these changes may require working on them without interfering with current development. (also refer to chapter “Managing Multiple Development Streams”)

TEST-AUTO The test environment on which ONLY automated tests are performed – this includes Instance Scan checks and ATF tests. An application version that does not pass all tests on this instance should not be deployed to any downstream instance. A TEST-AUTO instance mainly protects the TEST environment from too many – unnecessary – changes. Especially in environments with large test teams this allows for better planning and more stability in the TEST environment.

TEST The main test environment where testers and stakeholders perform manual tests and confirm that the installed release package is fit for purpose.

TEST2 A secondary test environment to confirm fit-for-purpose of new future versions – which are typically developed on a secondary development instance, and which may have backward incompatible changes to their current major versions.

PRE-PROD An instance that is as close to the production instance as possible to verify the deployment of a release package before deployment to production.

HOTFIX This environment is kept in sync with production and is on stand-by to allow the development and test of a hotfix version of an application version that is already deployed to production. Ideally the HOTFIX environment is cloned from production after each production deployment.

PROD The production instance where regular users perform their work and value is generated for the business.

CONTROLLER If possible any ServiceNow-based pipeline automation tools do not reside on a development, test or production instance. Instead, these tools ideally run on a separate instance. That however may only be an option for larger clients who maintain multiple production instance stacks.

These names can be referred to as the “**pipeline names**” of the instances (as opposed to their technical or domain names).

The following chapters propose three different instance setups with varying complexity (and costs).

Default

3 instances: DEV, TEST and PROD

This is the typical setup for many ServiceNow clients. It does not allow for parallel development of future versions and does not support hotfixing.

This makes the overall process less sophisticated but if the number of dependencies is low and complexity of application changes is low, this setup is sufficient.

All automated tests should be carried out on the TEST instance.

Balanced

5 instances: DEV, TEST-AUTO, TEST, HOTFIX, PROD

This setup adds the hotfixing and separate automated testing capability to the process. Since the setup allows for quick defect correction on PROD without interfering ongoing development and testing work – the setup is more robust and recommended if the frequency of deployments is low, but the complexity of the changes is higher – which makes it more difficult to baseline new application versions from the current state on the DEV environment.

Juggernaut

9 instances: DEV, DEV2, TEST-AUTO, TEST, TEST2, HOTFIX, PRE-PROD, PROD, CONTROLLER

This setup supports all aspects of the process: parallel development on multiple major versions of applications (for backward incompatible changes), parallel testing operations, hotfixing and an additional safety layer through an additional deployment test on PRE-PROD.

This setup is recommended for highly complex application eco-systems with numerous dependencies between these applications and a mix of applications that are changed at a low or high frequency.

Note that the ServiceNow-based pipeline automation tools run on a separate, dedicated CONTROLLER instance.

The request may look excessive and depending on the commercial agreement with ServiceNow may appear unrealistic to many customers. The author has seen such setups in real-life, and these were built and run for good reasons.

Making a Choice

The following overview should help to decide which setup fits best:

	Default	Balanced	Juggernaut
Required Instances	3	5	9
Deployment Frequency	Multiple deployments per hour	Multiple deployments per day	Multiple deployments per week
Separate automated testing	NO	YES	YES
Hotfixing and Backporting	NO	YES	YES
Simultaneous development on future application versions	NO	NO	YES
Additional deployment verification	NO	NO	YES
Dedicated instance for pipeline tooling	NO	NO	YES

It may appear counter-intuitive to operate more instances for deploying less frequently. This is less about what the teams intended to do, but what happens. More instances are required to perform more intense testing which reflects more complex deployments of more apps with more dependencies. As complexity grows, deployment frequency get lower.

Yet, multiple deployments per week, however, is a frequency many organizations dream of today.

Considerations for the (missing) CONTROLLER instance

Maintaining a separate CONTROLLER instance to operate any ServiceNow-based pipeline tooling may not be an option for all clients – for various reasons – e.g., the associated license and infrastructure costs.

However, if an organization yet decides to use ServiceNow-based tools to automate their deployments or manage the process (e.g. by using the “Deployer” and/or “Agile” apps), these apps need to be installed and run somewhere.

The question is, where – if there is no dedicated CONTROLLER instance available – should these apps be running?

Unfortunately, there is no satisfying answer.

If pipeline tools run on DEV, the DEV instance may need to interact and perform impactful operations to the PROD environment. This may not be compliant with an organization's security rules. The same may be applicable to TEST environments.

An advantage (or disadvantage) of running pipeline tools on DEV is that it can be maintained and used directly by developers (who may not have access to PROD directly).

If pipeline tools were run on PROD these tools would perform impactful operation on the very same instance. The risk is in fact not that high, but this may also conflict with technical governance rules of the organization.

An advantage of running pipeline tools on PROD is that its use can be controlled by the actual access management mechanisms and that it can be tightly integrated with other IT processes (e.g. Change Management) performed on PROD.

Deployment Variants - Tankers, Speedboats and Lifeboats

Some changes to the code are bigger and more complex, these require intense testing and preparation, other changes may be small and independent. Some may be extremely urgent, as they are needed to resolve issues directly impacting users.

A mature development and deployment process must be able to support all possible scenarios.

To describe these different scenarios, a maritime metaphor is used: Tankers to represent the slow-moving complex releases, Speedboats are small but fast releases, and Lifeboats for urgent ad-hoc emergency (i.e. hotfix) releases.

Regular Releases (Tankers)

Tankers follow a defined route. Loading takes its time, they leave the port in a controlled, planned manner, ship over the seas in predictable routes, and usually arrive on time. Tankers are not necessarily fast, but what they lack in speed, they compensate in terms of efficiency. There is no other way to transport such huge loads given the invested resources.

Tankers represent the "regular" release – which may happen once a quarter, once a month, once a week – depending on the organization's pace and appetite for change.

The development of a regular release is done in the development environment. Once all affected applications are baselined, a release package is deployed to TEST environments. The development team may then provide one or more updated application versions to fix defects revealed during the test and then a fully validated release package is installed to the PROD environment.

Ideally the release package is represented by a version of the Platform App (as described in chapter "Application Architecture and Dependency Management").

Interim Releases (Speedboats)

Waiting for the tanker to arrive, and the next tanker to be prepared might not be fast enough in some cases. A new capability or bugfix may have to be deployed faster. A speedboat can easily be prepared, leave port, and pass by a tanker on the open sea, and arrive at the destination earlier – and multiple speedboats may follow suit.

A single application version could be prepared after an earlier version has been made part of a release package. This newer version is then deployed to the TEST environment, be tested separately, and validated for production use.

However, one condition must be met for a speedboat release:

The application version does not depend on any capability that is "loaded on the current tanker" – so the dependency requirements must be met by the current state in production.

This "independency" from anything that is just part of the current release package in TEST is yet difficult to prove. The speedboat application version is tested (in TEST) under different conditions than it will meet in production.

It is certainly faster taking the speedboat, but the open sea is more dangerous for smaller vessels.

The higher speed is gained by taking higher risks.

Development teams must have a good understanding of an application's dependencies and should be able to assess the risks associated with a speedboat release.

Like regular releases the speedboat application version(s) are created in the development environment and deployed to TEST for testing.

Speedboats and tankers follow the same route, but at a different speed.

Hotfix Release (Lifeboats)

In case of emergency, the only option may be to enter the lifeboats to save passengers and crew. Obviously, this should be an exception on the open sea. Lifeboats are designed to save people from drowning, and they can provide shelter for a while. It is not a real solution though; other ships must come to the rescue.

A hotfix can only be duct tape until the next release is deployed. A hotfix is created, when an urgent fix is needed in PROD, and the state of the affected applications in DEV and TEST are already out-of-sync with PROD – which means the development state either does not allow a timely preparation of a version baseline or too many changes have been made that prevent a fast interim release (Speedboat).

A hotfix is prepared outside the development environment on a separate HOTFIX (or PRE-PROD) instance – where the state of the installed applications matches PROD.

Hotfixes bypass the usual process, as they are not created on DEV, nor deployed to TEST.

This ultimate speed comes at a price. Just like lifeboats, hotfixes can only be a temporary solution. Whatever change has been made in a hotfix application version must be backported into development, and any pending tanker or speedboat may have to wait until the next regular or interim release is prepared that contains the same or adapted fix for the issue.

Rollbacks (Life Vests)

Swimming in the open sea is not fun. It is deadly. The water is cold – freezing cold. The salt burns in the eyes and when swallowed causes nausea. Waves make it difficult to keep one's head above water. A life vest can buy a few more minutes to reach the nearest lifeboat – not more, not less.

Performing rollbacks is like jumping off the ship with only a life vest – extremely risky. Rollbacks should be the last resort – the ultima ratio – the last of all option - if all else was ruled out.

Some good reasons not to roll back:

- After a release to PROD data is processed based on changed logic. Reverting to an earlier version may combine older logic with newer data structures – which may cause even more problems
- If the data model has changed during the most recent release, reverting to an older version of an application may delete new fields and tables – which would cause data loss
- Assessing the impact of a rollback is extremely difficult – and to do it right may require as much time as it would take to prepare a hotfix – so the potential advantage in terms of speed is questionable
- The more time passes after the latest release the more dangerous is the rollback – as any data-impact of the rollback will get bigger over time

Even considering a rollback puts a team between a rock and a hard place. Any risk mitigation - which takes time – increases the risk.

Yet, many organizations demand operations teams to prepare for that scenario – and will not accept the hotfix option (even when backed up with a dedicated instance) as the only fallback option if a release goes south.

For most teams, dealing with an emergency that requires immediate response in form of updated software is a rare occurrence – a black swan event.

That means that most teams do not have the routine and discipline to conduct such emergency responses properly – without causing more harm.

Given all these constraints, it becomes obvious that performing emergency responses – that includes interim releases (Speedboats), hotfixes (Lifeboats) and Rollbacks (Life Vests) – must be well-prepared, trained and rehearsed by the team.

At the end of the day the risks of having only limited means to respond (e.g. only considering interim releases) must be balanced against the costs of maintaining the additional infrastructure (e.g. a HOTFIX instance) and the additional effort required for training and exercise to perform hotfix releases or even rollbacks.

Techniques

Scoped and Global Applications – and App Customizations

The most critical technique (to be learnt) is to ship all features, all customizations, all changes, as an app – and the scoped app is the preferred choice.

Before we can investigate how that works, a myth must be debunked: Yes, scopes have been introduced as a security boundary – to limit access to sensitive data to a specific set of users. But scopes have evolved into something different. With the introduction of source control support, scopes are in fact a packaging boundary. A source control system's repository would contain all application files associated to a specific application – hence the role of the scope as a security mechanism has been superseded by the role of a packaging mechanism. For the means of the process described in this document, a scope cannot be both. Its role as a security mechanism for custom applications must be ignored, its role as a package mechanism must be pronounced.

The chapter “Cross-Scope-Scripting” contains the details on how to “ignore” the security aspect of scopes.

The ask to ship everything as an app is less of a statement what to do – it tells us more about what to avoid as part of a deployment:

- Capture and deploy changes in Update Sets
- Changes to OOTB records (other than through Installation Scripts)
- Changes to records in plugin or store application scopes (other than through Installation Scripts)
- Manual XML exports and imports of any sort
- In fact, ANY manual activity as part of a deployment

Instead, all necessary changes should be shipped as part of an application.

“Application” can mean:

- Scoped Applications
- Global Applications

For the sake of completeness, also “App Customizations” should be mentioned. “App Customizations” allow to make customizations – i.e. add and modify records in a store app or plugin scope and capture these customizations in a source control repository.

The App Customization is always bound to a specific version of the underlying customized application – that prevents it to be installed over an incompatible version.

However, using App Customizations comes with several downsides:

- App Customizations are not applications – i.e. they are NOT derived from sys_app – that means that any technical capability around applications that is required to support this process would have to be created in two flavors: for genuine apps and for app customizations (e.g., dependency management, installation scripts, deployment automation etc.).
- App Customizations are “highlanders” – there can only be exactly one per scope. If different capabilities are to be built “on top” of a plugin, all of these must be implemented in the same App Customization – this reduces flexibility and limits options on how to design the overall application architecture.
- When upgrading the underlying plugin or store app any customizations must be migrated (pretty much like any change to OOTB records) which can be a laborious and time-consuming process.

With all these downsides, using App Customizations should be limited to cases where any other form of shipment for whatever reason is downright impossible. It should be the ultima ratio. A scoped application with installation scripts acting across scopes any OOTB record can be deactivated and replaced by a modified copy shipped as part of the scoped application.

	Scoped Application	Global Application	App Customization
Scope and namespace	Application specific	global	Same as customized plugin
Supported in source control	YES	YES	YES
Namespace	All Application Files contained in the application share the same namespace	All contained Application Files share the same namespace as all other records in the global scope and those contained in other global applications	All contained Application Files share the same namespace as the records in the plugin that is customized
When to use	Always as a default	ONLY as a fallback	ONLY if complex customizations already exist

Generally, scoped applications are ALWAYS the best choice. Global applications or customization applications should ONLY be used if complex changes to records in global or plugin scopes have already been made at the time of the introduction of this technique. In this scenario the efforts and risks of refactoring the same features into scoped applications may just be too high.

The following cases should be addressed through installation scripts:

- New or modified ACLs are required where scope boundaries prevent their use
- New records where a requirement exists that these records must be in a specific OOTB scope (global or plugin)

If many records are affected and such changes are very complex, consider the technique described in chapter "Data as Code".

For detailed guidance on how to make customizations to OOTB records and plugin scopes, refer to chapter "Refactoring Customizations into Applications".

In summary: scoped applications are always the best and preferred choice to transport changes of all kinds, including changes to records in other scopes.

One application making changes to other applications may appear risky and uncontrollable. As these changes are part of an application's defined and repeatable behavior - which can be validated and tested - the risk is acceptable.

This implies a fundamental change in how many changes are made to the ServiceNow platform. The implications (both technical and organizational) of this guidance should not be underestimated.

Application Architecture Design

Implementing the idea of shipping everything as an app leads to more questions in practice. These are different questions than most ServiceNow developers had to answer in the past.

The most important question regarding the application architecture is: what capability, what “file” should be in which application?

How should the superset of “everything” be distributed into applications?

The challenge is there is no simple answer for that and certainly no one-size-fits-all approach. As so often in software development – or any other complex challenge: it depends.

Architecture in this context means to solve an optimization problem with conflicting goals:

1. Applications should be small – so that they are easy to understand and to maintain
2. The number of applications should be limited – so that it is easier to keep track of them
3. Applications should have as few dependencies as possible – so that they can be changed and deployed independently
4. Code duplication should be reduced – so that a defect does not have to be fixed in multiple places

The conflicts are obvious:

- Smaller applications lead to more applications, which leads to more dependencies, which leads to more complex deployments
- Larger applications lead to artefacts that are more difficult to maintain
- Applications maintained in different teams lead to more code duplication (yes, it does)

There are some guardrails that can help make these decisions:

1. One team (within a larger development department) should be responsible for one or more applications
2. Applications should only serve a single purpose or “concern.” Examples are:
 - a. One process (that applies to both new custom processes as well as when extending OOTB processes like “Incident Management”)
 - b. A cross-process or even cross-platform capability like “base data management” or “data segregation”
 - c. A specific user interface like a workspace for the ServiceDesk, a Portal for end users, a mobile user interface for a specific purpose
 - d. One external system to connect to (A “Spoke” or “Integration” in ServiceNow terms)
 - e. Re-usable components for a specific purpose

And there are special applications that further help structuring the application architecture:

1. The “Platform” app – the app that no other app depends on, but which depends on all other apps – this should be the one and only app that describes a deployment package
2. The “Core” app – the app that all other apps may depend on because it contains re-usable components specific to the instance.

The key element to this technique is that decisions on how to distribute capabilities into applications – and hence when to create new applications and how to name them – are made consciously and not ad-hoc. Any product owner may describe a new requirement and may have the power to set priority for it. But this should not automatically lead to ad-hoc decisions regarding the application architecture. Every developer should be aware of the need and value of making conscious and considerate decisions and discuss the options with peers (or escalate the case to the architects) before just starting the implementation.

The Communities of Practice are an excellent forum to discuss and make such decisions. Discussing and deciding on such matters publicly fosters awareness and knowledge alike so that the likelihood of better decisions by every member of the team is increased over time.

Application Naming Convention

Any naming or category system will turn out to be insufficient at some point in time. Yet, no naming convention is even worse – so teams are well-advised to agree on a naming convention early on – and once its deficiency is obvious, adapt and refactor.

Names are not just identifiers, they also expose culture, tradition and mental models – hence defining and agreeing on a naming convention is inherently difficult.

The following proposal should be seen as a starting point for discussion. It is not the single source of truth and should not be considered as prescriptive guidance without conversation and review.

An application naming convention should cover 3 items:

1. Application display names
2. Scoped application scope names
3. Global application technical prefixes

The application display name is the user-visible name of an application – it applies to both scoped and global applications.

The scoped application scope name – as the name suggests – is only relevant for scoped applications. The scope name is used as a prefix for technical table names, as a namespace identifier when calling functions or classes in Script Includes from outside the scoped application and various other places. The scope name of custom applications by default starts with "x_" followed by a customer specific identifier, another underscore and then a developer-provided postfix. The total length of the scope name (including the leading "x_" and company identifier) must not exceed 18 characters – which requires the use of abbreviations.

Not that the current version of ServiceNow Studio does not even allow to edit the scope name during application creation – which requires the scope name to be modified using a script after creation.

The global application technical prefix is not a technical feature of the platform – it is a recommendation. The technical prefix should be used e.g. for Script Include names – and hence should be useable as a JavaScript object name prefix (i.e. should only contain capital and lowercase letters, must not start with numbers and should not contain special characters). This is to avoid name clashes with other application files in the global scope.

The following patterns are proposed here:

Application display names

"<Company>[<Project>][<Process>][<Type>][<Context>][Global]"

- <Company> - The name of the customer
- <Project> - Optional identifier of a customer has multiple, independent instance stacks for different organizational units or use cases
- <Process> - the name of the process (or business capability)
- <Type> - The type identifier
 - "Platform" – Platform apps (see chapter "Application Architecture and Dependency Management")
 - "UI" – User Interfaces
 - "INT" – Integrations to external systems (inbound and outbound) – but ideally not both in one app
 - "Core" – Contains components to be used by other applications
 - "" – No type identifier indicates an application that implements a process (or business capability)
- <Context> - Used for "UI" and "INT" applications to specify exactly which user interface or integration is implemented in this application
- <Global> - is a postfix to mark global applications

Scoped application scope names

"x_<company abbreviation>[_<project abbreviation>][_<process abbreviation>]<type>[_<context abbreviation>]"

- <company_abbreviation> - 2-4 letter abbreviation of the customer name – defined by ServiceNow
- <project abbreviation> - Optional 2-3 letter abbreviation of the project identifier
- <process abbreviation> - Optional 3-4 letter abbreviation of the process (or business capability) name
- <type> - Same as for the application display name (but in lowercase letters)
- <context abbreviation> - Option abbreviation of the context name using whatever space is left within the 18 characters constraint

Examples

- "ACME Incident Management" / "x_acme_inc"
ACME's scoped app that defines customizations to the Incident Management Process. The fictitious ACME company in this example does not distinguish between different projects.
"Incident Management" is the process name, abbreviated by "inc".
For processes the type and context is blank.
- "ACME Incident Management Global"
This global application acts as a sidekick to "ACME Incident Management" which contains additional records related to the Incident Management process which for whatever reasons must be in the global scope.
- "ACME UI Service Portal" / "x_acme_ui_sp"
ACME's scoped app for Service Portal customizations.
This fictitious Service Portal covers many processes and is hence not process specific, so the process part is blank.
The type is "UI", the context is "Service Portal".
- "ACME UI Service Portal Global"
This global application acts as a sidekick to "ACME UI Service Portal" which contains additional records related to the Service Portal which for whatever reasons must be in the global scope.
- "ACME INT Jira" / "x_acme_int_jira"
This scoped application contains the integration with ACME's Jira instance.
Hence the type is "INT". "Jira" is the context and is short enough so that no abbreviation is required.
- "ACME ITSM Core" / "x_acme_itsm_core"
"ITSM" is a "Process Group" and the type is "Core" – such applications contain data model extensions, ACLs and roles related to multiple processes.
- "ACME Core" / "x_acme_core"
This process independent scoped app contains reusable code and capabilities that may be used by all other applications. The type "Core" indicates that the app contains components to be used by other applications – including data model extensions and roles.
- "ACME Platform" / "x_acme_platform"
ACME's highlander platform application – that depends directly or indirectly on all other applications and hence acts as a release manifest.
the platform app also contains the persona roles and sets platform-wide configuration settings.

Global application technical prefixes

"<Prefix>"

- <Prefix> - The prefix should be a shorter variant of the application name (but only consists of letters and numbers, but must start with a letter) – With this constraint it can be made part of JavaScript object names and is compatible with Script Include names which either represent the class or function they contain.

Examples:

- "AcmelIncMan"
The prefix for the "ACME Incident Management Global" application, which is the sidekick of the scoped "ACME Incident Management" application.
E.g. a possible Script Include function could be named
"AcmelIncManIdentifyAssignmentGroup()"
- "AcmelTSMCore"
The prefix for the "ACME Core Global" application, which is the sidekick for the scoped "ACME Core" application.
E.g. a possible Script Include function could be named
"AcmelTSMCoreGetTasksByConfigItem()"

Cross Scope Scripting

When developing scoped applications developers face several challenges. Some Glide API features are not available in scoped applications, and some tables prohibit access from other scopes.

When trying to access assets across scopes chances are high that a developer sees one of the following error messages:

- "Security restricted: Access to api [some function] from scope [some scope] has been refused due to the api's cross-scope access policy"
- "Security restricted: [some operation] operation against [some table] from scope [some scope] has been refused due to the table's cross-scope access policy"

The good news is there are ways to overcome these restrictions.

Cross scope restrictions come in two flavors:

- Inability to read, create, modify, or delete specific records
- Inability to use some Glide API features because they are not made available to scoped apps

Whenever you create your own scoped app and try to access records which reside in any other scope than your own you may bump against the cross-scope restriction wall. Likewise, when you try to use several Glide API methods your script may produce errors indicating that the method is not available for scoped applications.

Prominent examples of Glide API features that are only accessible in global scope are

- `GlideRecord::setWorkflow()` – which is used to avoid business rules to be executed when updating or inserting a record
- `GlideImpersonate` – which is used to impersonate a different user – which is crucial in scripted ATF tests

To understand these restrictions, it is helpful to take a closer look at what scopes are in the first place.

Scopes represent a boundary around an application and its data. Anything within a scope has free access. But access to both data and code in a different scope is limited. So, are scopes mainly an access control mechanism?

Not really. At the same time, a scope is a packaging mechanism. That may not have been so clear when mankind still used update sets to ship their product increments from one instance to another. However, when following the guidance provided in this document, applications are the only means to transport code – and scoped applications are defined by their scope.

The scope is the natural boundary around a piece of versioned software that we ship.

Over time scopes have emerged into two things at the same time: an access control mechanism (for application code, not for users) and a packaging mechanism.

Both are valid intentions. We want to control access from one area to another. We also want to package software for deployment and re-use.

However, these two intentions lead to a conflict - as scopes try to do both at the same time. The package we want to ship and the security boundary we consider fit-for-purpose may not overlap.

This conflict becomes more obvious when we think of a scoped application - let's say an application that deals with highly confidential data. No other application shall be able to access data stored in any of the tables of that application. We appreciate the cross-scope restrictions that can be configured for these tables. All fine so far.

Now think of several customer specific adaptations and extensions to this application. We want to develop these additions in a separate package, give it a separate version and name. And we want to ship it separately. We can think of these additions as "plugins" or "extensions" to the original application.

It appears straight forward to implement these plugins as a new separate scoped application. But cross scope restrictions will make it impossible to access the table in the first application from the second. The packaging boundary and the security boundary are different - and in that scenario that is a challenge.

The best way to connect two applications would be for the first application to expose an API or extension point to allow other applications to extend the first application in a defined and structured way - however in many cases such APIs or extension points do not exist.

Since Quebec and Rome developers can create customizations to a store application which allows us to create a new application that shares the same scope of an application that has been installed from the store. Since Rome this also includes installed plugins. However, this option is not available for applications that were created by customers or the open-source community.

As mentioned earlier yet another type of scope restriction applies to the Glide API. Some features are just not accessible if executed from a script that belongs to a scope other than global. These restrictions are - like many other features of complex systems - the result of many conscious decisions made over time:

At some point scoped applications were seen as something coming from the outside - built by clients, partners, community developers - not by core ServiceNow developers - and hence were met with suspicion. Applications - according to this line of thinking - are inherently insecure and as such should not have access to all the potentially dangerous privileged features of the Glide API - like impersonating users (through `GlidImpersonate`) or doing changes to records without triggering their business rules and other automated workflows (using `GlideRecord::setWorkflow`).

However, with the introduction of the "global scoped application" one can build applications that have their own scope as a packaging boundary, but which execute in the global scope and hence do not experience any API related restrictions. So, considering applications as dangerous per se is not the real issue. To create global scoped applications, you must add a system property to an OOTB instance first. Although openly documented, the global scoped applications remain unknown to many developers.

Another argument is that the implementation of the APIs for global and for scoped applications differs and that every API implementation must be adapted - mainly due to the namespace implications - and that this work has not been finished yet. Some of the restrictions were removed over time - so more and more Glide API features become available in scope - as the API evolves over time.

Conversely there are API features which are not communicated to be available to scoped apps although they are. One example is `GlideUser::getMyGroups()` which is documented in global but undocumented in scope.

Some of these API limitations are seen as an enforcement of good practices when developing applications. Whenever a scoped app is using features from the global Glide API that should be

done with great care and be the result of a conscious decision. Developers should consider both security, architecture, deployment, performance, and governance aspects.

At the end of the day, scopes remain to be two things at the same time: a means to package a set of assets as a version of a software module and a security boundary.

To make this process work – the security boundary needs to be ignored – as developers need full cross scope access for their applications.

How to cross scopes?

Cross scope restrictions apply when code running in scope tries to access data from a different scope and when code makes global-only API calls.

Tables can be configured to restrict access from other scopes per operation. One can define whether create, read, update, or delete operations should be allowed from other scopes or not. That is the granularity one can define. Not more, not less. Which means that in the example of one application “extending” another application, the only choice is to open the table with the highly confidential data to all other scopes and not just to the application that serves as a trusted plugin of the first application.

In addition, modifying the configuration of these tables - which might be part of OOTB features, plugins, or 3rd party apps - represent a change to assets that may be updated at some point in the future - and that creates more complexity when upgrading. That should be avoided.

As mentioned, in an ideal world the first application provides a defined API that other applications can use to implement extensions - but in many cases that API is not available.

We should keep in mind that these restrictions apply on a script level. They control what one script can do with data in a different scope. What users can or can't do is fully controlled by ACLs. That is a totally different story.

So, configuring a table to allow cross scope access may not be what you really want if access should only be granted to specific other applications.

As stated before, access to tables from one scope to another is only a part of the issue. Not being able to execute all Glide API functions from a scoped application is the other. So, what we need is a way to execute code in either global scope or a scope of choice from within a scoped application.

If a script could be executed in a scope of choice that script would also be able to avoid the table-based scope restriction, since the create and modification functions of GlideRecord will work fine if executed in the same scope as the protected table.

How to execute any code in any scope?

An obvious option to execute code in a different scope than the application that needs it would be to create a Script Include in the corresponding scope. But that's not really satisfying since the aim is to ship applications as one package - and the scope of your application serves as exactly that packaging boundary. To ship applications AND additional Script Includes in different scopes in additional global scoped apps or (even worse) as additional update sets should be avoided.

If additional Script Includes that need to be shipped are not an option, what else can be done?

A Glide API class comes to the rescue: GlideScopedEvaluator. This class allows us to execute a script that is stored in a field of a record in any table. The script contained in this record is then executed in the same scope context as the record.

While the record used must be persistent in the database, the actual script to be executed does not. That means, any record containing a script field that is in the target scope can be used with `GlideScopedEvaluator`. After the record is loaded, the script field can be set to whatever script is required and then executed by `GlideScopedEvaluator`.

There is one precondition for the `GlideScopedEvaluator` class to work across scopes:

The global system property `"glide.record.legacy_cross_scope_access_policy_in_script"` must be set to `"true"`.

Before doing that, we need to keep in mind that this will enable any script from any application to execute code in the global and any other scope. This may or may not be considered as a security risk. So, check back with the responsible governance body before making that change.

The following example code is a single function `Script Include` that illustrates how a scoped application can use the `GlidImpersonate` class which is only available in the global scope:

```
function ImpersonateUser(strUserSysId)
{
    Var strScope = 'global';
    var grSI = new GlideRecord('sys_script_include');
    grSI.addQuery('sys_scope','=',strScope);
    grSI.setLimit(1);
    grSI.query();
    if (grSI.next() == false) return false;
    grSI.script =
    "var glidImp = new GlidImpersonate(); result = glidImp.impersonate(user+')+";";
    grSI.sys_scope = strScope;
    var evaluator = new GlideScopedEvaluator();
    evaluator.putVariable('result',null);
    evaluator.putVariable('user', strUserSysId);
    evaluator.evaluateScript(grSI, 'script');
    return evaluator.getVariable('result');
}
```

The code example demonstrates how it works – however a clear recommendation should be made to implement a single re-useable function that makes use of the `GlideScopedEvaluator` class – so that all scripted cross-scope activities are channeled through this single function. This allows us to control and audit such cross-scope activities.

Not using the `GlideScopedEvaluator` anywhere else may also be part of the Coding Guideline enforced using Instance Scan.

As a summary: When developing scoped apps developers may run into cross scope restrictions as the required security and packaging boundaries of a scope may not overlap. This is particularly likely in an application's installation script – as changes may have to be made in

different scopes than the application scope. Using the `GlideScopedEvaluator` class in combination with the system property `"glide.record.legacy_cross_scope_access_policy_in_script"` set to true is a solution to that.

The open-source application "DevTools" contains the technical capabilities to execute code from any scope in any scope. Refer to the functions `RunScriptInScope()` and `RunScriptInGlobalScope()`.

Installation Scripts

Installation Scripts aim at making modifications to a target instance that for whatever reason cannot be transported as Application Files through source control and hence cannot be deployed in regular ways.

This includes the following cases:

- System properties which need to be set to different values on different target systems (e.g., logging verbosity levels)
- Changes to OOTB global or scoped records (e.g., deactivating specific OOTB UI Actions or Business Rules, or modifications to ACLs)
(The benefit over shipping these OOTB records as part of a global or customization app is when such records are updated as part of a family or store release – in such cases all updated records can be accepted and the installation scripts can be re-executed after the upgrade – but also note that such changes should only be done if there is no alternative.)
- Creating records in other scopes than the application scope (e.g., ACLs on tables defined in other scopes)
- Modification to records that are in tables not derived from sys_metadata – and hence are considered data
- Creation or modification of users and groups and assignment of roles to such groups and users (e.g., technical users required to run Scheduled Jobs etc.)
- Configuration of Scheduled Jobs – as they need to be associated with a user to run as
- Configuration of (existing) database views
- Creation of database indices
- Modifications to other applications – however that must be limited to downstream dependencies

To perform such changes, an installation script needs to be built in a way that it

- is shipped as part of an application
- is only executed ONCE as part of the deployment of an application – after the application has been copied to a target instance and static code analysis checks have been performed, but before any automated functional tests are run
- is idempotent – which means that even if it is executed multiple times on the same instance, the result is always the same (so it does not create duplicate records etc.)
- can make changes to records that are either in no scope (i.e., represent data), in the scope of the application or even in any other scope
- performs all modifications through a specialized API that allows the execution of the script to either do all the changes or to create documentation on what is planned to happen during installation (that is however optional)
- complies with a specific naming convention and/or interface definition so that it can be executed as part of an automated deployment process
- returns success or failure so that an automated deployment process can respond to errors

This set of requirements implies some unexpected superpowers and constraints:

An installation script MUST NOT be shipped as a fix script – since fix scripts do not allow any control on whether or when the script is executed when regular OOTB application deployment mechanisms are used. Fix scripts do execute immediately after copying an application to a target instance – e.g., through source control “Apply Remote Changes” – and this is not acceptable

An installation script must be able to make changes to records in other scopes and eventually access Glide API functions that are only available in global scope – hence cross scope scripting techniques are required (refer to chapter “Cross Scope Scripting”).

As with any other feature Installation Scripts must be tested. The following ideas should be considered:

- Specific ATF tests that verify the state of records after running an installation script
- ATF tests that verify the application's behavior where the underlying logic depends on the successful execution of the Installation Script
- The execution of an Installation Script within an ATF test and the subsequent verification of changes made during its execution

These ideas can be combined or used in different scenarios – there is no one-size-fits-all solution.

The open-source application “DevTools” contains all technical capabilities to

- Run scripts in any scope from any scope
- Perform all changes through a specialized API
- Execute installation scripts for an application and all its direct and indirect dependencies

The open-source application “Deployer” allows to execute installation scripts on a remote instance for integrating with “Deployment Automation”.

Instance Specific Configuration

As the title suggests, some configuration needs to differ between different instances – that is different records, settings, values, options on a DEV, TEST, PREPROD, PROD or other instance.

Such configurations may represent:

- Credentials and connection configuration pointing to connected systems – where a DEV environment may not be connected at all, the TEST instance is connected to a mock environment of a remote system, and the PROD instance is connected to the PROD environment of that remote system
- Configuration on outgoing emails – e.g. to prevent emails to be sent to real users from within DEV and TEST environments
- Configuration on user sessions – like its duration, idle times, etc.
- Log levels
- Security configuration
- Titles, logos, and colors in the GUI to indicate to users which instance they are currently using
- System properties of all kinds

The use cases of instance specific configuration may be obvious, the answer to the question of how to deal with them in the deployment process, however, is not obvious at all.

Should such instance specific configuration be deployed as part of one or more applications?

Or should such configuration be applied and managed manually on each target instance explicitly.

As often in complex environments, the answer is: "it depends".

The following options exist:

- Manual changes made in each individual instance
- Changes made on DEV (or any other instance considered as a reference) and then manually exported and imported to downstream instances
- Deployed as part of applications – overriding any deviating value on the target instance
- Deployed as part of applications – maintaining any existing value on the target instance – thus allowing later manual changes

With these options at our disposal, for each individual use case a different choice can be made – but no matter which choice is made – it should be consistent.

Development and operations teams must decide and agree who "owns" specific types of configurations and who is responsible to maintain it and how.

Configuration that is deployed as applications should be "owned" by the development team as they direct changes made to the target instances through the applications they ship.

Configuration that is maintained manually on each target system should be "owned" by the operations team.

When instance specific configuration is deployed as part of an application the installation script is the most obvious place to check for the instance to which the configuration is applied and control the specific values and settings depending on the instance.

It is recommended to add such agreements into the coding guideline and implement corresponding Instance Scan checks to enforce it. This can at least prevent developers from accidentally setting configuration values that are in fact "owned" by operations.

However, using technical names of instances (i.e. the DNS or URL names) should be avoided as the applications otherwise become dependent on the specific technical infrastructure on which they run.

Take the following scenario of a company called "ACME" which has 4 ServiceNow instances (in the ServiceNow cloud as it seems):

Role	URL	Technical Name	Pipeline Name
Development	https://acmedev.service-now.com	acmedev	dev
Test	https://acmetest.service-now.com	acmetest1	test
Pre-Production Test	https://acmetest2.service-now.com	acmetest2	test
Production	https://now.acme.com	acmeprod	prod

Note that the URL and the technical name (as found in the pseudo system property "instance_name") sometimes differs – maybe for historical reasons?

Imagine, an instance (no matter whether DEV, TEST or PROD) is re-installed, physically, or logically moved or - for whatever reason - renamed – e.g. because the company is acquired and obtains a new name. In such a scenario, applications would have to be refactored to use the new technical instance names in the corresponding installation scripts.

Instead of using URLs or technical names to identify an instance during installation, applications should use so-called "pipeline names" instead of "technical names" in Installation Scripts.

A mapping is required between technical names ("instance_name") and the pipeline names – and this mapping must be made available to all applications to be used in Installation Scripts.

Installation Scripts should also be prepared to handle cases where no mapping is available – and respond with default values or errors.

To support this technique the following capability needs to be implemented:

- A mapping between technical names and pipeline names – which can be maintained independent from applications to be deployed
- Installation Scripts can query the pipeline name of the current instance

The open-source application "DevTools" contains all these technical capabilities ("GetInstancePipelineName").

Data vs. Code

What is data and what is code? And why is it important to differentiate?

To get this question answered, we need to get the terminology in order. This document describes a process by which applications are developed in a development environment and then deployed to other instances. For the sake of this question, we will consider everything that is shipped as part of one of these applications as **Code**.

Data on the other hand is whatever is created or modified by users, administrators, or integrations in production or what is created or modified in sub-production instances by testers, developers, test integrations for the sake of testing.

One might argue that this differentiation is arbitrary and irrelevant in the ServiceNow context. Code (in this wider sense) which may include anything from “real” code artifacts like a Script Include to all sorts of “Configuration” (like the selection of fields visible on a form) is stored in database tables. From a pure technical point of view there is little difference between an incident, a user, a group, a company, or a configuration item (“Data”) and Script Includes, Business Rules, a Decision table, or a UI Page – all of them are stored as records in one of the many tables.

There are however important differences – because we treat Code and Data records differently in this process – and we need to define different people (or roles) to be responsible and accountable for them.

Here is a collection of examples – by far not exhaustive – to ensure the reader can spot what is Code and what is Data going forward:

Code	Data
The Application record itself (sys_app)	Users
Script Includes	Groups
Business Rules	Group Types (surprisingly)
UI Builder Pages	Users assigned to Groups
Client Scripts	Roles assigned to Groups
Catalog Items	Companies
Portal Pages and Widgets	Tasks / Cases / Incidents / Problems / Changes
Tables	Attachments
Fields	Emails
Roles	Configuration Items
Credential Aliases	Events
Connection Aliases	Credentials
Scripted Web Services	Connections

As a rule of thumb, the following distinction works well:

Code is anything stored in a table derived from sys_metadata or sys_scope.

Data is anything stored in a table NOT extended from sys_metadata or sys_scope.

Code is transported as part of an application through the process described – it is always changed in a development environment
(Note that Code that is modified by Installation Scripts on downstream instances remains Code).

Data is maintained directly on the different instances through some kind of operational process in production or as needed for testing purposes on sub-production instances.

That concludes that **Code MUST NOT depend on Data**.

(E.g., this means that a Flow must not reference a specific group for a task assignment directly. The group should rather be configured in an instance specific system property and be read by the Flow.)

So far so good. There are, however, several grey zones where the distinction is not that obvious:

Scheduled Jobs

A scheduled job is stored in the sysauto_script table which is extended from sys_metadata.

OOTB the table is excluded from being captured in source control through the attribute update_synch=false. Let's explore why:

The Scheduled Job record contains information about when and what to run. The "what" is a script – clearly, that is Code.

The "when" is debatable. Maybe a Scheduled Job should run in different intervals on a sub-production instance than on the production instance. Maybe on the development environment it should not run automatically at all – but only when the developer wants it to run – for testing purposes.

The exact orchestration of many Scheduled Jobs running during the day, or the week should be a decision to be made by administrators responsible for the production instance and should not lie in the hands of developers who might not know enough about the full set of jobs that need to be run. So, developers may not be able to make informed decisions for when their job should be run.

Even worse, Scheduled Jobs also contain a reference to a user that should be used as the execution context for the Job. As users are clearly Data, how can this be? Code must not depend on Data, so how can a Scheduled Job (Code) contain a reference to a User (Data) and hence be dependent on it?

Despite some research no conclusive answer could be found by the author. It just is what it is, and it cannot be fixed easily due to the need for backward compatibility.

But the question remains: how to deal with Scheduled Jobs? They are a mix of Code and Data.

There are various options:

Consider Scheduled Jobs as Data and NOT include them into applications to be deployed. Administrators must then add Scheduled Jobs manually on each instance – and eventually copy the to-be-executed code from documentation provided by developers

Consider Scheduled Jobs as Code and

- change the attribute on sysauto_script to update_synch=true and
- add Installation Scripts that
 - add technical users for the execution on target instances automatically
 - configure the Scheduled Jobs to use these dynamically created technical users

Consider Scheduled Jobs as Code and ship them without associated technical users and delegate the responsibility to configure technical users and timing to administrators. Such configuration could eventually be shipped as a separate application which is focused on instance specific configuration, but which is maintained by administrators.

In this case no clear guidance can be provided as the best suited solution depends on the organization, the existing separation of duties and applicable security policies.

The option where the applications create their own technical users is appealing as it is pragmatic and requires little coordination between teams – but some organizations may just not accept users being created (and security privileges being assigned to them) by applications as part of their installation scripts – as such an activity may strictly fall into an operations team's responsibility.

Other scenarios can be identified in which the distinction between Data and Code is not that clear, where OOTB structures do not support this clear differentiation or where Code may rely heavily on specific data structures.

An example for such a scenario is product definitions – which technically are Data, but which may only change as part of a corresponding Code change and hence may have to be treated like Code. Refer to chapter “Data as Code” for guidance on how to do that.

It is important, however, to make conscious decisions on how to treat these cases explicitly and define responsibilities and technical processes to create, update and delete such records.

Data as Code

As pointed out in chapter “Data vs. Code” there are scenarios in which certain data records should be treated as code.

A prominent example are product definitions as used by the ServiceNow product Order Management for Telecommunications and other industry workflow products.

Product definitions contain – as the name suggests – information on complex offerings with many variables and variations. Imagine a large telecommunication company offers network services to large clients. A product definition in that context may consist of hundreds if not thousands of individual records.

A design decision has been made by ServiceNow to consider these product definitions as data – so that users can modify these product definitions in production to adapt them in day-to-day operations. This implies that all code must be designed to work on any potential constellation of data.

An equally valid approach is to consider such complex product definitions as code that is shipped as part of applications. Imagine there are strong and complex dependencies between product definitions and integrations with 3rd party systems, user interfaces and the logic to process orders based on these product definitions.

This is just one of many examples where the boundary between data and code is blurry and there are good arguments for either considering records as data or to consider them as code.

The question is: what if there are records in tables that are NOT derived from sys_metadata - and hence are in fact data records – but these records should be shipped as part of an application – and hence should be treated as code?

How to ship data as code?

Any application can define new tables that are derived (“extended”) from sys_metadata. This means that all applications can contain any kind of data which is stored in records in such extended tables. In this document such tables are referred to as “transport” tables.

To include data records in an application, a mechanism must be created to copy such data records INTO a transport table. This mechanism can be used by developers at any time when appropriate – the latest point in time would be during baselining the application. During deployment the records in the transport tables are installed onto the target system (as part of the application) and would then be copied into the actual data tables. This can be done in Installation Scripts.

There are mechanisms available on platform OOTB that allow them to include individual records as application files. However, these OOTB capabilities are not fit-for-purpose to ship large complex networks of records.

To support this technique the following capability needs to be implemented:

- A scoped application defines a transport table that stores data records' metadata and content
- A mechanism that copies data records stored in the transport table into the original data tables on a downstream instance (to be used in installation scripts)

Both mechanisms should support idempotency to avoid duplicate records

The open-source application “DevTools” contains all these technical capabilities (“Record Transporter”).

Guidance on Script Includes

As a rule of thumb: move as much code into Script Includes as possible.

Script Includes should contain the main part of the server-sided logic, data retrieval and storage, and decision making.

The code contained in Script Includes should be agnostic to the context in which the code is executed. The same Script Include function should be callable from a Business Rule, a (server-sided) UI Action, a Scheduled Job, or a Scripted REST API.

The default code snippet added by the platform when creating a Script Include created a myth: Most ServiceNow developers believe that the only code structure of a Script Include is the class.

It is not.

In most cases, the single function Script Include is the better choice.

Single functions are easier to test.

And the following code pattern does not really support readability:

```
var strResult = new x_some_scope.SomeClass().SomeFunction(strSomeParameter);
```

A class should ONLY be used if one or more of the following conditions are met:

1. The class represents a real object – and the object must maintain an inner state (a mere collection of static and independent functions does not qualify as a real object)
2. The class implements a defined interface (like a client callable script or an extension point)
3. The class extends an existing class

In any other case, use single function Script Includes.

The code contained in a Script Include should cover one concern only – this may in fact lead to many smaller single function Script Includes.

Each of the Script Includes should have a dedicated scripted ATF test to validate its capabilities.

Code that is stored in other server-side scripts (Business Rules, UI Actions, REST APIs, etc.) should only:

1. Prepare parameters to be passed into a Script Include function (or class method)
2. Process return values provided by the code in the Script Include
3. Prepare user output (where applicable)

For example, the code contained in a REST API Resource should only

1. Retrieve URL parameters
2. Perform error handling and construct the appropriate HTTP response object (e.g. to return applicable HTTP response codes)

All actual business logic, data retrieval and data storage should be performed within a Script Include.

Managing Translations

Some aspects of the ServiceNow platform are a bit confusing – one of these aspects is how translated text is stored and maintained.

There are multiple tables that explicitly contain translated text

- `sys_ui_message`
- `sys_translated`
- `sys_translated_text`

There are numerous tables that contain text visible to users which do not have any specific attributes to specify the language of the text. Examples are:

- `sysevent_email_action`
- `sysevent_email_template`

In total there is no consistent mechanism to store – and hence to deploy translated text across the platform.

To gather insights on the coverage of translations for a given language compared to a base language (e.g., English) is a form of art. Guidance on how to manage, maintain, test, and validate translations however is NOT (yet) subject of this document.

The guidance in this document is focused on the deployment aspect.

Most tables that contain translated text are derived directly or indirectly from `sys_metadata` – so these tables contain Application Files – which are regularly captured into a Source Control repository and are deployed in the same way as any other Application File. For those records, no special treatment is needed.

However, there is an outlier: The table `sys_translated_text`.

Any table can contain string (or HTML) fields that are “translated”. The content of such fields is English by default but if a user selects a different language for their session and modifies the contents of such a field, a record in `sys_translated_text` is created for that specific language.

A record in `sys_translated_text` refers to the record, the table, the field, and the language for which the field was edited.

And here comes the challenge: As stated, any table can contain a translated field – that includes Application Files (i.e., tables derived from `sys_metadata`) – so a user (a developer) may add translated text to a record that is an Application File, and which is part of an application to be deployed.

The mechanism of translated fields is universal, so `sys_translated_text` records can both refer to records that are **code** (Application Files) and to records that are **data**.

The table `sys_translated_text`, however, is NOT derived from `sys_metadata` – i.e., these records are NOT Application Files – and hence not captured as part of an application and not committed to the Source Control System.

If the intention is to ship translated text for fields contained in an Application File (e.g., the name and description of a Catalog Item stored in table `sc_cat_item`) – the corresponding records in `sys_translated_text` – which are technically **data** records - must be treated as **code**.

To support this, the same principles can be applied as described in chapter “Data as Code”.

In addition, a capability must be implemented to identify all `sys_translated_text` records that are associated to Application Files contained in a specific application.

The open-source application “DevTools” contains all these technical capabilities (Refer to function “`RecordTransporterPackagePackTranslatedText`”).

Test-Driven Development

"By writing clean code that works and demonstrating your intentions with automated tests, you give your teammates a reason to trust you." Kent Beck, *Extreme Programming Explained*.

This chapter provides an overview on good practices when writing ATF tests and maintaining an ATF test suite that provides value in the long run. Moreover, the author of this document clearly advocates the practice of test-driven development – or short "TDD" – which implies that tests should be authored as soon as possible – which may even be before the first line of production code is written.

As we all love lists, meet the 10 rules to follow to build and maintain a test suite that provides value over time:

1. Ship tests as part of apps

Each application should have a default test suite which is named exactly like the application. This test suite is then used to validate the application during development, before baselining and during deployment to non-production instances. All tests that validate an application's behavior should be part of the suite and be part of the corresponding application.

This way code and tests always stay in sync.

The application's tests are always part of that application and will also be deployed to production – where they will probably never run – unless specifically designed.

2. Create all required test data within the test

Tests must not have any dependencies to data, the environment, specific configuration. All tests must be able to always pass. That implies that any data that is required to simulate a behavior must be generated as part of the test itself.

This may seem cumbersome in the beginning, especially when complex data structures are required to simulate a use case – e.g., think of an incident related to a CI which is impacted by other CIs which is follow-ed up by a problem that later results in a change. Many teams struggle with the creation and maintenance of test data on development and test instances. By creating code that creates test data, not only automated tests can make use of it – but also manual testing.

The platform's Automated Testing Framework undoes any data changes performed during an ATF test, which includes the creation of test data as part of a test. So, creating test data as part of the test also ensures that test data does not clutter the instance over time.

3. If a behavior can be tested without the GUI, don't use the GUI

It is tempting to use existing manual test plans and procedures as a template to create automated versions of it – and thus just trying to replace human testers performing scripted manual tests by a robot. However, this approach does not produce any value. Resulting tests are complex, depend heavily on the GUI and are costly to maintain.

Complex manual tests are used to validate many different aspects of an application's behavior into one single test – for practical and efficiency reasons. When creating automated tests, the tests should focus on the narrowest scope possible – and often a specific part of the functionality of an application can (and should) be tested without using the GUI.

This often requires to re-design the whole test suite, as an existing manual test suite just cannot (or should not) be used as a reference for creating automated tests.

This sometimes also requires refactoring code.

E.g., imagine an UI Action that performs a complex operation on one or more records. To

test this behavior without the GUI, the code may have to be moved into a Script Include first, and the UI Action script only calls the function in the Script Include, and eventually provides feedback to the user.

Such a function could then be tested without the GUI by creating the necessary test data, then calling the function directly from a test script and then validating whether the data was manipulated in the expected way.

4. **Verify one behavior in each test**

Each test should focus on one behavior or functionality only. A failing test provides useful information to developers. If only one specific behavior is validated in a test, it is easier to track the root cause if tests have a narrow scope.

5. **Prefer many smaller tests over few larger tests**

To narrow the scope of tests inevitably leads to many small tests. And that is a good thing.

Big is bad, small is good. There is no balance to be kept on this one.

Is it more difficult to maintain many smaller tests? No, it is not. Any change, intended or unintended, may cause tests to fail. If a change is intended, and tests must be updated to reflect the new, modified or introduced behavior, it is again easier to modify many smaller tests, then modifying larger more complex tests.

Smaller tests also help better in understanding the real impact of a change.

6. **Use mock interfaces instead of real external systems**

Tests should have no other dependency but the code and logic they are validating. This also includes external systems. If data is imported or ingested from an external system, the investment in mock interfaces or mock data objects almost always pays off. It may be difficult and cumbersome in the beginning, but as soon as the foundation is laid, any additional test scenario can be added quickly – thus increasing test coverage at a high pace.

External systems may not be connected to all instances, and not all external systems may support interaction with automated tests.

7. **Avoid using asynchronous processes**

Whenever asynchronous processes are involved tests may become flaky – that is for no obvious reasons they may fail sometimes. This is frustrating and annoying. One of the most frequent reasons for flakiness are tests that trigger asynchronous processes – and that must wait for the process to finish to evaluate the result. Sometimes this hits a timeout, and the test fails.

Refactoring code in a way that allows to execute the functionality directly as part of the test – without the asynchronous processes around it often helps to create both more robust code and more reliable tests.

8. **Make conscious choices on (GUI-based) end-2-end tests**

While stating that the GUI should be avoided, does not mean that it should not be used at all. Each application may have a very small number of automated GUI based end-2-end tests. However, such tests should be the exception, and the test cases should be chosen with great care. The intention of such tests should also explicitly validate the GUI itself – and not the underlying functionalities.

9. **Prefer scripted tests over separate test steps**

The Automated Testing Framework supports various GUI-based and server-only test steps that can be used to assemble complex test procedures. Each test step can then end the test with a failure if a defined condition is not met.

In theory this works well – especially for GUI-based tests – but for unit tests – that is the validation of individual JavaScript functions or classes this approach is not helpful. Whenever it is feasible, use scripted tests that contain only a single “Run Server Side Script” test step.

Within this script, create the necessary test data, execute the code to be tested and validate the results.

When different test cases – that is different combinations of test data and function parameters – should be tested, it is strongly advised to add these test cases into a single script.

The tests will execute faster, as test data constellations may be re-used and do not have to be created (and removed) for every single test case.

Another advantage of multiple test cases in a single script is that if one or more of these test cases fail, the test script can continue to validate all other remaining test cases within the script. If the test cases were implemented in separate scripts, the first failing test step would stop the test – which would prevent the remaining test cases from running - and less information is available for developers to investigate the issue.

10. **Re-use test code using Script Includes**

The Automated Testing Framework supports so-called Test Step Configurations – these are re-usable scripts that can be integrated into an ATF test. Experience with larger teams working on complex applications shows that maintaining re-usable test code in Script Includes is easier to handle and such re-usable test code can be executed more selectively from within other scripted tests.

Wiring the inputs and outputs of an ATF Test step into other scripts requires additional effort and does not provide any real value – and it requires to split the test code into several script steps.

The most common re-usable test code is a function that generates test data. If implemented in a Script Include the same function can also be called outside the ATF test – e.g., to prepare test data to be used in a manual test.

The open-source application “DevTools” contains capabilities to

- Write clean unit tests using the TestAPI class
- Create test data using the TestDataAPI class
- Create new tests in a test suite
- Run tests from within test step and test result forms

The open-source application “CodeSanity” contains Instance Scan checks to

- Validate each application has a test suite
- Ensure there is a test for each Script Include contained in an application

Semantic Versioning

Versions are not just numbers with dots in between. Versions convey an important message. They indicate progress.

There are different ways how version numbers can be used. To support the process described in this document best, version numbers should be “semantic”.

Semantic Versioning means that version numbers indicate whether the difference between two versions of a software module (an application in ServiceNow's terms) are

- Only bugfixes
AND/OR
- New features
AND/OR
- Contain backwards incompatible changes

A semantic version tells developers and users what to expect from a new version. They manage expectations. They indicate required activities – especially if an application depends on a technical interface exposed by another application. If a new version indicates a backward incompatible change, it may mean that developers of the application that depends on the modified application must adapt to these changes so that their application continues to function properly.

A semantic version is a prediction of compatibility expressed by the team that sets the version.

A prediction however is not a guarantee. The responsible developer team might make a mistake – but once the version is set and the application is baselined – the only way to fix it is to release a new version that expresses the actual compatibility correctly.

Semantic versions are crucial to set expectations and to communicate the need for further changes in other applications in a larger developer community.

This is how a semantic version is formed:

Given a version number MAJOR.MINOR.PATCH (e.g., “1.0.0”)

Increment the...

- MAJOR version when incompatible API changes are made (e.g., “2.0.0”)
- MINOR version when functionality is added in a backward compatible way (e.g., “1.1.0”)
- PATCH version when backwards compatible bug fixes are made (e.g., “1.0.1”)

The use of semantic versioning has been widely adopted in the open-source community and many commercial software projects and products.

But that is not always the case.

Many companies use versions to indicate a release plan or even use version changes for marketing purposes. The change of the major version in such cases rather indicates a significant number of new features, a new design or just the fact that the marketing department wants to draw attention to a product.

It might just feel like an understatement that hundreds of new features have been added in version 1.1.0 following version 1.0.0.

At the same time, it might appear overdone to increase the major version if there is only a single, small change done to an API. Imagine an application exposes a REST API with hundreds of

endpoints and in one single endpoint, a single parameter name is changed – a tiny little change - almost unnoticeable. But even that single small change breaks the contract between the API provider and its consumers. And the developers of the applications that depend on that API must check their code and eventually adapt their code to use the new parameter name.

Semantic versions communicate compatibility. They do NOT communicate the number of changes.

A detailed description of semantic versioning can be found here:

<https://semver.org>

Determining Minimum Version Requirements

It is not only important to know which other applications an application is depending on, but also which minimum version requirement exists.

One step of the Application Version Baseline Procedure is to insert and update an application's dependencies – and this includes to define the minimum version of the application that the to be baselined application is depending on.

An example:

An application named "Alpha" depends on an application "Beta" – more specifically, a Script Include in "Alpha" calls a function from another Script Include contained in "Beta".

In the development environment, the version of application "Alpha" is set to "1.5.0" and the version of application "Beta" is currently set to "1.3.0".

These versions are expected versions, as the current state of both applications in the development environment is not yet baselined.

The team responsible for application "Alpha" decides it is time to baseline the current state of application "Alpha" as it contains completed work that should be shipped and deployed to downstream environments for testing.

When application "Alpha" is baselined, the responsible team should now update the dependency to application "Beta" to instruct the deployment procedure which version of application "Beta" is required on the target environments.

The team baselining application "Alpha" now must make a choice: what is the right dependency version for "Beta"?

There is an easy way and there is the right way.

The easy way is to set the version dependency to the current version of "Beta" in the development environment. Since the current version of "Alpha" has been tested in the development environment with the current work in progress state of "Beta" around, so the assumption that "Alpha" will work well with the next baselined version of "Beta" (presumably 1.3.0) is safe.

The problem is that if everyone would act that way, this would delay if not even deadlock the deployment of application "Alpha". To install "Alpha" in version "1.5.0" one needs to wait until the new version of "Beta" in version "1.3.0" is also baselined.

The right way is to do some research and figure out if an earlier version of "Beta" (e.g. the latest baselined version "1.2.0") is sufficient to support application "Alpha". If it was, application "Alpha" in version "1.5.0" can be deployed right away, since "Beta" is already baselined in version "1.2.0".

But how can a team figure that out?

In this example, application "Alpha" calls a Script Include in application "Beta", so the question is here, did the Script Include in Beta change since the recent version "1.2.0" was baselined? And if it was, does it matter for what application "Alpha" needs it for?

Ideally, all teams maintain Release Notes with reference to all changes made so far. Reading the Release Notes of application "Beta" should already do the trick.

If no Release Notes are available, or the documentation is not clear or conclusive, the developers might just have to talk to each other.

If that is not possible – due to absence, lack of time, or whatever – the change history of the Script Include might provide a clue – however this also requires to match actual changes made to the Script Include to the points in time when application “Beta” versions have been baselined.

By doing the research the team that wants to baseline application “Alpha” might even get to know that application “Beta” in its current form is work in progress and application “Alpha” may rely on a just introduced capability that is not yet stable and may be subject to change before the next version of “Beta” is going to be baselined. In that case, the baseline of application “Alpha” may just have to wait if the dependency cannot be avoided.

Figuring out the best minimum version requirement is not trivial and requires organized communication between teams.

The absolute bare minimum should be Release Notes that contain all the changes made to an application. This is tedious and requires discipline. But not doing it causes more work for other developers and teams.

Backporting

The chapter “Hotfixing and Backporting” described the scenarios and use cases in which backporting comes into play – and how this activity can be handled from a process perspective.

The following chapter provides insights on how that can be done on a technical level.

Given there are two development states of the same application for whatever reason:

1. One state where a change (often a bugfix) has already been applied (the “source state”)
2. A second state where the same change needs to be applied as well (the “target state”)

The “same change” in this context does not mean that the exact same modification of code or records needs to be made.

The “same change” means that the same specific outcome should be achieved – but sometimes the same outcome can or has to be achieved using different modifications.

Backporting may be necessary in case of

- Hotfixing
- Working in multiple development streams – where multiple (in most cases major) versions of a single application are in development simultaneously

For the sake of the description of technical options in this chapter, we use the hotfixing scenario as a “case study” – but the techniques described can be applied to all possible backporting scenarios.

Backporting is always a manual process. Forget any approaches that mechanically identify changes or differences on a record level and then copy them from one instance to another. That may work in some corner-cases, but most of the time, it will not – as other changes made in the meantime may make the hotfix no longer valid in the current development code state.

There are several techniques that may play a role in backporting:

- Copy selected sections of code from a record in the source state to the same record in the target state
- Copy selected sections of code from the source state, insert AND modify that code in the target state
- Implement new code in the target state
- Remove existing code in the target state and re-implement the feature in the target state
- Deprecate the affected feature in the target state and declare the next version baselined from the target state as a backwards incompatible change

“Code” may refer to actual JavaScript code and any form of low-code, no-code or other configuration assets contained in an application.

Which of these techniques should be applied depends on:

- The changes that have already been made between the source state and the target state
- Whether the already implemented change to the source state is sufficient to achieve the same outcome on the target state
- How robust the change to the source state is

There are different ways to transport the code from the instance with the source state to the instance with the target state:

- Selected sections of JavaScript Code or other pure text-based assets:
Copy-paste through a workstation's clipboard
- For limited configuration changes on a limited set of records:
Re-do the changes manually in the GUI on the instance of the target state
- Where one or more records from the source state can be used as-is in the target state:
Update Sets or XML exports
(Yes, that is the only scenario in which Update Sets may play a role.)

Once the code on the target state is updated, and is now part of one or more applications, it is deployed as usual going forward – and will at some point in time replace the original hotfix or secondary development stream.

Managing Unfinished Work

In the chapter “Technical Debt Management” it is established that “unfinished work” is one out of several cases that we refer to as a technical debt.

This chapter describes in more detail how the development team can handle unfinished work in the code.

In an ideal world the technical debt record is being created immediately when the work on a new feature starts – if it is to be expected that the feature will not be completely implemented at the time of the next version baseline.

In the real world, most teams won't be able to predict with certainty that this situation may occur. There may be a plan for the next version baseline – but changes in priorities, upstream dependencies, or an urgent bugfix might require a version baseline earlier than planned. As a result, it is difficult to predict if the work on a new feature needs to be documented as a technical debt and if a switch needs to be built into the new feature.

Does it make sense to create a technical debt in all cases and even build in switches for every new feature? Certainly not – each team will need to develop a sense on whether it is necessary and if so when.

The latest possible point in time is when the application version baseline should take place. If the plan is to have a sprint review on Friday morning and to perform an application version baseline in the afternoon – a team might spend the lunchbreak with building switches around a new feature that they expected to deliver, but the product owner just did not consider to be ready. Obviously, that is not what we want.

There are certain aspects that a development team should consider when designing new features in the first place:

- Build new features as if there was the requirement to be able to switch them on or off
- Split the code and configuration to be created into smaller pieces

In addition to the code design, a team can also reduce the likelihood of having to manage unfinished work by carefully planning when which feature is to be built and by whom.

Please refer to chapter “Reducing Unfinished Work” for more details on how to avoid having to manage unfinished work in the first place.

One of the technical requirements for day-to-day technical debt management described in chapter “Technical Debt Management” is to provide a function that can be used to query the state of a technical debt switch.

This function can then be used to control if a feature should be active and visible to users.

The open-source application “DevTools” contains the function `GetTechnicalDebtSwitchState()` that can be used to control feature activation and visibility.

Here are some specific examples of how to implement such switches:

- Script Includes – use the function directly in code
- UI Actions – use the function in the condition script field
- Client Scripts and UI Scripts – Either set the active flag to false/true or add logic in the client-side script – however this requires a display business rule to transport the switch state from the server to the client.
- UI Policy – Set the active flag to true or false
- Table - Configure advanced read ACLs and use the function in the scripted response – note that ALL read ACLs for the table must implement the logic
- Fields – Set the active flag to true or false
- Flow – Either publish/unpublish the Flow or create an if statement querying the technical debt record directly

For all types of application file records where it is not feasible to add scripted logic and use the function, but which have an active flag – or any other field that controls whether the application file is active – one can also set these fields as part of the Installation Script of the application (see chapter “Installation Scripts”).

Reducing Unfinished Work

Unfinished work is Application Files in an application that are still work in progress.

Unfinished work should be avoided for three reasons:

1. When baselining an application, unfinished work needs to be managed – i.e. switches need to be introduced and later removed, documentation must be maintained – see chapter “Managing Unfinished Work”.
2. When there is urgent demand for a defect correction, any unfinished work makes the fast baseline of an application more difficult – thus increasing the time (and efforts) for defect resolution. This is not only because any unfinished work must be handled on short notice, but the state of the application may have changed since the version for which the defect correction is needed has been baselined.
3. Any work that has been done should yield value to users as fast as possible. Anything that has been done but is not used, increases capital costs – and that is bad for the business.

Unfinished work can however be reduced through the following tactics:

- Split code into smaller chunks – and prefer single function Script Includes over larger classes (where the various functions do not share any states and the class only acts as a namespace). It is easier to finish and extend smaller functions. See also chapter “Guidance on Script Includes”.
- Move complex code logic to Script Includes instead of keeping code in Business Rules, UI Actions, Flows, etc. This way, unfinished code is less likely to impact application behavior. This does not necessarily reduce unfinished work per se, but it reduces the likelihood that it must be managed as such.
- Plan the work on multiple backlog items impacting a single application within the development team so that there is as little overlap as possible and aim at finishing one backlog item off before starting the next.
On the other hand this may slow down a team if team members would have to wait for each other.
- Baseline more often. Even if not all work has completed (or even started) within a sprint creating an application version baseline – even if there is no plan to deploy it – reduces the likelihood that unfinished work interferes with already completed capabilities. This may, of course, consume some time which could otherwise be used for development work.
- Refactor the application architecture to split capabilities into more but smaller applications to reduce the likelihood of unfinished work at the time of the baseline of a single application. That in return increases the number of applications to manage and the dependencies between them. So, a balance needs to be found.

Treat Evaluator Warnings as Errors

The Evaluator is the component in the ServiceNow platform that reads, interprets, and executes JavaScript code. Whenever the Evaluator fails to continue executing code a warning is logged in the syslog table, with the source field set to "Evaluator".

The execution model of the ServiceNow platform is robust. Errors encountered by the Evaluator do not crash the platform or in any way hinder the platform from continuing to function. But a script that runs into an error will just stop on the spot. This may lead to an undefined state if data was modified in the part of the script that already ran, but this data modification was not completed as the script stopped.

The fact that a failing script does not produce any visible, obvious error message and is "just" logged as a "warning" in the syslog table conceals the fact that every single Evaluator error must be treated as a critical bug that must be fixed. Not more, not less.

To ship applications that are fit-for-purpose and work as intended it should be self-evident that such errors should be tracked and fixed before an application is baselined and presented for further deployment.

As stated above these errors remain invisible unless a developer explicitly searches for them in the log. This leads to many of such errors remaining unnoticed – even though hundreds, on some instances thousands of such errors are being logged every day.

The fact that even OOTB components produce such errors on a regular basis does not help either. Some developers even interpret the fact in a way that even ServiceNow does not care about these errors too much (in fact they are logged as "warnings") – and conclude that such errors are not relevant at all.

The opposite is true. Any script that fails due to syntax errors, the use of undefined variables, or other reasons that lead to a sudden and unhandled stop of a script can cause significant damage. That damage may pile up silently in small drops over days, weeks, and months. The outcome can be functional issues, but it may also be compliance or security problems.

A mature development and deployment process should consider to

- Treat the Evaluator "warnings" as errors – because that is what they are
- Surface such errors to developers to make them aware of failing code in the first place
- Establish a zero-tolerance policy against such errors – e.g., if such errors are being logged during a test, the affected applications must be corrected, and the test must be considered as failed

The open-source application "DevTools" contains all technical capabilities to

- Display recently logged Evaluator errors to developers (while hiding many Evaluator errors that were caused by OOTB components)
- Convert Evaluator warnings to errors

Defect Correction and Root Cause Analysis

To fix a defect, in most cases you do NOT need a full Root Cause Analysis (RCA).

Rather, to fix a defect you need to understand the cause of the symptom and you need to figure out a way to mitigate the negative effects.

Fix forward is the best approach by far in most cases. Many organizations are a bit obsessed by always having the option to roll back – preparing for a rollback is good, but fixing forward is better. Planning is a good exercise to be prepared when shit hits the fan – even if the plan becomes obsolete in the very moment.

To fix forward by fixing the defect one does not necessarily need to understand why all that happened in the first place. One needs to understand what it takes to fix it.

The RCA is done to prevent the same thing happening again. In ITIL terms – we do the full RCA as part of problem management, not as part of incident management. For sure stakeholders may expect that – but that expectations needs to be managed.

Many defects show up AFTER a deployment – which may indicate that many defects appear as regressions: Something worked, then it stopped working. Usually in such a subtle way that it was not seen during a test or even by users within the first days after deployment.

Knowing that it was a particular deployment that introduced the issue is helpful but again I believe it is not required to fix the defect.

When you can track it down to a particular point in time - when the symptoms were first observed - and this correlates with a deployment, then great, you can use a comparison between the different states to spot the code that may be causing the defect.

There are different ways to do that:

- Compare the version branches in the Source Control System using file-based comparisons
- Deploy the version branch representing the state before the last deployment on a SANDBOX instance and compare the PROD instance and the SANDBOX side-by-side
- Track down the point in time when the defective version was baselined (i.e., forked from the dev branch) and then look at the change history in the DEV instance and look for changes that were made before the time of the baseline directly in the change history of the DEV instance

However, these are secondary considerations.

The individual change may or may not be related to the defect and the current observable behavior; other changes done to other applications may also play a role.

So, focus on understanding what exactly drives the behavior on the defective instance right now and then focus on a way to fix the defect by modifying the current state of the code.

Please refer to chapter "Deployment Variants - Tankers, Speedboats and Lifeboats" for a detailed description of deployment variants and why rollbacks are so dangerous and risky – and hence why Root Cause Analysis may be a distraction when fixing defects in production.

Personas and Roles

Managing access to specific users or groups of users can be confusing. It is helpful to agree on terminology in the first place:

A **role** (in the ServiceNow context) is a type of configuration that links users or groups to specific access rights (like the write access on record of a specific table, a UI Action, or access to a specific REST API endpoint). A role is a technical asset – a record stored in the `sys_user_role` table.

A **persona** is a description of a group of human beings (or an individual). This description is provided from a business perspective. A persona is agnostic to any technical implementation. Some personas may not even interact directly with a system (specifically with a ServiceNow instance) and yet be very relevant to a business process.

The tricky part is to transform requirements which usually contain references to personas into technical configuration and logic.

As a “Help Desk Agent” (the persona) I want to be able to see all open incidents.

The OOTB role “`sn_incident_read`” provides read access to incidents, so assigning the role to whatever group constitutes the “Help Desk Agents” sounds like a good solution.

It is generally accepted good practice NOT to assign roles to users directly but to groups instead.

Let's assume there is one (or more) groups that contain “Help Desk Agents” so assigning the `sn_incident_read` role to these groups appears reasonable.

This approach raises several questions:

1. Is the fulfillment of the above requirement targeted at the development team or at the operations team? Assigning roles to groups is a data change, not a code change.
2. How can the fulfillment of this requirement be tested?
3. What if “Help Desk Agents” are not only supposed to read incidents, but also to modify them – and do many more things: shall all the different roles that are required then be added to these groups?

If the answer to the first question is: the requirement is targeted to the development team and hence the implementation should materialize as some form of code (in a wider sense) then the approach to assign the specific role to the group cannot be the solution.

After all, code must not depend on data. But what is the alternative?

Introducing the concept of the **persona role**. A persona role is a role that represents a persona. Such persona roles are named after the persona and contain whatever specific other roles required to allow (and deny) access to the users that belong to that persona.

As every other role it should not be assigned to individual users but to groups.

In the example the “Help Desk Agents” would be represented by a “`persona_helpdesk_agent`” persona role (which is part of an application), and the `persona_helpdesk_agent` role would then contain `sn_incident_read` role (and probably also `sn_incident_write` and many other roles).

Persona roles should be prefixed by the application scope name followed by “`.persona_`” – e.g., “`x_yourprefix_platform.persona_helpdesk_agent`”.

The implementation of personas (as an expression of business requirements) is now represented as a code asset which is fully testable with no dependency to production data.

If there is no documentation or concept of personas available yet, here is a list of usual suspects to start with:

- Business User
- Business User Team Manager
- ServiceNow Administrators
- Service Desk Agents
- Service Desk Supervisors
- IT Service managers
- CIO
- CIO Staff
- IT Expert (Fulfiller)
- IT Expert Team Manager
- HR Business Partner
- HR Business Partner Team Manager

Note that personas should be designed with the production environment in mind. A group of users might have different access rights in different environments. E.g. developers might have admin access on the development instance, but not on test or production. Instead, they might “only” have access to a backlog management tool in production and read-only access on the test environment. The developer persona in that scenario should be designed to reflect developer access in production and only allow access to the backlog management tool.

Persona roles may contain roles defined in other applications and OOTB roles – which implies a dependency from the application that contains the persona role to those applications which define other, more specific roles.

As a result, the best place to define the persona roles is the **Platform application** (as described in chapter “Application Architecture and Dependency Management”). The Platform application is the root of the dependency tree – nothing depends on it, but it depends on every other application. This makes the Platform application the natural place for persona role definitions.

It should even be avoided to establish any logical dependency to a persona role. No ACL, no business rule, no UI Action, nor any other application file should depend on (i.e. reference) a persona role. This is not only to avoid circular dependencies, but also to make a clear distinction between a regular role, which allows access to specific capabilities and data, and a persona role that should act as an umbrella for the access. Ideally any custom defined role is part of a re-usable application that may provide value to multiple production instances within the same organization or that may even be re-used across organizations. Persona roles are typically specific to the instance and its use case and tightly coupled with the demands (and naming conventions) of the organization.

When implementing persona roles, it should be considered to

- Validate that only persona roles are assigned to groups directly
- Roles are never assigned to users directly
- Ensure that persona roles are defined in a specific application (ideally the "Platform" app)

The open-source application "DataSanity" contains all technical capabilities to

- Validate that only persona roles are assigned to groups directly
- Roles are never assigned to users directly (except to technical users)

The open-source application "CodeSanity" contains all technical capabilities to

- Validate that an application does not depend on another application that contains persona roles – and as a result, persona roles are defined only in platform applications that do not have any upstream dependencies

Persona Access Matrix

Security starts with a proper definition of who should be able to access what. Any requirement should contain such statements. But making these statements in a meaningful way is difficult without the necessary pre-work.

The Persona Access Matrix is one (out of many) way(s) to develop and document access and security requirements.

As the name suggests it is a matrix with personas on one dimension and the allowed access on the other dimension.

A “persona” refers to a group of users with identical access to a system. This does not mean that each member of that group sees the exact same data.

For example, the persona “End User” may see tickets that they opened by themselves. This statement is applicable to every single user of that persona, but each user sees different tickets.

The following example matrix can serve as a starting point – it is in no way complete:

	Employee	Service Desk Agent	IT Team Member	IT Manager
Access to Employee Center Service Portal	YES	YES	YES	YES
Can see and edit incidents opened by themselves	YES	YES	YES	YES
Can see and edit all active incidents currently assigned to their team		YES	YES	YES
Can see and edit all active incidents		YES		YES
Can create reports on all incidents				YES

The persona Access Matrix defines both accesses controlled by roles and access that is controlled by filters on data – i.e. data segregation requirements.

The Persona Access Matrix should be owned by business stakeholder(s) who explicitly state their perspective on who should be able to see and do what.

It may also contain actual technical roles – but this makes it less practical as a means of communication with the business. An overview on which technical role is associated to each persona could also be generated automatically from the instance.

Technical Users

A Technical User is a user account that does not belong to a human being. Technical users e.g. are widely used to run Scheduled Jobs. Scheduled jobs run in a specific user context – and organizations are well-advised NOT to link scheduled jobs to real (human) user accounts which may be deactivated or deleted once the person leaves the company.

It is at least noteworthy that the platform does not have a specific representation of technical users in its OOTB data model. No “type” field on “sys_user” nor an extended table from “sys_user” or anything alike.

There are however the two fields “Web service access only” (web_service_access_only) and “Internal Integration User” (internal_integration_user) which are often associated with technical users.

This makes it at least difficult to even identify technical users if there is no policy, consensus, or convention within a team on how to set up technical users.

The following convention worked well for several teams:

- A technical user has either field “Web service access only” or “Internal Integration User” or both fields set to “true”
- A technical user’s “First name” is ALWAYS “Technical”
- A technical user’s “User ID” ALWAYS starts with “technical.”
- A technical user’s “Last Name” contains a hint on what the technical user is intended for (e.g., “Integration SAP”, “Performance Analytics”, or “Incident Cleanup”)
- A technical user’s “User ID” ALWAYS ends with the lowercase, dot-separated equivalent of its “Last Name” (e.g., “technical.integration.sap”, “technical.performanceanalytics”, “technical.incident.cleanup”)

Note how the dot is used to express a hierarchy in the naming scheme.

By following that convention, platform operators can:

- Filter for technical users
- Identify a technical user as the source of a log entry or any kind of errors
- Prevent that technical users are used in interactive sessions
- Apply automated processes to technical users
- Report on technical users (e.g., for security audits)

These considerations are important for the process for two reasons:

- As described in chapter “Data vs. Code” a conscious decision must be made whether technical users are to be treated as code. If that is the case, such users should be subject to a convention described in the Coding Guideline and ideally enforced through Instance Scan checks
- The Definition of Ready should define that a requirement description (in a backlog item) should define which technical user(s) are affected (created, changed)

Application Manuals and Release Notes

As always there are many feasible solutions, but the following pattern proved to be very effective for several teams:

Each application (scoped or global) should contain an application manual – and “contain” really means “contain”. Technically the application manual is implemented as a UI page, which is part of the application, and which follows a naming convention (that considers both scoped applications and global applications).

Application manual naming conventions

- for a scoped application: “<scope_name>_manual.do”
- for a global application: “<unique_shorthand>_manual.do”

Since global applications not only share the global scope but also the global namespace for UI pages, a unique shorthand must be defined for each global application.

The contents of a UI page can be authored in plain HTML – or use functions that generate parts of the page.

Most application files (Script Includes, Business Rules, UI Actions, etc.) contain fields that can be used for documentation. The technical section of the application manual can be (partly) auto-generated by surfacing the data already contained in application files.

One section of the application manual should be the release notes.

Each version should be a sub-chapter. The sub-chapter title should contain the version number and the date of the version baseline.

It is recommended to define minimum requirements to the application manual in the Coding Guideline and enforce these through Instance Scan checks.

The open-source application “DevTools” contains all technical capabilities to

- render parts of application manual UI pages

The open-source application “CodeSanity” contains Instance Scan checks to

- enforce that an application manual page exists

Managing Multiple Development Streams

Teams may choose to set up additional development instances for various reasons.

Maintaining multiple development streams however is beyond just maintaining multiple development instances – it requires a set of procedures and guardrails to manage these.

But why would teams choose to set up additional development instances?

- To perform insulated and potentially destructive experiments – e.g. to try new plugins before making the decision to really adopt and use them
- To work on backward incompatible changes to applications for future releases

The first reason calls for a SANDBOX instance. A SANDBOX instance is either a fresh z-booted instance or a clone of a development instance that is used temporarily and once the experiment is completed, it is thrown away. The new sandbox capability allows the creation of such temporary instance in an ad-hoc fashion (refer to chapter “Sandboxes”). If a new application is built as part of the experiment which is to be further used and developed it would then be transferred to the development environment before the sandbox is dismantled.

To work on both a “near-term” and a “long-term” version of the same application or even multiple, interdependent applications is yet a different challenge.

For disambiguation: if there are some applications that undergo more frequent, eventually smaller changes than other applications which are changed (or released) less frequently – and the interdependence between such Speedboats and Tankers is low – then both groups of applications can be developed, tested and shipped within one single development stream.

Managing multiple development streams is the right choice if at least one application is to be prepared for an upcoming release (“near-term”) and at the same time backward incompatible changes required for a future release (“long-term”).

In that case multiple development and test instances must be maintained.

As a convention the near-term instance stack consists of DEV1 and TEST1.

The long-term instance stack consists of DEV2 and TEST2.

The following sequence describes how applications and instances are used:

1. An application is prepared for the next release (“near-term”) on DEV1 – changes are committed to the “dev” branch
2. The work on a long-term change on the same application begins, the application is installed from the current “dev” branch to DEV2.
3. A new branch “dev2” is created for the application on DEV2.
4. (Backward incompatible) changes for the long-term release are made to the application in DEV2 and committed to “dev2”
5. Changes made for the near-term version (new features and bug fixes) to the same application on DEV1 are committed to the “dev” branch AND backported to the application on DEV2 if applicable
6. The “near-term” version is released and after a grace period the long-term version of the application becomes the new near-term version (see below for details)

Applications under development for the near-term release should be linked to branch "dev" – this should always be the case for all applications in DEV1. Applications on DEV2 can be linked to the branch "dev2" on DEV2 – however if no long-term work is required for an application the application may stay linked to the "dev" branch on DEV2 – but should not be modified on DEV2.

As stated above if there are both near-term and long-term changes to an application on both DEV1 and DEV2 – any change done in DEV1 must be reviewed and backported to DEV2 if applicable.

The procedure is comparable to the backporting procedure after performing a hotfix (see chapter "Hotfixing and Backporting").

Multiple development streams require more planning, discipline and maintenance than a single stream – and hence it must be a very conscious choice.

- Is it impossible to do the intended changes for the long-term version in the current development stream?
- Is it really required to design the changes backward incompatible?
- Can these long-term changes not be managed as unfinished work (and deployed in an inactive state) together with near-term changes?

Feedback from some teams indicate that handling unfinished work (as proposed in this document) is sometimes a challenge – it implies more careful planning and design, additional development work and documentation.

Keep in mind that managing unfinished work is – after all – a compensation strategy for the platform's inability to store all Application Files in a merge-friendly format in source control – although with the release of the ServiceNow IDE there is light at the end of the tunnel.

Managing unfinished work however is an alternative to feature branches and merges.

It is sometimes overlooked that merging also requires analysis, thoughtful planning and causes cognitive load.

Maintaining multiple development streams (for near-term and long-term changes) is a de-facto alternative to managing unfinished work on a larger scale.

But it does not reduce the management overhead, and it does by no means reduce the additional work required for managing unfinished work – as any near-term changes must be backported – multiple development streams usually increase the workload – and provide more opportunity for human mistakes.

As a result: do not maintain and manage multiple development streams if you don't have to – it is the most expensive choice and the ultima ratio.

Consider the following aspects when managing multiple development streams:

- Backlog items must specify if a requirement applies for the near-term and long-term OR only for the long-term version
- Test results must always specify the test instance on which the test was performed
- Backlog management system should support the “process fork” that results from the backporting of near-term changes
- Communication to the team regarding which application is to be modified in which instance
- Define and communicate guidance on whether long-term work is to be carried out on the DEV2 instance for ALL applications or only for a selected choice of applications

Initial setup of long-term version

1. On DEV1 baseline version from branch “dev”
2. On DEV2 import the application from source control from “dev” branch
3. On DEV2 create (and switch) to branch “dev2”

Turning the long-term version into the new near-term version

1. Baseline near-term version from branch “dev” on DEV1
2. Delete or archive (rename) the “dev” branch
3. Commit changes for application on DEV2 into “dev2” branch
4. Rename branch “dev2” to “dev”
5. On DEV1 apply remote changes (from branch “dev”) – this will overwrite the state on DEV1 by the former long-term state from the previous long-term branch (“dev2”, which is now renamed to “dev”)
6. On DEV2 switch to “dev” branch (which is the former “dev2” branch, so no changes to be expected)
7. Wait if applicable, continue work on DEV1 and commit changes to “dev” branch
8. On DEV2 apply remote changes from branch “dev”
9. On DEV2 create (and switch) to branch “dev2”
10. Continue working on near-term and long-term changes separately

Backporting near-term changes to the long-term version

1. On DEV1 apply near-term change and commit to “dev” branch (as usual)
2. Analyze the change for relevance for the long-term version. If the change is relevant (probably most of the cases) backport the change to DEV2 (via swivel chair or update sets)
3. On DEV2 commit changes to “dev2”
4. If possible, baseline the near-term and long-term versions, deploy the near-term version to TEST1 and the long-term version to TEST2 to perform simultaneous tests

Decommissioning Applications

Sometimes an application reaches its end of life: it may be replaced by something new; the capability is now provided OOTB, or the contained feature is no longer needed.

When decommissioning an application, the following aspects should be considered:

- Any tables or columns defined by the application are going to be removed and the contained data is going to be deleted. There is no undo option available to bring back deleted tables or columns – other than using a backup. It needs to be ensured that all data contained in such tables or columns is no longer needed.
- Other applications – for some reason or the other – may still depend on the application. Such dependencies must be identified, and the dependent applications need to be refactored to remove these dependencies.

The following procedure contains 3 phases:

1. Deprecation
2. Deactivation
3. Uninstallation

Phase 1 – Deprecation

“Deprecation” in this context means to document and communicate the planned decommissioning of an application so that everyone else can prepare for the actual uninstallation of the application.

1. Verify that the capabilities of the to-be-decommissioned application are no longer required
2. Communicate the deprecation of the application in the developer community
3. Identify any yet undocumented dependencies to the to-be-decommissioned application
4. Establish a list of applications that have dependencies to the to-be-decommissioned application and plan for changes required to resolve these dependencies
5. Establish a list of all associated technical users, credentials, connections, and other configuration assets associated with (but not contained in) the to-be-decommissioned application. Keep in mind that such assets may also be used by other applications!
6. If multiple applications are to be decommissioned in one go, the exact order for their uninstallation must be determined. Apps should be uninstalled in reverse order of their original installation – i.e., their dependencies. If App A depends on app B. App B must be installed first, then App A. When uninstalling, uninstall app A first, then uninstall app B.
7. Prepare a release package with updated versions of all affected applications where the dependencies to the to-be-decommissioned application are removed

Phase 2 – Deactivation

“Deactivation” of an application in this context means to turn the application into a state in which it does not actively trigger any activities in production. All its capabilities are still present, but they should not impact user experience, perform no data processing, or interact with external systems. This should give anyone the opportunity to identify any service degradation caused by the deactivation which has not been identified in Phase 1.

1. Identify means to “deactivate” the to-be-decommissioned application. This may include:
 - a. Deactivate Scheduled Jobs
 - b. Deactivate Business Rules and Flows
 - c. Prevent access to user interfaces
 - d. Prevent interaction with external systems
2. Prepare the “inactive” version of the to-be-decommissioned application
3. Deploy the “inactive” version of the to-be-decommissioned application

Phase 3 – Uninstallation

The “Uninstallation” is the physical removal of the application from the target instance. This includes data contained in tables and columns defined in the application.

It is good practice to wait for a couple of weeks or even months before even starting the uninstallation phase – just to make sure that really no one misses the deactivated capabilities.

1. Export any data contained in to-be-deleted tables and from tables to which the to-be-decommissioned application adds columns
2. Verify that all formerly dependent applications are updated and that these new versions no longer have dependencies to the to-be-decommissioned application
3. Delete the application(s) according to the reverse order of their initial installation on the target instance
4. Delete the application(s)' sys_repo_config records
5. Delete all associated technical users, credentials and connection aliases

Organizing Documentation

This guidance refers to documentation usually maintained in a knowledge management system – like Confluence or any other wiki-style system.

No matter what system is in use to document the application, work procedures, goals, technical architecture, and all other aspects related to the operation of a ServiceNow platform – the challenges related to documentation are mostly the same:

- Specific information is difficult to find
- Information is often outdated and badly maintained
- Multiple, often conflicting versions of documentation exist

The following guidance helped improving documentation quality and documentation effectiveness for teams:

- Structure information based on the process roles: Information should not be structured by “topics” or by “the team that created it” but by the audience it addresses. Each process role (e.g., Developers, Product Owners, Deployers) has their own section.
- Each of these process role specific sections is owned by that person or a delegate.
- Each owner must review the available information in their sections on a regular basis – and rigorously remove anything that is out-of-date or no longer relevant to the team.
- Each process role specific section should contain a “need to know” area – where new team members can find all information they are required to know.
- Each process role specific section should contain a “how to” area – where practical guidance can be found on how to do things.
- The documentation around the communities of practice should also be visible on the highest level - since the participants of the communities of practice in most cases have various process roles.
- The Definition of Ready and the Definition of Done should also be placed on the highest level to be as visible as possible.

An example table of contents

1. Definition of Ready
2. Definition of Done
3. For Product Owners
 - a. Onboarding / Need to know
 - b. How to
 - i. Author great backlog items
 - ii. Prioritize backlog items to maximize business value
4. For Developers
 - a. Onboarding / Need to know
 - b. How to
 - i. Getting to done
 - ii. Scan Application Files against the Coding Guideline
 - iii. Handle unfinished work
 - iv. Create unit tests for Script Includes
 - v. Create new CodeSanity (Instance Scan) Checks
 - vi. How to update open-source applications from GitHub
5. For Deployers
 - a. Onboarding / Need to know
 - b. How to
 - i. Prepare a deployment
 - ii. Perform a deployment
 - iii. Handle a failed deployment
6. Communities of Practice
 - a. Agile Backlog Management
 - b. Architecture
 - c. Development
 - d. Deployment and Operations
 - e. Manual Testing

High Performance Background Jobs

Whenever large amounts of data, or continuous streams of data need to be processed, solutions must be built with execution performance in mind.

Here are some generic guiderails that should help make better solution design decisions:

1. **Scripted Scheduled Jobs vs. Flows**
Use Scripted Scheduled Jobs instead of Flows. Flows impose some additional processing time to every single step. The ease-of-use of Flow Designer comes at a price.
Flows may be easier to build from scratch, but performance optimization is all about refactoring and rearranging parts of the logic. This is easier and faster with plain vanilla JavaScript than with Flow Designer.
2. **Script Includes**
Implement the actual complex logic in Script Includes. This allows for flexibility on how the complex logic is triggered (Scheduled Jobs, Business Rules, Flows, etc.). Ideally the Scheduled Job Script is a one-liner calling a single function from a Script Include.
3. **Idempotency**
All steps must be implemented in a way so that re-processing the same data yields the same repeatable desired results. A raw data item being processed twice (erroneously) should not produce duplicate records of processed data.
4. **Parallel Processing**
Design for parallel processing of large data sets. This can be done through setting up a Scheduled Job that executes more often than it is expected to run – which leads to multiple instances of the same Scheduled Job to run in parallel. E.g. a job is triggered every minute but expected to run for 5 minutes would result in up to 5 instances of the job running in parallel at any time.
Alternatively, a Schedule Job can start multiple backend processes using the `GlideScriptedHierarchicalWorker` API.
This requires that these multiple instances of a single Scheduled Job or process have awareness in real-time which parts of the larger data set have already been processed and which are not. Whenever a process picks a chunk of unprocessed data, such data must be flagged so that other instances of the same process would not start working on the same chunk of data. That allows multiple instances of processes working on the same data – each on different chunks.
5. **Transaction Caching**
Processing multiple raw data items may require additional data retrieval. If multiple processes working on different data chunks in parallel, the same existing data records may be retrieved from the database repeatedly.
In some cases, these duplicate data retrievals can be avoided by caching the results in memory and look them up in memory instead of asking the database again for the same information.
Caching in memory can be done by storing retrieved data in a key-value-JavaScript object that remains in memory during the runtime of a single process instance.

6. Stream Processing

If the processing of data can be split into different steps it might be feasible to separate these steps into separate Scheduled Jobs – each of which may or may not run in parallel. E.g. If data is first received from an external system, separating the retrieval step may reduce the overall idle time of jobs processing the data due to network latency between systems.

Another example: if one step is more dependent on retrieving a lot of data from the local database, while another step is more expensive in terms of computation, separating these steps can help making each of the processes working more efficiently. This pattern may require storing raw, partly processed, and processed data in separate tables. E.g. The data received could be stored first in a table that represents its unprocessed form (which may cause additional processing time) and then multiple processes to pick chunks of data to be processed from that table to transform the data into its final state.

Refactoring Customizations into Applications

The approach to ship everything as applications is radical. It changes the way changes are transported and managed.

But it also requires a change in how to make changes in the first place.

This chapter provides guidance on how to make changes to application files that are often modified in global scope (for the typical ITSM use cases) or in plugin scopes (like CSM).

The following guidance is equally relevant for customizations that already exist or any new customizations that are about to be implemented.

Developers should keep the following general guidance in mind:

3. Everything must be shipped as an application – no update sets or manual imports are allowed
4. Code (anything derived from sys_metadata) must not depend on data (anything that is NOT derived from sys_metadata)
5. Code must not contain credentials, client secrets or access tokens (as these would otherwise end up in the source control repository)

Unfortunately, in the past ServiceNow has made some architectural decisions that make it challenging to keep up with these principles. The good news is that there are strategies available to make it work – although some are not obvious.

DevTools

To ship everything as an app, there are four tactics available. This chapter will outline when to use which and provide specific guidance on selected examples.

Please note that most guidance provided in this document is agnostic to the use of the referenced open-source application DevTools – but following the guidance is way easier with DevTools.

This chapter is different. The ServiceNow platform (still) has some built-in limitations that make the use of applications as the one and only deployment mechanism more difficult than it should be.

The open-source application DevTools contains various capabilities to work around these limitations.

See <https://www.wildgrube.com/servicenow-devtools> for detailed documentation of DevTools capabilities.

Four Tactics

The four tactics are:

1. Create a new Application File in a scoped custom application
2. Create new or modify existing Application Files through Installation Scripts
(See chapter "Installation Scripts" and the InstallerAPI contained in DevTools, see <https://www.wildgrube.com/servicenow-devtools>)
3. Create a new Application File in a global custom application
4. Transport records using DevTools Record Transporter
(See <https://www.wildgrube.com/servicenow-devtools> chapter "Record Transporter")

The four tactics are sorted by preference – they should be explored in exactly that order – always.

To apply these tactics and the following guidance, two new concepts must be introduced: **Highlander Records** and the **Copy-And-Deactivate-Approach**.

Further it must be understood what **hard and soft cross scope limitations** are.

Tactic 4 – relies on the capability to "pack" any record into an application – which even may include packing Update Sets – using the DevTools **Record Transporter**.

Use the **Decision Tree** to choose the best combination of tactics. One can find more specific guidance for specific Application File types.

Highlander Records

Application Files usually have some sort of reference – this is often a table (e.g. Business Rules, Flows) or an event (Email Notifications) – and more of the same type can be added without constraints.

That is not the case with **Highlander Records**. As their name suggests, there can only be one.

Examples are:

sys_db_object – the definition record of a table – there is only one per table

sys_dictionary – the definition record of a table field – for each field in a table there is exactly one Dictionary record.

sys_dictionary_override – a record that defines deviations from settings in a Dictionary record for the same field in an extended table – for each field in an extended table there is only one Dictionary Override record.

sys_properties – A System Property has a unique name.

sys_script_include – A Script Include has a unique name – which also forms its API name by which it is called in code.

sys_ui_form/sys_ui_list – Form and List view definitions are unique per “View” and “Table”, and all their related records are captured in a single update record so their individual parts cannot be spread across multiple scopes.

sys_choice_set – A choice set is unique per field, and its sys_choice records are captured into a single update record so their individual parts cannot be spread across scopes.

This list is not conclusive, but these are the most common and important **Highlander Records**.

Any other Application File type allows many records to coexist with the same reference pattern. E.g. there can be any number of Business Rules that act on the “update” event on the “incident” table, any number of UI Actions on the “task” table or any number of Scheduled Jobs running at a defined point of time.

Copy-And-Deactivate-Approach

When making changes to OOTB records the official guidance from ServiceNow is to modify the original OOTB record. This will add these records to the "Skipped Records" list during a family upgrade, so that administrators can make decisions what to do: keep the customizations, use the updated OOTB version or backport the earlier customizations into the newer version.

To be precise, this is the guidance for most Application File types. For some newer (and more complex) Application Files (like Flows or AI Skills) the standard procedure is to copy the OOTB version into a scoped app and THEN to modify the copied version. For some of these Application File types, the platform prevents any modification to the OOTB version and only allows developers to modify a copy.

For over a decade many ServiceNow administrators worked through the list of "Skipped Records" once or twice a year. In many organizations this period causes a full stop of all development work for a week or even longer.

For large, highly effective high-velocity teams this is not practical (See also chapter "Family Upgrades").

To ship everything as apps, these changes to OOTB records are also inherently difficult to deploy, hence the guidance in this document is to:

Re-create a new record (in a scoped application), replicate the capabilities of the original record and deactivate the original (through Installation Scripts) – or short: **copy-and-deactivate**.

As in many cases, this implies to re-create code that is run from global scope or a plugin scope into the scope of a custom application. To do this, cross-scope-scripting techniques are required.

See chapter "Cross Scope Scripting" on how cross scope scripting works.

Refer to the DevTools functions `RunScriptInGlobalScope()` and `RunScriptInScope()` on how to execute any code from any scope as if it was run from a different scope.

Refer to DevTools `InstallerAPI` class and its `Activate*` functions on how to deactivate existing records.

Hard and Soft Cross Scope Limitations

Some cross scope limitations are “**soft**” – this means that these can be worked around through cross scope scripting techniques, or the standard platform GUI simply does not allow to make the appropriate changes (i.e. by not offering fields or tables from other scopes).

DevTools modifies the form views for some (yet not all) Application File types to enable cross scope configuration to work around these **soft** cross scope limitations.

Some cross scope limitations however are “**hard**”. Even if the record was configured properly, it would just not work.

Examples are:

6. Scripted ACLs on tables (if the ACL is not exactly in the same scope as the table)
7. Client Scripts where the scripts executed in the client do not have access to fields defined in different scopes

Some cross scope limitations are no limitations at all. Some (yet not all) OOTB tables are shipped with the flag “Allow Configuration” set to “false” which causes them to be hidden in the table dropdown. Setting the flag to “true” will make them selectable in most cases.

Record Transporter

The Record Transporter is a DevTools capability that allows developers to pack any record (code or data) into a so-called Transporter Package.

This can be achieved in three ways:

1. Manually in the GUI
2. Via script that is executed when needed or each time when the application is baselined
3. Through Update Sets (see chapter “If All Else Fails – Defaulting to Update Sets”)

The Record Transporter was originally built to capture data records into applications – but can also be used to capture Application Files.

Application Files can be packed directly into a Transporter Package or indirectly through Update Sets.

The latter should be considered as a fallback option if all else has failed – see chapter “If All Else Fails – Defaulting to Update Sets”.

For more information, refer to the DevTools documentation:

<https://www.wildgrube.com/servicenow-devtools> chapters “Record Transporter” and “Update Sets in Record Transporter Packages”.

Decision Tree

The following decision tree provides guidance on when to use which of the four tactics or a combination of them.

- Modification of an existing OOTB record
 - a. Record is a **Highlander Record**
 - i. Tactic 2: Modify existing Application Files through Installation Scripts
 - b. Record is NOT a **Highlander Record**
 - i. **Soft** (or no) Cross Scope Limitation
 - 1. Tactic 1: Create a new Application File in a scoped custom application
 - 2. Tactic 2: Deactivate existing Application Files through Installation Scripts
 - ii. **Hard** Cross Scope Limitation
 - 1. Tactic 2: Modify Application Files through Installation Scripts
 - 2. If Tactic 2 is impractical
 - a. Tactic 2: Deactivate Application Files through Installation Scripts
 - b. Create a NEW record
- Create a NEW record
 - a. **Soft** (or no) Cross Scope Limitation
 - i. Tactic 1: Create a new Application File in a scoped custom application
 - b. **Hard** Cross Scope Limitation
 - i. Tactic 2: Create Application Files through Installation Scripts
 - ii. If Tactic 2 is impractical
 - 1. Tactic 3: Create a new Application File in a global custom application
 - iii. If Tactic 3 is impractical
 - 1. Tactic 4: Transport records using DevTools Record Transporter

The following chapters provide guidance on how to modify different types of OOTB Application Files (in alphabetical order).

ACLs

ACLs are **NOT Highlander Records**.

Access Control List Records come in two flavors:

1. Not Scripted – no cross scope limitations apply
In that case, use Tactic 1: Create a new Application File in a scoped custom application
And if applicable use Tactic 2: Deactivate existing Application Files through Installation Scripts
2. Scripted (Advanced) – hard cross scope restrictions apply
If the ACL must contain a scripted condition, the ACL must be in the same scope as the table. This applies to OOTB tables and custom tables defined in custom applications.
In that case use Tactic 2: Create Application Files through Installation Scripts.
Use the DevTools function `InstallerAPI::RecordInsertOrUpdate()` – unless there is a more specific function available.
And if applicable use Tactic 2: Deactivate existing Application Files through Installation Scripts.

To apply Tactic 2 on ACLs the DevTools function `InstallerAPI::ActivateACL()` should be used.

A scripted ACL that sits in the same scope as the table to which it applies may require some Cross Scope Scripting techniques to access Glide APIs which are only available in global scope scripts.

Business Rules

Business Rules are **NOT Highlander Records** and can be added as part of scoped applications even when they relate to tables in other scopes.

An OOTB Business Rule should always be “modified” through the “Copy-and-Deactivate-Approach”.

So, use Tactic 1: Create a new Application File in a scoped custom application and Tactic 2: Deactivate existing Application Files through Installation Scripts.

The DevTools function `InstallerAPI::ActivateBusinessRule()` can be used to apply Tactic 2.

A scripted Business Rule that is contained in a scoped application (and not in the global scope) may require some Cross Scope Scripting techniques to access Glide APIs which are only available in global scope scripts.

The DevTools function `RunScriptInGlobalScope()` can help to execute code as if it was run in global scope from within a scoped script.

Choices

Choices are the selectable line items in the dropdown in a form.

Choices (or to be precise the Choice Set) is like a **Highlander Record** as it is also shipped as a monolithic package.

As a result, a single Choice to an existing Choice Set cannot be added by a custom application.

The only feasible option is to apply Tactic 2: Modify existing Application Files through Installation Scripts.

The following DevTools functions can be used to modify OOTB Choices:

8. `InstallerAPI::ChoiceInsertOrUpdate()`
9. `InstallerAPI::ActivateChoice()`
10. `InstallerAPI::ChoiceSetDependentValue()`

Flows

The regular official guidance on making changes to OOTB records is to modify the original record. The guidance in this document is to do the exact opposite: to apply the copy-and-deactivate approach.

Interestingly, for Flows ServiceNow recommends – even enforces – the copy-and deactivate approach.

Many (yet not all) OOTB Flows are shipped with a read-only policy. These Flows can only be deactivated once – and never be reactivated.

In Workflow Studio, a Flow can be copied into a different application using the “Copy flow” option.

No further magic is required here?

Not exactly, flows (and scripts executed as part of a flow) are subject to the same cross scope limitations – and may not be able to access tables or API capabilities when executed from a different scope than its original – so some adjustments may be required in the copied flow.

Form and List Views

Forms and List Views are tricky.

Form and List view definitions must be considered **Highlander Records** per combination of table and view.

Forms consist of a collection of Form Sections. Form Sections are made up of fields, formatters and annotations. Forms are also associated with Related List definitions that consist of references to tables to be displayed as tabs under the form.

List Views are also **Highlander Records** per combination of View and table and consist of the columns to be displayed in the list.

This sounds as if there are a lot of records involved, and that is true from a database perspective. However, when deployed (no matter whether as update sets or as part of an application through AppRepo or source control) these items are packed together into a monolith which can only be shipped as one. Hence the concept of the **Highlander Record** applies – even though, these Form and List Views are made up of multiple records.

The practical implication is that A Form or List view for a table in a specific View cannot be mixed with records provided by other scopes (and hence be extended by an application).

Bottom line is, to modify OOTB views the ONLY feasible way is through Tactic 2: Modify existing Application Files through Installation Scripts.

Since the data model is not trivial the DevTools class InstallerAPI contains several functions to make changes to existing Form and List views:

11. InstallerAPI::DefineForm()
12. InstallerAPI::DefineFormSection()
13. InstallerAPI::DefineRelatedList()
14. InstallerAPI::DefineList()

These functions allow us to modify all aspects of an existing OOTB view.

Note that view definitions are cached and any change made might require clearing the cache before taking effect.

However, there is an alternative: Instead of making changes to OOTB views, new views can be defined (within the scope of an application) and then enforced by adding View Rules (sysrule_view).

This has several consequences:

- The user can no longer choose the view by themselves – which takes away some flexibility but at the same time streamlines the user experience and causes more consistency
- The form and list views must be designed to cover multiple use cases at once – which may require more analysis and design work upfront

Whether these are advantages or disadvantages is subjective.

Script Includes

All Script Includes are **Highlander Records**. They identify by a unique API name. There can only be one single Script Include using a specific API name – and any function calls and class instantiations will be handled by the implementation contained in that Script Include.

Modifying existing OOTB Script Include thus requires a special method when applying the two tactics:

1. Create a new Application File in a scoped custom application
2. Create new or modify existing Application Files through Installation Scripts

The existing OOTB Script Include will remain the “Entry Point” to the customized script.

Needless to say, it should be checked if a customization or modification of an OOTB Script Include is even required in the first place.

Can the actual hook point – i.e. the component that uses the code contained in the OOTB Script Include (e.g. a UI Action or Business Rule) - be replaced?

Can an additional Business Rule or Flow be added to trigger the intended, deviating behavior?

The following steps should only be considered if all other options have been ruled out.

To “modify” an OOTB Script Include the “Copy-and-Deactivate-Approach” needs to be slightly adapted.

In case of a Script Include it is NOT “Copy-and-Deactivate” but “Copy-and-Code-Injection”.

Most OOTB Script Includes contain a class with one or more member functions.

As part of a customization, the behavior of one or more of these member functions needs to be changed.

The interface (i.e. names and structure) of the class contained in the Script Include must remain unchanged.

To apply tactic 1, create one or more Script Includes that contain single functions.

To apply tactic 2, write installation scripts that inject the execution of these functions contained in the new Script Includes into the member functions of the original OOTB Script Include.

Some OOTB Script Includes are already prepared for this pattern. In some cases, there are combinations of a class “SomeClass” (intended for modification) and a parent class “SomeClassSNC” which is meant to remain unaltered.

The following two Examples demonstrate these two variants:

Example 1 – Script Include “ChangeConflictAPI”

The ChangeConflictAPI Script Include encapsulates a functionality to detect conflict for a change. One member function returns the role required to use its functions.

```
var ChangeConflictAPI = Class.create();
ChangeConflictAPI.prototype =
{
    CHANGE_CONFLICT_ROLE: "change.conflict.role",

    initialize: function(gr)
    {
        this._log = new GSLog(ChangeCheckConflicts.CHANGE_CONFLICT_LOG, this.type);

        if (this._log.atLevel(GSLog.DEBUG))
            this._log.logDebug("[initialize]");

        this._gr = gr;
    },

    [...]

    _canCheckConflict: function()
    {
        return gs.hasRole(gs.getProperty(this.CHANGE_CONFLICT_ROLE));
    },

    type: 'ChangeConflictAPI'
};
```

Assuming that there is a requirement to not only check a role (defined in a system property) if a user is entitled to start the process more complex checks need to be performed. The function `_canCheckConflict()` must be replaced.

Let's say, the user should not only have that role but must explicitly be able to read the given record. A new function would have to be created in a new Script Include in a custom application that could look like this:

```
function IsUserEntitledToCheckConflicts(grChange)
{
    If (gs.hasRole(gs.getProperty(this.CHANGE_CONFLICT_ROLE)) == false)
        return false;
    return grChange.canRead();
}
```

Then an installation script should “inject” the new function into the OOTB Script Include. This injection must be done idempotently – so that repeated executions of the installation script always lead to the same result:

```
var ChangeConflictAPI = Class.create();
ChangeConflictAPI.prototype =
{
  CHANGE_CONFLICT_ROLE: "change.conflict.role",

  initialize: function(gr)
  {
    this._log = new GSLog(ChangeCheckConflicts.CHANGE_CONFLICT_LOG, this.type);

    if (this._log.atLevel(GSLog.DEBUG))
      this._log.logDebug("[initialize]");

    this._gr = gr;
  },

  [...],

  _canCheckConflict: function()
  {
    return x_my_scope.IsUserEntitledToCheckConflicts(this._gr);
    return gs.hasRole(gs.getProperty(this.CHANGE_CONFLICT_ROLE));
  },
  type: 'ChangeConflictAPI'
};
```

The first line in the member function “_canCheckConflict” contains a “return” statement and hence will always prevent the execution of the rest of the code in the function. This is intentional to preserve the original code for reference.

This may however raise findings with linter checks which should be ignored or muted in this case.

Example 2 – Script Includes “ChangeConflictSchedule” and “ChangeConflictScheduleSNC”

The “ChangeConflictScheduleSNC” Script Include contains functionality to calculate and detect conflicts. The Script Include itself is read-only and cannot be changed.

The Script Include “ChangeConflictSchedule” is explicitly intended to be changed. The OOTB version looks like this:

```
var ChangeConflictSchedule = Class.create();
ChangeConflictSchedule.prototype = Object.extendObject(ChangeConflictScheduleSNC,
{
  initialize: function()
  {
    ChangeConflictScheduleSNC.prototype.initialize.apply(this, arguments);
  },
  type: 'ChangeConflictSchedule'
});
```

Assuming that there is a requirement to change the wording provided in case of unavailability, the function `getReasonForUnavailability()` (which is defined in Script Include “ChangeConflictScheduleSNC”) can be overridden and a separate function from a separate Script Include should be called. This modification should again be done by an installation script with idempotency awareness:

```
var ChangeConflictSchedule = Class.create();
ChangeConflictSchedule.prototype = Object.extendObject(ChangeConflictScheduleSNC,
{
  initialize: function()
  {
    ChangeConflictScheduleSNC.prototype.initialize.apply(this, arguments);
  },
  getReasonForUnavailability: function(reason, formatValues)
  {
    return x_my_scope.GetReasonForUnavailability(reason, formatValues);
  },
  type: 'ChangeConflictSchedule'
});
```

This maintains the defined interface of the parent class `ChangeConflictScheduleSNC`.

Service Portal Widgets

Widgets are not **Highlander Records**. There can be any number of copies or variants of a Widget. However, Widgets are used in Service Portal pages – and at any given time there can only be one Widget be embedded in a specific page at a specific place.

While Widgets are not Highlanders, the `sp_instance` records which arrange Widgets into pages are.

For Widgets (and Pages alike) the Copy-And-Deactivate approach is to be slightly adapted: It is not “copy-and-deactivate”, but “copy-and-rewire” – as the new, copied version of an OOTB Widget is arranged into a page by a `sp_instance` record, it is the `sp_instance` record that needs to be modified in order to use the new customized version of a Widget in an existing page.

The following tactics are to be applied:

1. Create a new Application File in a scoped custom application
2. Create new or modify existing Application Files through Installation Scripts

To use a customized version of a Widget in an OOTB page (`sp_page`) the corresponding (OOTB) `sp_instance` record must be adapted – ideally through an installation script.

Widgets consist of client side and server side code. The server side code is subject to the same rules as scripts contained in Business Rules, Script Includes and any other server-side JavaScript code.

Consequently, an OOTB Widget (in the global or a plugin scope) that is copied into a scoped application may require some code refactoring to work.

The chapter “Cross-Scope Scripting” contains guidance on how to refactor code that faces cross-scope restrictions.

For many OOTB Widgets, this refactoring is also a perfect opportunity to apply the idea of separating concerns more clearly. A lot of Widgets contain code that represents business logic and data retrieval directly in the server-side scripts – instead of using Script Includes to do the heavy lifting.

A server-side script of a Widget should only proxy parameters into functions provided in Script Includes and prepare data provided by these functions for client-side use.

Ideally even the use of `GlideRecord` in a server-side Widget script is sanctioned by the Coding Guideline.

System Properties

System Properties are **Highlander Records**, so for existing System Properties Tactic 2 is always the right choice: to modify existing Application Files through Installation Scripts.

This also applies to System Properties which are defined in the platform but do not (yet) exist on an Instance. The ServiceNow documentation is full of references to System Properties which are supported but do not exist on a z-booted instance.

The DevTools function to be used is `InstallerAPI::SetProperty()`.

For NEW System Properties – i.e. System Properties that are used to configure behavior of capabilities and features built into custom applications – there are two choices:

1. Adding System Properties as records to a custom application (and hence make them part of the application scope).
Use this approach if the System Property defines behaviors that shall change during development and deployment – and hence be controlled by the development team.
2. Manually set System Properties in the global scope.
Use this approach if the System Property defines behavior that can be modified ANYTIME by the administrators on the production instance.
An Installation Script may set a default, if the System Property does not exist yet, but should not overwrite any existing value that may have been changed since the last execution of the Installation Script.
The DevTools function to be used is `InstallerAPI::InstallGlobalSystemProperty()`

Tables, Dictionary Entries, Dictionary Overrides and Attributes

All the records that define the low-level structure of the data model require special handling. All of them are **Highlander Records**. OOTB records can only be modified using Tactic 2: Modify Application Files through Installation Scripts.

This chapter applies to the following Application File types:

- Tables (sys_ob_object)
- Dictionary Entries (sys_dictionary) – both for table columns as well as records specifying additional attributes for tables
- Dictionary Overrides (sys_dictionary_override)
- Attributes (sys_schema_attribute_m2m)

Most OOTB Application File records can be identified by their Sys IDs – as their identifier is identical across all instances in the world. But not for the Application File types defining the data model. Many (not all) of them have globally unique Sys IDs – which differ from instance to instance. The practical consequence is that to make scripted changes to these records their Sys ID cannot be used to identify them.

The DevTools class InstallerAPI contains several functions to make changes to these Application Files:

15. InstallerAPI::SetTableValue()
16. InstallerAPI::SetTableDefaultOrder()
17. InstallerAPI::DictionarySetValue()
18. InstallerAPI::DictionaryOverrideSetValue()
19. InstallerAPI::SetTableAttribute()
20. InstallerAPI::SetTableColumnAttribute()

UI Policies and UI Policy Actions

UI Policies are NOT **Highlander Records**, any number of UI Policies can be associated to a table, UI Policy Actions WITHIN a UI Policy, however, are **Highlander Records**. For each field there can only be one UI Policy Action per UI Policy.

OOTB the UI Policy form does not allow a scoped UI Policy to be associated to an out-of-scope table. Even the "Allow Configuration" setting does not change that. The "DevTools" view for the UI Policy, however, allows to enter any table to a UI Policy.

Configuring the UI Policy Action associated to a cross scope UI Policy is however tricky.

The UI Policy Action form does not allow the user to select any fields for an out-of-scope table.

The DevTools class InstallerAPI contains a function to create and modify UI Policy Actions:

```
21. InstallerAPI::DefineUiPolicyAction()
```

If All Else Fails – Defaulting to Update Sets

"History Doesn't Repeat Itself, but It Often Rhymes"

Mark Twain

If the reader made it so far, this sounds absurd: but it is true: Sometimes Update Sets may just be the only practical, straight forward way to capture a change. If a team has decided to ship everything as applications, this would be a problem. The DevTools Record Transporter comes to the rescue.

An Update Set can be packed into a Record Transporter Package. An admin user can select the "Transporter Package Application" for an Update Set on the Update Set form. If the Update Set is in state "Complete", an admin user can select the link "Pack Update Set into Record Transporter" on the Update Set form. The Update Set (and its siblings – the Update XML records) are then packed into a new Transporter Package in the selected application.

The Transporter Package is automatically configured to be unpacked automatically when running the application's installation scripts (see chapter "Installation Scripts"). Further the Update Set(s) contained in the Transporter Package will be automatically committed.

Since the order of Update Sets should be maintained, the Transporter Packages are assigned an increasing order number which is used when unpacking and committing.

Please note that this method should be considered as a fallback method if there is no feasible way to capture and transport changed Application Files. Once used, Technical Debt records are created for each packed Update Set that should remind the developer to find a better solution in the future (see chapter "Technical Debt Management").

This capability was built to make it easier for teams to transform their Update Set-based process into a more robust applications-based deployment process.

However, it should NOT be seen as a card blanche for not educating a team and not doing the refactoring on the long run.

Making the Change

Pills and Surgery vs. Life Change

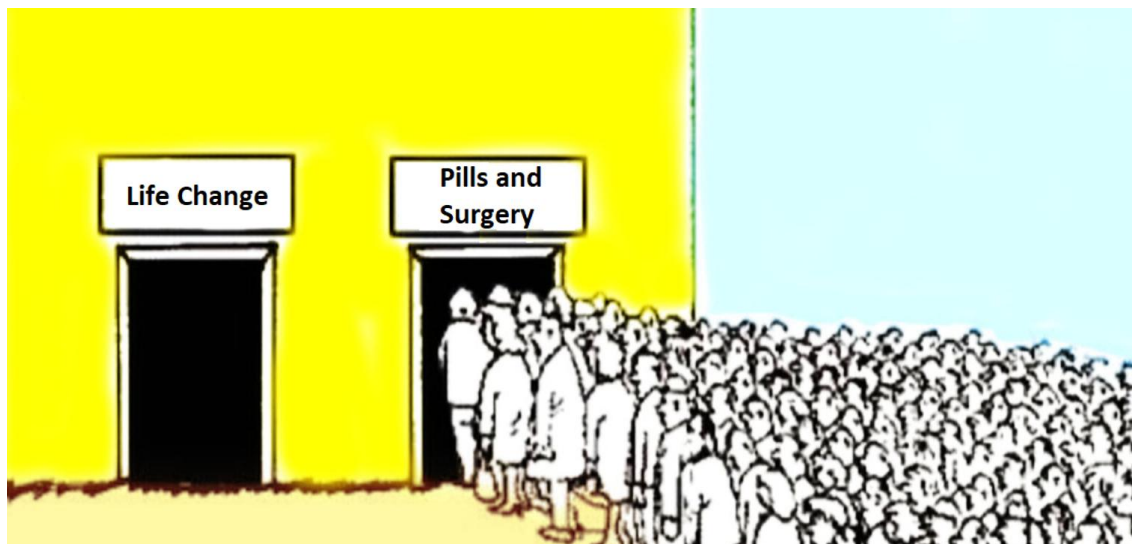
The process proposed in this document requires a substantial change. The change is technical. But that is the minor part. The bigger part of the change is about the change of people's behaviors.

As you can guess, changing people's behaviors is the bigger challenge. No new technology, no newly defined process or organization structure will yield any tangible results if people stick with their long-engrained, well-rehearsed routines.

When facing personal or health problems, many people ask for pills and surgery instead of doing the hard work of personal development and changing their habits to change their life.

The same dynamic can be observed in organizations.

So, keep in mind: technology is secondary, it's all about people!



Getting out of the Comfort Zone

It becomes clearer what the actual challenge is. It is not to write a few documents, get some technical capabilities in place, define the coding guideline, and set up some new rules for the team.

The actual challenge is to lead a team to a different way of working than what they are used to. Things must be unlearned. Established routines must end. New techniques must be acquired, and new routines be rehearsed.

This forces people to get out of their comfort zone.

Different people will respond in different ways to these asks. Some might welcome the initiative with open arms, be fully supportive, creative, turn into champions and lead the pack.

Others may respond with criticism, cynicism, silent blockades, or even open revolt.

Be prepared to get to know co-workers, contractors, managers, and other stakeholders in a way you did not see them before – in a positive and negative sense.

Zero-Tolerance Mind-Set

At various occasions the guidance in this document is not to accept a single deviation from set rules. All code must be fully compliant with the coding guideline. Not a single automated test may fail. A deployment instruction for a release package must not contain a single reference to an update set or manual installation step.

This sounds hard – but it is crucial. Any deviation, any exception may have detrimental effects. If on one occasion it is ok to accept failing tests, then maybe it is also ok on other occasions, and then, what is the point to ask the team to write these tests in the first place?

It must be clear to everyone that established rules apply to all teams, to all individuals, in all cases.

These rules, however, be they the coding guideline, the definition of ready or any other guidance, must be malleable and the team should have a say in their definition. Ideally it's the team itself that writes, adjusts, and agrees on the rules. Only then can we expect full support for what is being asked.

The Process Evangelist

Some readers of this document – especially those who made it so far into this chapter – may see themselves as facilitators, as agents of change, as someone who is or may be tasked to implement the guidance that is described in this document. Those readers – you – may become a Process Evangelist.

As every organization, team, project, environment is different, providing a definitive role description is downright unrealistic – hence the Process Evangelist is not one of the defined process roles.

The following list is an attempt to describe the role – but rather than copying it – use it to start the discussion:

- Evangelizes the process to management, developers and all other stakeholders
- Guides the organization on the steps and provides the draft backlog for the introduction of the process
- Coaches and supports all governance roles
- Temporarily embodies governance roles until the right person is identified and established
- Initiates the application architecture design
- Facilitates the Community of Practice meetings until permanent facilitators are identified
- Coaches and supports architects and developers to enable them for execution
- Coaches and supports product owners and everyone who defines requirements

No doubt, being a Process Evangelist requires courage, empathy, persistence, and an undaunted character.

Aligning the Organization and Technology

"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."

Melvin E. Conway

This statement is widely known as "**Conway's Law**". To understand and to act on it is in fact crucial to the success of the process described in this document.

Chapter "Application Architecture and Dependency Management" describes that all code, all customizations should be shipped as applications and that these applications have technical dependencies to each other. The applications are the nodes; the dependencies are the edges of a network.

Chapter "Application Architecture Design" provides hands-on guidance on how to make decisions about what nodes and which edges should exist.

These decisions form a technical network.

One of the most important aspects of this design is the decision of which team (within a larger developer community) should be responsible for which application.

This turns teams into nodes and their communication lines into edges of yet another network: an organizational network.

The technical network and the organizational network must be aligned. The organizational network must be designed to produce the desired technical network.

In other words: the team structure must be designed to produce applications that fit into the application architecture design.

Developers that work on applications that technically depend on each other must communicate. The team structure and the resulting communication structures should support these communication lines.

In an ideal world, the best possible application architecture could be designed without any organizational constraints - ignoring people, their skills, their location, language, culture, distribution to different organizations etc.

In the real world, it is all about people. The most brilliant technical design – brilliant from a pure technical perspective – is useless if people and organizations are unable to build and support it.

When designing the application architecture, the organizational structure, the teams, the assignment of applications and the assignment of individuals to teams must all be considered.

If this is not done, or not done right, an organization may accumulate **Social Debt**.

The paper "What Is Social Debt in Software Engineering?", published by Tamburri et. al. (2013) defines "Social debt in software engineering" as "unforeseen project cost connected to a suboptimal development community. The causes of suboptimal development communities can be many, ranging from global distance to organizational barriers to wrong or uninformed socio-technical decisions (i.e., decisions that influence both social and technical aspects of software development)."

Aligning the organization and technology is crucial.

Refactoring existing Code and Customizations into Applications

If an organization adopts the process and techniques described in this document right from the start, in a greenfield project on a fresh z-booted instance, great. There are no existing customizations to be refactored – which does not make the task easier, though.

Most organizations do not enjoy this luxury. For most organizations the reality is that customizations have been made - these were deployed through update sets in the past - and instances were cloned from time to time. Probably a few apps have been created, some of them may have been deployed to production – and some of these apps may still come with installation instructions that involve manual xml imports or (again) update sets to be deployed.

This chapter provides guidance on how to plan and organize the refactoring of existing code (i.e., customizations) into applications.

For starters, this does not mean to “refactor” update sets into applications – which is a common misconception.

Update sets of the past contain versions of application files (some in global some in scope). These application files may be refactored to become part of different applications. As a result, the update sets that have been deployed in the past are irrelevant for the refactoring of the contained application files into applications.

For the refactoring there are two extreme approaches:

Big Bang - refactor everything at once

In a huge, labor intensive, big-bang initiative everything that has ever been modified, customized, changed is moved and refactored into new to be defined applications. This process may take several weeks or months to complete. Any new feature development and defect correction comes to a halt as all developers are focusing on the refactoring.

The advantage is obvious: at some point in time, everything is contained in applications – a consistent deployment process can be used and CodeSanity checks can always be applied. There are no uncertainties how specific application files are to be shipped: is this specific part already refactored; does it already have a place in one of the new applications? Is there an application already created for it? Everyone can follow the same process. The bonus: new instances can be set up in almost no time by installing relevant application versions from source control.

The disadvantage is even more obvious: the business will not get any new features for that period, and the cost of the refactoring becomes obvious – which may motivate some stakeholders to put the whole initiative in question.

Continuous Improvement – accept the current state as a baseline and refactor on change

The current state is accepted as it is – with all its customizations and modification - as a baseline. Whenever new features are being built or existing application files are modified, they must be refactored into new or existing applications. This will distribute the refactoring effort over many backlog items (“stories”) over a long period of time – making it less tangible and not measurable. Whether this is an advantage or not really depends on the organization.

The disadvantage is that the velocity – no matter if properly measured – will go down. Product Owners will sense that the development team produces less output than before. The risk is that the refactoring may become subject to negotiation between the Product Owners and the development team – and compromises will be made. This will impair the positive effect of

adopting the new process in the first place and discourage the organization to carry on with its implementation.

Both extremes yield unfavorable side-effects – so the ideal approach is probably somewhere in the middle.

Some processes, some capabilities – maybe those which have seen less customizations so far, may be refactored at once in a (set of smaller) big bangs.

Others – often the more complex ones – where the application architecture design may also be more difficult – start with the continuous improvement approach. This will then also provide more clarity on how to split these more complex capabilities into applications leading to even better decisions on the long run.

There is no one-size-fits-all approach. Any initial architecture design will have to adapt and change over time anyway – teams change, requirements change, the organization learns what works well and what not.

For some organizations the introduction of the new process may be combined with a “back-to-baseline” initiative – where decisions must be made which of the existing customizations should be transferred to the new instance and which should not – which then provides guidance on which parts should be refactored into applications, and which should not.

Developer Onboarding

Being a member of the team while the transition is happening may be challenging, interesting, or even inspiring. A developer that experiences the introduction of this process and its techniques from the beginning will have the time to learn and adapt to the new normal and most importantly they do so together with their peers.

A new developer joining the team after the transition has been made, however finds themselves in a different situation.

They do not only need to adapt to their new role, the team, the stakeholders, the context, the client's business domain but also must unlearn many things they know about how to do their thing in ServiceNow and instead learn new techniques.

This can – and in many cases will be – troubling and hence new team members need special attention. A defined and thoughtful onboarding protocol may come to the rescue.

The following list contains some ideas on where to start – many are independent of the recommendations in this document – but these are included anyway:

- Read this document – or a tailored version edited with the specifics of the organization
- Verify all needed access is granted
- Verify the developer is added to all relevant mail distribution lists and other communication channels
- Verify the developer is invited to all relevant meetings
- Become a ServiceNow Certified System Administrator
- Become a ServiceNow Certified Application Developer
- Review the Definition of Ready
- Review the Definition of Done
- Review the Application Architecture Overview
- Review the Lighthouse Story (see the Definition of Ready, Definition of Done and the Architecture Principles implemented as a comprehensible example)
- Review the Instance Setup
- Review the Application Baseline procedure
- Review the Coding Guideline
- Create their own “lab” application on the dev environment, implement a small feature and make their application pass all automated QA checks (including ATF and Instance Scan checks)

This list is certainly not complete, and each team and organization may have its own take and perspective on it. The most important aspect is that a new team member joining may face a bigger challenge to get up-to-speed than before the transition – as many technical and procedural aspects are very different from what they have experienced in earlier ServiceNow-related projects they have worked in.

Communities of Practice

Communities of practice are groups of people discussing options and making decisions on various topics. Consider establishing communities of practice (also sometimes referred to as "Chapters") for the following topics:

- Deployment and Operations ("CICD")
- Development / Code Guideline / Automated Testing
- Architecture
- Backlog Management / Agile Practices
- Manual Testing / Test Management

In the early days of a ServiceNow implementation, especially when the team is yet small, it is perfectly reasonable to combine these topics into a single community of practice – as the team grows it makes perfect sense to establish separate communities of practice for the different topics.

For larger development teams it makes sense to start with the "Deployment" Community of Practice as a forum to organize the Proof-Of-Concept and Pilot phases.

Each community of practice should meet at least once, better twice a week. The community of practice is a decision-making body, so the meeting cadence must allow us to respond to urgent matters and questions and must be able to provide guidance and decisions on short notice.

Introducing the process proposed in this document is demanding. People need to learn new skills and adopt new ways of working. Communities of Practice are designed to make sure everyone has a say in how exactly this transition should be designed. There must be a place where concerns, objections, sometimes even fears can be openly expressed.

These Communities of Practice meetings should not be confused or mixed with the typical rituals of agile methods. These meetings should focus on how things are done, not to discuss new requirements or to review the completed work.

The topics being discussed should be as relevant to the complete audience as possible. If topics are discussed in detail, especially in a virtual meeting environment, many people may mentally walk away and perceive such meetings as boring and irrelevant. Moderating Community of Practice meetings is hence about maintaining a balance between going into details but also providing answers and decisions to specific questions.

The results and decisions made by the Communities of Practice should be respected and defended by management. Empowerment is key to the successful implementation of the process.

Moderating Community of Practice Meetings

Each community of practice is moderated by a facilitator who sets up the meeting, leads the sessions, and documents the decisions and outcomes.

The following techniques worked well for many teams the author worked with:

Documentation of the community of practice on one single wiki page containing

1. "Waiting For"
The "Waiting For" list contains all topics that are work in progress and assigned to someone. "Waiting For" means that the community of practice is waiting for someone to come back with a result to make decisions
2. "Next Action"
The "Next Action" list is a list of topics to be discussed by the community of practice. Everyone can add new topics to the end of the list
3. "Results and Decisions"
Results and decisions are documented - with the latest entry on the top of the list. This helps everyone who may have missed a session to catch up.
When added the date of the meeting should be added in front of the result item.

Invite everyone who may be interested in the topic

Participation is strongly recommended but not mandatory

Apply democratic processes in making decisions

Let everyone be heard – especially the less "aggressive" speakers

The facilitator is well advised to follow the following agenda in all sessions

1. Opening the session, welcome participants.
2. Ask for urgent topics that should be put to the top of the "Next Action" list. Those entries are marked in red color.
3. Ask for status of "Waiting For" topics. If any of the topics are done, they should be moved to "Results and Decisions". If further discussion is required, it should be put after any identified urgent topics in the "Next Action" list.
4. Discuss topics from the "Next Action" list from top to bottom. Document all results in the "Results and Decisions" section or move topics to "Waiting For" if someone is assigned a task as a result. All topics moved to "Waiting For" must be assigned to an individual person – otherwise the community will wait forever.

How to involve the less "aggressive" speakers?

It is difficult to get everyone's opinion heard – especially in a virtual meeting environment. In most teams there is a loud minority and a less vocal majority. This inherently causes a bias in the decision making. There are numerous techniques to involve participants that are less vocal:

- (Virtual) polls
- (Virtual) boards with sticky notes
- Breakout rooms

Proof-Of-Concept – Pilot – Production Readiness

Before going all-in organizations need to tip their toes into the cold water. One approach is to start with a rather technical **Proof-Of-Concept** with dummy applications to demonstrate the deployment process. Then to choose a real feature that exists in production or is about to be delivered to production as a **Pilot** – to demonstrate that the automated pipeline works end-to-end supported by a small team of developers. Those developers can then act as multipliers when educating and coaching the wider development community – or even as Facilitators (see chapter “Education Program”).

Finally, **Production Readiness** aims at involving the whole team and ship everything as an app automatically – and to establish the new process as robust long-term practice.

The chapter “Backlog” provides a detailed task list for each of the three phases.

The goals of the three phases are:

Proof-Of-Concept

1. To test the technical setup and establish access to the Source Control System and provide a technical proof that it works
2. Create confidence with technical and business stakeholders

Pilot

1. Validate the approach end-to-end
2. Create confidence that the process works and will provide the intended benefits
3. Gain experience and spread know-how within the team

Production Readiness

1. Enable the whole team
2. Optimize and establish this new process

While the number of people involved may yet be small during the **PoC** phase, it will grow during the **Pilot** phase and in the **Production Readiness** phase everyone will be involved.

This allows the Process Evangelist, the management stakeholders and technical decision makers to adjust their approach and communication – and to carefully manage objections and concerns.

Some organizations may want to rush and try to get everything at once – and yet skip these phases. A strong advice must be given to slow down. This transition is not about technology; it is about people.

Backlog

The following list can serve as the foundation for a backlog for the project to introduce the process described in this document.

It is partitioned into the "Proof-of-Concept", "Pilot" and "Production Readiness" phase:

Proof of Concept

1. Setup of the Community of Practice meeting for "Deployments" or ("CI/CD")
2. Choose instances on which the proof-of-concept implementation can be done (usually DEV, TEST or SANDBOX instances)
3. Setup the PoC Development instance – usually the DEV instance
 - a. Install DevTools
 - b. Install CodeSanity
 - c. Install Deployer
4. Setup the PoC Controller instance (that coordinates the deployment) – usually the DEV instance
 - a. Install DevTools
 - b. Install Deployer
5. Setup the PoC target instance (the instance to which the PoC apps are deployed) – usually a TEST instance
 - a. Install DevTools
 - b. Install CodeSanity
 - c. Install Deployer
6. Create at least two PoC apps which have dependencies to each other
7. Establish access and set up the Source Control System
8. Setup repositories for the PoC apps and link the PoC apps to their repositories
9. Baseline and deploy PoC apps to target instances – leveraging the dependency support in the Deployer app
10. Make changes to the PoC apps
11. Baseline new versions of the PoC apps
12. Deploy the PoC apps again – to demonstrate the version updates caused by updated dependency version requirements

Pilot

1. Define a preliminary Application Architecture
2. Choose an existing or new application as the pilot application – the ideal candidate is an existing app that is not undergoing significant changes during the pilot phase
3. Recruit a pilot team of developers to work on the pilot
4. Prepare the selected pilot application (fix all Instance Scan findings, add ATF tests, refactor existing customizations outside the app into the app)
5. Define the instance setup
6. Set up actual instances (from DEV to PROD) according to the defined instance setup
7. Baseline and deploy the pilot app to PROD using the automated pipeline but following all regular procedures that exist in the organization – the technical deployment method may be different – the existing governance rigor however must be respected

Production Readiness

1. Educate the whole team about the process as a whole and its building blocks
2. Assign the different gatekeeper roles to individuals
3. Educate all gatekeepers on their specific duties and responsibilities
4. Assign the various governance stakeholder roles to individuals
5. Educate all governance stakeholders on their specific duties and responsibilities
6. Define the Backlog Management process
7. Configure the Backlog management System
(e.g., by installing and configuring the "Agile" App or configuring an existing Jira instance available in the organization or by using the ServiceNow Agile plugin)
8. Assign gatekeepers for each quality gate of the Backlog Management process
9. Setup the Community of Practice for Backlog Management (which decides about changes to the Backlog Management process)
10. Educate the team about the Backlog Management process
11. Set up and structure the team's Knowledge Management System (e.g., Confluence)
12. Create and publish documentation on abbreviations
13. Create and publish documentation on terminology / glossary
14. Educate the whole team about Application Architecture and Dependency Management
15. Implement Dependency Management (e.g., by installing the "DevTools" app)
16. Set up the Community of Practice for Architecture (which decides on changes and extensions of the application architecture)
17. Define, agree on, and publish the Definition of Ready
18. Define, agree on, and publish the Definition of Done
19. Define and publish the Application Architecture
20. Inform the team about the Application Architecture
21. Create the "Platform" app to act as the root node of the dependency tree
22. Setup the Community of Practice for Development (which decides on changes to the Coding Guideline) – it should be a recurring meeting (1-2 times a week) where all developers and architects are invited
23. Define a process for the Community of Practice for Development to make decisions if there is no consensus in the team
(e.g., by democratic vote by majority or by delegation to the Community of Practice for Architecture)
24. Define, agree on, and publish the first version of the Coding Guideline
25. Implement automated checks to verify applications against the Coding Guideline
(e.g., by installing the CodeSanity app)
26. Setup a "Nightly QA" scheduled job to run all ATF tests and scan all applications against the Coding Guideline based on the dependency tree of the "Platform" app.
27. Educate the development team on Automated Testing
28. Educate the development team about Installation Scripts
29. Define and document the Branching Strategy
30. Educate the development team about the Branching Strategy
31. Educate the development team about Semantic Versioning

32. Define and publish the Application Version Baseline Procedure
33. Educate the team about the Application Version Baseline Procedure
34. Define and publish the Deployment procedure
35. Educate the responsible team about the Deployment procedure
36. Implement Deployment Automation (e.g., by installing the "Deployer" app)
37. Define and publish the process for Technical Debt Management
38. Educate the development team about Technical Debt Management
39. Define and publish the process for Hotfixing and Backporting
40. Educate the development team about Hotfixing and Backporting
41. Educate the development team about their obligation to write Release Notes
42. Define and publish the onboarding curriculum for new team members
43. Define and implement the Lighthouse Story
44. Present the Lighthouse Story to the whole team
45. Create a Runbook
(e.g., by installing the "Runbook" app)
46. Establish data quality checks in production
(e.g., by installing the "DataSanity" app)
47. Establish a process to handle findings from data quality checks in production
48. Identify personas
49. Create the Persona Access Matrix
50. Apply the persona role concept

Education Program

Teams are often diverse with regards to their knowledge and experience on the platform – for many people many of the recommendations in this document are new.

An education program may be required to bring the whole development team up-to-speed and enable everyone to work effectively in the proposed process.

Note that the Developer Onboarding checklist aims at the same goal, but for those joining the team later.

People learn in different ways and no one-size-fits all approach will yield consistent results for everyone.

The following education program is designed to

- address different learning styles
- apply self-organized learning principles
- be as little time-consuming for every participant as possible
- work for small and large teams alike

One or more Facilitators are responsible to organize the process. They act as organizers and stewards to help the team perform their learning activities:

- Attendance at classroom presentations performed by subject matter experts
- Individual and collaborative hands-on work on “Masterpieces” (apps to be built by the participants)
- Group work on a demo or presentation
- Presenting content to other group members
- Presenting the implemented “Masterpieces” to another group
- Engaging in conversations with other group members and the wider community

Introduction Session Topics

1. Introduction / Goals / Aims
2. Process Overview / Finding a common language
3. Education Program Overview and Logistics
4. Introduction to the Whitepaper and other learning material
5. Process Quality Gates, Process and Governance Roles
6. Masterpieces – Apps to be built by the participants

5 Expert Topics

1. Application Architecture and Dependencies
2. Coding Guideline (using Instance Scan and the CodeSanity Scan Suite)
3. Automated Testing (using ATF)
4. Source Control, Baselining and Deployments
5. Refactoring existing Customizations, Installation Scripts, Code vs. Data and Record Transportation

High-Level Track

Non-developer roles (Scrum Masters, Product Owners, maybe Architects etc.) only attend the Education Program Assembly meetings – and hear the introduction sessions and high-level classroom presentations of the 5 expert topics. They may continue to attend when the series evolves into the Community of Practice meetings – but this is optional.

Procedure

The education program follows this procedure:

1. All participants are informed about the program, the goals, the introduction session topics and the 5 expert topics
2. The participants are split into study groups of at least 5 members
3. Within each study groups each member picks one out of 5 “Masterpieces” – each of the 5 “Masterpieces” must be worked on by each study group
4. Each participant starts working on their “Masterpiece” individually and starts collaborating where required
5. Subject matter experts provide high-level classroom presentations to all participants on the 5 expert topics
6. Within each study group each team member picks an expert topic and becomes the expert for the topic within the study group – each of the 5 expert topics must be covered
7. Experts form expert groups and work on a demo or presentation
8. Experts present their topic to their study groups
9. Study Groups prepare the presentation of their “Masterpieces”
10. Each study group presents their “Masterpieces” to one other study group
11. Wrap-up session with all participants

Preparations

The following preparations are required:

1. Setup an instance on which participants can work on their “Masterpiece”
2. Distribute participants into study groups (with at least 5 members)
3. Setup the Education Program Assembly Meeting series (for introductions, classroom presentations to the full audience and Q&A) – this meeting series can then evolve into the Community of Practice meeting series
4. Set up the study group meetings
5. Set up the expert group meetings
6. Setup the “Masterpiece” presentation meetings (which involve two study groups each)

Masterpieces

The “Masterpieces” are apps that are to be built by the participants. This is an individual work assignment for each participant of the program. However, for some requirements participants need to collaborate.

Acceptance Criteria for all “Masterpieces”:

1. The app scope must be prefixed by the customer specific prefix, then followed by “mp_”, then followed by the first three letter of the participants first name and the first three letters of the participants’ last name. The app display name consists of the study group name (chosen by the study group) and the name of the app (“Avengers Alpha”, “Elephants Beta”, “Team Red Gamma” etc.)
2. The app must not produce any CodeSanity findings when scanned
3. All ATF tests contained in the app must pass
4. The app must contain a “manual” UI Page
5. The app must contain a Script Include that contains a single function which performs a mathematical calculation

Acceptance Criteria for “Masterpiece App Alpha”

1. The app depends on app “Beta” and “Gamma”
2. The app defines a persona role that contains at least one OOTB role and the role defined in app “Epsilon”
3. The app contains a UI-based ATF that validates that a user with the persona role uses the view defined in app “Beta” when opening the form of the table defined in app “Delta”

Acceptance Criteria for “Masterpiece App Beta”

1. The app depends on app “Delta”
2. The app contains a form and list view for the table defined in app “Delta”
3. The app contains a View Rule that enforces the view for users with the role defined in “Epsilon” when viewing the table defined in “Delta”

Acceptance Criteria for “Masterpiece App Gamma”

1. The app depends on app “Delta”
2. The app contains a “update” business rule on the table defined in app “Delta” that uses a function from app “Epsilon”

Acceptance Criteria for “Masterpiece App Delta”

1. The app depends on app “Epsilon”
2. The app defines a table and ACLs that allows all access to the role defined in “Epsilon”

Acceptance Criteria for “Masterpiece App Epsilon”

1. The app defines a role
2. The app defines an Instance Scan check that validates “Masterpiece” application scope names and display names. The Instance Scan check is part of the “CodeSanity” suite.

Daily Stand-Up Meetings

Setting up daily stand-up meetings is a common practice when initiating project teams or working groups.

DON'T

Agile has come a long way and has changed a lot over the years. From being a way to protect software development teams from the rigid organizations they are embedded in and to foster innovation it often turned into the re-incarnation of a sweat shop.

Amongst many other practices that were introduced with good intentions, Daily stand-up meetings become one of the most hated and debated routines of the "Agile" toolbox.

The recommendation clearly is not to establish daily stand-up meetings. Because:

- They do not yield any value
- They eat up time to do productive work
- They annoy people
- They are a genuine expression of distrust
- They are a manifestation of the traditional command and control approach

In most professional environments, individuals are part of multiple teams, work in different customer assignments, are part of multiple work groups or initiatives. Many have a considerable number of recurring meetings already – which often are not well conducted and are often perceived as not relevant, boring and time-consuming already.

If daily stand-up meetings come on top, frustration is guaranteed.

Most daily stand-up meetings follow the pattern of everyone giving a short summary of what they did, are doing, are planning to do, and where they are blocked. Or sometimes, the most vocal individuals just capture and dominate the meeting to discuss their special interest topic in detail.

One may argue that if led well, daily stand-up meetings would be more productive, informative and less annoying – but the reality is that they often are not well run.

And even if they were, the simple math of limited available work hours in everyone's calendar demand for a better way to distribute information, share concerns, discuss details and collaborate.

The practice of Community of Practice meetings – as described in this document – is a better, more formal way of running productive meetings. The frequency for any recurring meeting should not exceed two meetings per week to allow people to work on the agreed tasks and the next steps. There is no value in meeting everyday reporting that there is no progress (often due to too many meetings).

An important aspect is to identify the need for detailed discussion and delegate these to smaller groups – to avoid forcing a larger group to witness a detail discussion of a smaller group.

Some even establish focus days (days without meetings) every week to do (implementation) work because there is no chance to get into any sort of productive flow on days that are packed with meetings – and that concept is inherently incompatible with daily stand-up meetings.

Platform Capabilities

Classic Studio

The “Studio” is available on all instances – no additional license required. The Studio provides an IDE-like user interface and surfaces many (although unfortunately not all) application files contained in a scoped or global application – as well as app customizations.

As of the release of this document Classic Studio has been deprecated by ServiceNow – but is yet widely used by many developers.

Source Control Integration (Studio)

In Studio, applications can be linked with a source control repository. On each instance an application may be linked to exactly one branch in the repository at any given time. Studio allows users to commit changes into a repository or apply changes from that repository back into the instance. The current branch can be switched, and new branches can be created directly on the platform.

Such source control operations can also be triggered through OOTB REST API endpoints.

Automated Testing Framework

The Automated Testing Framework is an OOTB platform capability that allows creating and running automated tests. Tests consist of test steps which simulate user interaction or any other code execution on the platform. Each test step can conclude with a success or failure result. If the latter, the test is considered failed. Any data changes conducted during a test are rolled back after the test is completed.

Tests are organized in Test Suites. One Test can be part of multiple Test Suites.

All tests, their steps and Test Suites are application files and hence can be part of a scoped or global application.

Data changes during a test are rolled back, but the effects of external systems to which the code executed during a test is not. This means that mail is really sent out, data is really sent to external systems, requests to external systems are really logged.

This remains a consideration to not run ATF tests in production instances – unless any external communication is marked as for testing purposes and ALL external systems understand these markers.

Imagine a test simulating a major incident – and part of the major incident management process may be to inform all users of an organization via mail. One wants to know if this process works properly – but one doesn't want to bug tens of thousands of users as part of such a test.

Instance Scan

Instance Scan is an OOTB platform capability that uses so-called checks to investigate a defined set of records and based on the configuration or code contained in the check produces a finding.

Findings are produced as the result of a scan – which can be run against an application, an update set, or a single file.

One or more checks can be organized as a scan suite. It is recommended to create a suite that contains all checks relevant to the Coding Guideline.

Instance Scan is shipped with a limited set of checks – which mainly serve to produce security configuration insights – OOTB there are no checks that assess code or application quality.

A finding is the binary result of a check executed against a single record – by itself a finding does not mean anything. It is neither good nor bad. It depends on its interpretation.

One could use checks to count records that match a defined criteria and – based on thresholds – recommend any kind of action.

In the context of this process, Instance Scan is used as a static code analysis tool, and the superset of related checks constitutes the coding guideline. A finding in this context indicates non-compliance with the coding guideline and must be addressed before the affected application can be deployed further downstream.

In the context of data quality management, security management and the implementation of the persona roles concept Instance Scan checks can help to validate data records, role assignments and security configuration in production.

Instance Scan should not be confused with Health Scan – which is a ServiceNow service offering using comparable – but not the same – technology.

Health Scan results, however, may provide input and inspiration for new checks to be created as part of the coding guideline.

Impact Health / Scan Engine

The Scan Engine emerged from the acquisition and integration of Bravium (a ServiceNow partner).

The Scan Engine serves almost the same purpose as Instance Scan. Scan Engine comes with a more sophisticated integration in the ServiceNow user interface than Instance Scan – as it scans code while the developer is still typing the feedback loop is shorter.

Scan Engine is only available to Impact customers in more expensive license models and at the time of the release of this document the number of custom checks was limited to 10.

To support the process recommended in this document Scan Engine is not fit-for-purpose – as the process relies heavily on the team's ownership of their Coding Guideline and the enforcement of the specific architectural decisions of the organization – which can only be done by customized checks – hence Instance Scan remains the tool of choice – due to its flexibility and availability to all customers.

Impact Health / Proactive Code Check

As of the Yokohama release ServiceNow provides the "Impact Health" plugin to Impact customers. It contains an Instance Scan Suite of 60+ Instance Scan Checks that aim at flagging bad coding and configuration patterns.

The guidance in this document includes that a team should own the Coding Guideline – and the Coding Guideline should be implemented as Instance Scan Checks. The Checks contained in the Proactive Code Check Scan Suite can be integrated to the team's own Scan Suite to further improve code quality.

The Scans contained in the Impact Health plugin are set read only and their source code cannot be investigated – which means that if the team wants to modify one of the Checks, it must completely re-engineer the Check.

At the time of the release of this document, this capability is deprecated and replaced by Scan Engine.

Build Agent

The Build Agent is ServiceNow's answer to the idea of prompt-based coding – where a user just describes what they want to build and AI is used to produce the code.

The capabilities of the Build Agent are growing and expanding over time, and it is to be expected that more implementations will be made fully or partly by AI using Build Agent – at a higher pace than today.

If not for any other reason, Build Agent should be the final argument why maturing an organization's development and deployment process is critically important.

ServiceNow Studio

The "ServiceNow Studio" is introduced as a free-of-charge store application with Xanadu. It aims to replace the classic Studio as a one-stop shop for app development on the platform while combining and integrating pro-code and low-code/no-code tools.

ServiceNow IDE

The "ServiceNow IDE" is introduced as a free-of-charge store application since Xanadu. It allows development of (new) applications using the Fluent language and the ServiceNow SDK directly on the platform.

Sandboxes

As of the release of this document version "Sandboxes" are currently introduced. Sandboxes are "copies" of an instance that can be used to work on changes individually or in smaller groups in isolation. Sandboxes can be created and deleted ad-hoc. They allow developers to make changes to the same application files independently that may result in conflicting changes – which may not be straight-forward to resolve. They do however allow developers to perform isolated experiments without having to "undo" them on a shared instance.

Some capabilities are limited on Sandboxes – e.g. developing integrations is limited as sandbox instances use separate subdomains.

It is yet a very promising addition to a development team's toolbox.

Open-Source Applications

The tools mentioned below are open-source contributions. ServiceNow does not provide any support or warranty. They have been developed based on the insights gained when implementing the described process with clients – and hence might help to shortcut some of the technical implementations required to fully implement the automated QA and deployment procedures described in this document. It is at the customer's discretion to either use these tools as they are – at their own risk – or use them as inspiration on how to implement their own technical solutions.

DevTools

The DevTools application contains hundreds of re-usable functions and classes that extend or encapsulate Glide API or JavaScript features. DevTools has the tools required for cross scope scripting techniques and installation scripts. DevTools also contains an implementation of Dependency and Technical Debt management.

<https://github.com/saschawildgrube/servicenow-devtools>

Deployer

The Deployer application implements full pipeline automation from installing applications from a source control system to running automated tests.

The Deployer application also allows us to verify if an application version that is baselined and stored in a branch in a Source Control System is ready for deployment by looking into all its dependencies.

<https://github.com/saschawildgrube/servicenow-deployer>

CodeSanity

The CodeSanity application contains several Instance Scan checks to validate source and application files.

<https://github.com/saschawildgrube/servicenow-codesanity>

DataSanity

The DataSanity application contains several Instance Scan checks to validate data records (in production).

<https://github.com/saschawildgrube/servicenow-datasanity>

Runbook

The Runbook application can be used to render the operations manual (runbook) for an instance based on the actual configuration.

<https://github.com/saschawildgrube/servicenow-runbook>

Agile

The Agile application is a lightweight backlog management system that supports the ideal state flow to support this process. You can manage backlog items in multiple backlogs, components to be worked on and affected personas.

<https://github.com/saschawildgrube/servicenow-agile>

ArtificialDeveloper

The ArtificialDeveloper performs "Test-Driven Code Generation" (also referred to as "Test-Driven Generation" or "TDG") - a human developer writes the test and GenAI writes the code to fit the test.

The ArtificialDeveloper app is the world's first enterprise production ready implementation of the Test-Driven Code Generation pattern - on the ServiceNow platform.

<https://github.com/saschawildgrube/servicenow-ad>

R-IDE

The R-IDE (React-based Integrated Development Environment) is a custom application based on the React framework that is an alternative for (ServiceNow) Studio built for pro-coders.

<https://github.com/saschawildgrube/servicenow-ride>

Installing or Updating open-source Applications from GitHub

Follow these steps to install or update applications from open-source repositories hosted in GitHub (github.com). Some steps are only required if the target instance is “air-gapped” and no direct (or mid server relayed) connection to GitHub is possible.

1. If not done already create a GitHub account.
2. Fork the most current state of the open-source application's public repository (you may have to delete an earlier fork).
3. If not done already create an API token with read access to your GitHub account
4. If the target instance is NOT air-gapped
 - a. If the application is NOT yet installed on the target instance
 - i. Configure the GitHub API access token as a basic auth credential on the target instance
 - ii. Navigate to Studio
 - iii. Choose “Import from Source Control” and load the application from the forked repository
 - iv. Navigate to /sys_repo_config_list.do
 - v. Delete the corresponding repository configuration record
 - vi. If no repository in the organization's source control system exists
 1. Create the repository in the organization's source control system
 - vii. If no API access token exists that allows access to the repository, create one and set it up as a basic auth credential on the target instance
 - viii. Navigate to Studio
 - ix. Choose “Link to source control” and use the repository url and credentials relating to the organization's source control system. Choose “dev” as the branch name.
 - x. Make the “dev” branch the default branch.
 - xi. Delete all other branches
 - b. If the application is ALREADY installed on the target instance
 - i. Navigate to /sys_repo_config_list.do
 - ii. Delete the sys_repo_config record of the application
 - iii. Navigate to Studio
 - iv. Select the application
 - v. Select “Link to Source Control”
 - vi. Select the forked repository, the credential that contains the API key with access to the forked GitHub repository and choose “temp” as the branch name – this will create the branch “temp” based on the current state on the instance

- vii. Select "Switch Branch" and choose "master" (or the specific version)
 - viii. Delete the "temp" branch
 - ix. Navigate to /sys_repo_config_list.do
 - x. Delete the sys_repo_config record of the application (again)
 - xi. Navigate to Studio
 - xii. Select the application
 - xiii. Select "Link to Source Control" (again)
 - xiv. Select the repository in the organization's source control system, the credential that contains the API key with access to the organization's source control repository and choose "dev" as the branch name – this will update the existing "dev" branch based on the new state on the instance
5. If the target instance is air-gapped
- a. Create a developer instance on the ServiceNow developer portal
 - b. Configure the GitHub API access token as a basic auth credential on the developer instance
 - c. Import the open-source application from source control
 - d. Publish the application to an update set
 - e. Export the Update Set to XML
 - f. Transfer the XML file to a location from where it can be uploaded to the target instance
 - g. Import the XML Update Set file to the target instance
 - h. Preview the loaded Update Set
 - i. In case of issues, choose "Accept remote update" in all cases
 - j. Commit the Update Set
 - k. Ignore all errors
 - l. If the application is NOT yet linked to a source control repository in the organization's source control system
 - i. If no repository in the organization's source control system exists
 - 1. Create the repository in the organization's source control system
 - ii. If no API access token exists that allows access to the repository, create one and set it up as a basic auth credential on the target instance
 - iii. Navigate to Studio
 - iv. Choose "Link to source control" and use the repository url and credentials relating to the organization's source control system. Choose "dev" as the branch name.
 - v. Make the "dev" branch the default branch.
 - vi. Delete all other branches.

6. Baseline the application
 - a. Commit all uncommitted changes to the "dev" branch.
 - b. Create branch and name the branch exactly like the version of the application (e.g., "1.5.0").
 - c. Switch back to the "dev" branch.
 - d. Lock the new branch in the source control system

Maintaining Open-Source Applications

A concern that is raised often is how to maintain open-source applications on the long run. The use of software from the public domain usually comes without license costs but that does not mean the cost of ownership is zero.

In most cases – as with the open-source applications referenced in this document – the software comes without any warranty or support-plan options. If an organization chooses to use them, it owns them – and hence must take full responsibility for future maintenance.

This may appear like a disadvantage at first sight. However, when compared to the next viable alternative – to build everything from scratch – the perspective may shift.

An organization can use the existing software, provide their teams with some time (effort) to learn how they work, integrate available documentation and establish a protocol on what to do in case of identified defects or new requirements.

Or an organization can build the required capabilities using in-house teams or partners. In theory, the effort to learn the software is not needed in this scenario. Individuals and teams that build software may not always be the ones who maintain them on the long run. Personnel fluctuations drain any knowledge that may have been gathered during implementation anyway.

The advantage of building software in-house in terms of knowledge is an illusion. Person-to-person handovers only work on project plans – not in reality.

The only known solution to this problem resides in fit-for-purpose documentation, established processes and a mindset of curiosity and learning.

The following maintenance protocol may serve as a basis of an organization's process to be used if new requirements emerge or a defect is found in an open-source application:

1. A new defect or requirement is identified and a change in one or more open-source applications is likely to be required
2. Investigate if a newer version of the open-source application has been made available by the original author
 - a. If yes
 - i. Review the release notes and check if the assumed change has already been made
 - ii. If yes
 1. Update the open-source application from the original source
 2. Check if the new version resolves the defect or fulfills the new requirement
3. Reach out to the original author. Does the original author respond and indicates their willingness to support?
 - a. If yes
 - i. Wait for a to-be-defined period and see if a newer version is published by the original author, then return to step 2

4. Reach out to the community (e.g. via the ServiceNow developer community, GitHub or LinkedIn). As multiple organizations may not only use open-source software but may also have experienced comparable challenges, someone else may already have found a solution and is willing to share it.
5. Identify an implementation approach for changes to be made to the open-source application. Can these changes be made OUTSIDE the open-source application – i.e. without changing any application files in the open-source application scope? (This may also be done through installation scripts that modify the open-source application).
 - a. If yes
 - i. Implement changes with full backward compatibility and without “forking” the existing open-source application – this maintains compatibility to future versions
 - b. If not
 - i. Retry step 3 – and contact the original author
 - ii. Get approval by all relevant governance stakeholders to “fork” the open-source application as this implies future backporting efforts if a future version published by the original author is ever to be installed
 - iii. Write specific documentation on the changes made including the specific reasons for the change and the decision-making process
 - iv. Add hints to the documentation (and ideally the instance itself) that shall prevent unaware developers from installing a future version of the open-source application without considering the backporting requirements
 - v. Implement the change

Risk Assessment

The consideration whether open-source contributions may or may not be made part of an organization's code base is indeed not trivial. Who will maintain it? Who is liable if something is broken? And what about the additional risk of a supply chain attack?

The guidance documented in the whitepaper works with or without using any of the open-source apps referenced – but needless to say – to make it work without them would require a substantial development investment to reach feature parity. The open-source apps represent the work of multiple years now – and have been field tested by dozens of teams.

And even if an organization would decide to rebuild all required capabilities – can developers be trusted not to copy large parts from public sources? Reconstructing the existing open-source apps into the organization's code base may be tempting as a short cut.

A supply chain attack through infecting one or more public repositories with malicious code could potentially allow an attacker to inject a security backdoor and then use it to infiltrate the system or to extract data from it.

Of course, a public GitHub repository cannot be considered a trusted source per se – even if it is maintained by a “trusted” individual. The author of the referenced open-source apps currently does not allow any external contributions into the repositories directly, but there is no guarantee that this may not change in the future.

Any code that is “copied” into the organization's code base must be subject to rigorous security review, tests and validation.

In fact, that rigor should apply to ALL code produced and maintained by an organization.

One needs to keep in mind that a supply chain attack could also be performed by a developer working directly for the organization – as a contractor or employee. The “rouge employee” as a thread scenario is not more or less likely than the “external hacker”.

This means that zero-trust should be applied to all code, no matter if it was produced by a third party – trusted or not – or by an agent of the organization.

As a conclusion, if no code is trusted, and hence tested with rigor, the risk of a supply chain attack is managed no matter the origin – and in that scenario, using open-source contributions does not imply a higher risk than in-house developed code.

The recent famous supply chain attacks were severe not because the repositories were infected in the first place – they were severe because many development teams connected their code bases with the “latest” and not explicit versions of the infected code.

This allowed the infected code to become part of many code bases automatically and without further review. I do not suggest that most development teams would have identified the security problem when updating their dependencies manually – but the likelihood of infection before the public identification of the security problem would be way lower.

When updated versions of open-source apps become available, an organization should update their own repositories explicitly – and execute all security checks deemed appropriate. That reduces the risk of a supply chain attack to the risk imposed by the “rogue employee”.

Memorandum of Understanding

Some organizations welcome the idea of using open-source contributions, others are more reluctant. If you are consulting an organization that uses ServiceNow and the use of open-source apps is considered, the following text could be used as a Memorandum of Understanding that can be confirmed by the organization. This is to verify that the decision makers are aware of and confirm the use of open-source software and understand the implications for their organization.

___CUT___

We understand that some aspects of the mature development and deployment process deviate from official ServiceNow guidance (e.g. to ship everything as custom applications or to copy OOTB records instead of modifying the original). While we believe that these deviations contribute to a more robust process, we anticipate that these deviations may also be surprising to consultants and developers who are joining our team in the future. We are aware of the additional onboarding and training effort this may involve.

To implement the mature development and deployment process various technical capabilities are required which are not available out-of-the-box in the current ServiceNow release.

Some of these capabilities are implemented as custom applications which are published as open-source contributions – e.g. DevTools, Deployer, CodeSanity, DataSanity, Agile, DataSilos, Reactor, R-IDE, ArtificialDeveloper and others.

We hereby confirm that to implement the required capabilities such open-source contributions are installed and configured in our organization's ServiceNow instances.

We understand that neither the authors of such open-source contributions nor ServiceNow provide any support or warranty for these contributions and cannot be held liable for any damages that result from their use. We explicitly understand that this is also the case if such authors are current or former employees of ServiceNow.

These open-source contributions are integrated into our organization's ServiceNow instance and code base. We understand that these open-source contributions must then be maintained and supported by our organization itself – which requires initial and ongoing knowledge transfer and training – in the same way as our organization would support software developed by our organization itself or by contractors on behalf of our organization.

We use it. We own it.

___CUT___

Undo Open-Source

There might be various reasons why an organization may decide to change course and walk away from making use of the open-source applications mentioned in this document.

A lack of skills in the broader team, insufficient support by the management for the required change process, time pressure, word of mouth, pushbacks from suppliers, or maybe the people that drive the process in the beginning have made different career choices.

Whatever the reason might be, to “undo” the open-source apps, a plan is required to get there.

This chapter is written with the idea of resilience in mind – it’s good to be prepared for the unlikely, the unthinkable – and then never use that plan. I strongly discourage any organization from ever applying it.

Depending on which application is in use and for what, uninstalling them is anything between trivial and a substantial project of its own.

This chapter will not discuss “undoing” the change of mindset and the un-adoption of engineering work ethics and principles by the team. If that change has happened, it cannot be undone.

A team’s drive for improving code quality, extending test coverage, writing cleaner code, implementing the separation of code and data and for feeling real code ownership – is agnostic to tools being used anyway. I suggest NOT questioning these principles and hence will not make any suggestions to reverse them.

This is “only” to help a team to uninstall some apps:

For each of the applications the guidance described in chapter “Decommissioning Applications” should be followed with due care.

First things first: DevTools is not only a collection of re-usable scripts and helpful tools for developers, but also a dependency of all other applications – hence it must be uninstalled lastly – after all other applications have been uninstalled.

The order in which applications are uninstalled is important and directed by their dependency tree – so work on the applications in the following order:

Deployer

The Deployer can be uninstalled from target instances and the controller instance at any time.

This will take away the deployment automation capability for the team. However, everything that the Deployer does automatically can also be done manually by a user – this is, of course, tedious, time-consuming and error-prone. Before uninstalling the Deployer, alternative deployment automation tools should be evaluated and ideally integrated into the process.

CodeSanity

The CodeSanity app contains predefined Instance Scan checks. CodeSanity can be uninstalled at any time – which will remove the CodeSanity Scan Suite and the contained checks. If a team has created their own set of Checks anyway, then nothing is lost. If the team considers at least some of the contained Checks as valuable, these should be replicated and stored outside of the CodeSanity app.

CodeSanity also allows us to manage per-app exceptions for Instance Scan Checks so that fine-grained architectural decisions can be enforced by these Checks. Uninstalling CodeSanity also removed this capability.

As a result, if some of the Checks should only be applied to specific applications in the future, a comparable capability must be re-implemented.

DataSanity

The DataSanity app contains a set of Instance Scan Checks aimed at monitoring data quality in production. If these Checks are considered helpful, these can be replicated outside of the DataSanity app, before the app is uninstalled.

Agile

The Agile app is an alternative tool to manage a team's backlog. Before uninstalling it, the backlog item data should be exported and stored in a tool of choice.

RunBook

The RunBook app can be always uninstalled. This removes the capability to automatically create the operations handbook – hence this must be maintained manually going forward.

DataSilos

DataSilos adds a custom data filtering layer to the platform. Usually implement strict data protection requirements.

Technically, DataSilos can be safely deactivated via system property.

DataSilos can also be uninstalled - however, ACLs must be adapted manually after uninstalling versions prior to DataSilos 4.0.0. DataSilos 4.0.0 or higher provides a clean-up script.

If the requirements that led to the installation of DataSilos are still valid, then – before uninstalling DataSilos - a comparable solution that solves whatever data segregation requirements must be put in place.

Organizations do not choose DataSilos for efficiency gains or optimized workflows, DataSilos solves compliance challenges and legal requirements - so before considering its removal, alternatives must be implemented.

Artificial Developer

The Artificial Developer can be uninstalled without any side effects and further consideration. It is a nice add-on to boost developer productivity but does not have any production impact.

R-IDE

R-IDE can also be uninstalled without any side effects and further consideration. It is also a nice add-on to boost developer productivity but does not have any production impact.

DevTools

DevTools is a technical dependency of ALL other open-source apps. Its removal can be considered, once the last other open-source app is uninstalled.

Once all other open-source apps that depend on DevTools as a shared library are uninstalled, in theory, DevTools can be uninstalled, too.

However, it must be anticipated that the development team makes use of the re-usable functions contained in DevTools and that the code base now has numerous technical dependencies to DevTools. This is a developer's choice. To undo these dependencies, the code base must be refactored, and code must eventually be copied out of DevTools and integrated into the calling components.

Family Upgrades

Impact on Family Upgrades

Introducing the process described in this document has a huge impact on how ServiceNow platform family upgrades are handled.

Spoiler alert: it is going to be much simpler, faster, less error prone, less laborious, more reliable.

When everything is shipped as an app, and all changes to OOTB or plugin scopes are done through idempotent installation scripts, any family upgrade can be treated like a regular deployment.

Of course, a comprehensive test is mandatory. Any new behavior introduced by new or updated components must be tested against the existing applications.

Applications may have to be updated to fix these regressions and some new OOTB capabilities may have to be backported to customized versions to make use of new features in the platform.

But the laborious process of reviewing "skipped records" and the compilation of Upgrade Plans is no longer necessary.

ServiceNow has introduced great improvements to the Family Upgrade process over time – but by applying the proposed process, Family Upgrades are no longer the biggest event of the year, they turn into yet another regular deployment, nothing special, business as usual.

Procedure for Family Upgrades

Follow these steps for a Family Upgrade:

1. Upgrade development instances
 - a. Trigger the Family Upgrade
 - b. Review Skipped Records: Revert to Base System (*)
 - c. Run all Installation Scripts (e.g. to re-deactivate OOTB records)
 - d. Perform regression tests
 - e. Adapt applications to modified OOTB or plugin behavior
 - f. Re-test
 - g. Baseline adapted applications
2. Upgrade any downstream instances
 - a. Trigger the Family Upgrade
 - b. Review Skipped Records: Revert to Base System (*)
 - c. Deploy adapted application versions
 - d. Run installation scripts
 - e. Perform regression tests

(*) If really ALL customizations and changes ever made to the instance are captured in custom applications - and hence the superset of applications contains all customizations made, then Reviewing Skipped Records is simple: Select "Revert to Base System" for all of them.

Note that if some customizations are (not yet) captured in custom applications, individual decisions on skipped records are still required.

Cloning

Do you still need to clone?

The question alone might sound as if from another world. Do you still need to clone after all the guidance provided in this document is applied?

The answer is: maybe not.

If all guidance provided in this document is applied, one can tick off all the following check points:

- Everything is deployed as an app
- All historical changes to the global scope or plugin scopes have been refactored into applications and installation scripts
- All test data required for development, UAT and other manual test activities can be created automatically on demand

Once this state is reached – no cloning will ever be necessary.

Getting to a “clean” development or test instance can then be achieved by z-booting the instance, installing all applications, running all installation scripts, automatically creating all necessary test data, and performing a few manual configuration steps to set up instance specific credentials and connections – that's it.

This is, however, an ideal world scenario. Depending on the age of the instance, the level of customizations, and the technical debt that existed when the proposed process was introduced this scenario may never be reached.

So, for most organizations, cloning will still be part of their routine – although less often than before.

Activities when Cloning

Cloning comes with two challenges:

1. Defining the data that should NOT be cloned
2. Protecting unfinished work

The process described in this document does not help with challenge 1. Challenge 2 however, is much easier to master when using only applications that are stored in source-control repositories.

Instead of capturing unfinished work in Update Sets before cloning and re-applying these Update Sets after cloning, the following activities are needed:

Before Cloning (only on development instances)

1. Commit all uncommitted changes for all applications

After Cloning

2. (Re-)install all applications from source control
3. Execute all installation scripts

Maturity Model

The value of maturity models is debatable. For some such models provide good insights and are helpful tools to assess the current situation and make plans. For others maturity models are a means to shame teams or organizations for the lack of progress that has been made. The following maturity model does not represent any prescriptive guidance regarding what next steps are nor should it be used to assess progress being made. It should be used to get a high-level idea on where an organization is and where it may want to develop.

The term “maturity model” may appear a bit “passive aggressive”. An organization may “just” be on “Crawl” level and hence - according to the provided categories – appears to be “immature”. In the professional world, however, that is often not true. The same organization may already have gained great value through the ServiceNow platform, employ a team of highly skilled professionals and be an excellent steward for the interests of their customers, employees, management and other stakeholders.

Please use the following maturity model with great care and empathy!

Area	Crawl	Walk	Run	Fly
Application Architecture	No applications exist. All changes are made to the global scope, OOTB and plugin scopes.	Few applications exist but most changes are done outside applications in global or other OOTB and plugin scopes.	Applications and App customizations contains most of the code. Few changes are made in global scope, OOTB or plugin scopes. Dependencies between applications are documented and maintained.	All code is contained in applications. App customizations are NOT used. A “platform application” serves as the root node of the dependency tree which contains all relevant applications.
Deployment Method	Update Sets only	Primarily Update Sets, Source Control or AppRepo for few selected applications	Primarily Source Control or AppRepo, Update Sets are the exception	Only Source Control or AppRepo
Deployment Automation	None	Some steps are automated	All steps are automated to deploy a single application	All steps are automated to install all applications including their dependencies
Automated Testing	None	Some ATF tests exist as some developers took initiative	ATF tests exist for most capabilities. Tests are run during deployments.	ATF test coverage is controlled through Instance Scan checks. Zero-tolerance is applied to failing tests during deployment.

Coding Guideline	None	Some coding guidelines are documented in a wiki (or other knowledge management system) but are not enforced	Some code-oriented Instance Scan checks exist, are executed during deployments and part of the code is compliant.	Instance Scans form the basis of a coding guideline. All checks are applied to the complete codebase automatically during deployments. No findings are accepted.
Backlog Management	None or Excel or Jira using OOTB or one-size-fits-all configurations	The backlog management system is configured to support the process from ideation to production deployment.	"Walk" plus proper management of dependencies and resulting priorities.	"Run" plus the capability to automatically update item status based on deployments and the prevention of deployments in case of items marked as show-stoppers.
Data Quality	No data quality governance	Ad-hoc reviews of data quality in production take place from time-to-time and some findings are resolved	Some Instance Scan checks exist and are used from time to time. For some data (tables) owners or stewards are defined who act on ad-hoc findings.	Instance Scan checks are routinely executed to validate data quality in production and findings are resolved through an established process
Access and Security	No clear security requirements defined. Roles are assigned to users ad-hoc	Personas are defined but these definitions are not reflected in the technical implementation	Personas are defined and reflected in the technical implementation. But the compliant use of these roles is not yet monitored.	Full implementation of the persona role concept and permanent data quality monitoring ensures compliant assignment of persona roles to users via groups
Governance	None of the defined governance roles are assigned to individuals	Some responsibilities of some governance roles are executed by individuals or by the team ad-hoc	Some of the defined governance roles are assigned and some individuals are impacting the process according to their responsibilities	All governance roles are assigned, and the individuals are actively impacting the process by defining requirements and providing guidance
AI for development	Developers ask ChatGPT and other LLMs without guidance or governance	A policy exists on which LLM services may be used and for what	Developers have access to defined and approved AI tools within or outside the platform	Test-driven code generation is implemented and provides a safe process for code generation within the platform